



Bachelor-Thesis

Entwicklung einer Observability-Strategie für das Audience
Response System "frag.jetzt" basierend auf dem Konzept der
Four Golden Signals.

zur Erlangung des akademischen Grades

Bachelor of Science

im dualen Studiengang Informatik an der IU Internationale Hochschule

von

Leon Rausch

Matrikelnummer: 102201482

27. März 2026

Referent: Prof. Dr. Klaus Quibeldey-Cirkel

Korreferent: Prof. Dr. Philipp Diebold

Kurzfassung

Das Abstract besteht normalerweise aus einem Absatz, welcher die Hauptziele, Resultate und Schlussfolgerungen der Thesis zusammenfasst. Es ist auf Deutsch und Englisch zu verfassen; es soll ungefähr 200 Wörter (in jeder Sprache) beinhalten und insgesamt nicht über eine Seite lang sein. Des Weiteren sollten Schlüsselwörter unter diesen Absatz geschrieben werden. Schlüsselwörter sind drei bis sieben Wörter, die die Leserschaft das Thema erkennen lassen.

Abstract

Das Abstract besteht normalerweise aus einem Absatz, welcher die Hauptziele, Resultate und Schlussfolgerungen der Thesis zusammenfasst. Es ist auf Deutsch und Englisch zu verfassen; es soll ungefähr 200 Wörter (in jeder Sprache) beinhalten und insgesamt nicht über eine Seite lang sein. Des Weiteren sollten Schlüsselwörter unter diesen Absatz geschrieben werden. Schlüsselwörter sind drei bis sieben Wörter, die die Leserschaft das Thema erkennen lassen.

Danksagung

An dieser Stelle möchte ich meinen aufrichtigen Dank aussprechen.

Zuallererst danke ich meinen Betreuern **Prof. Dr. Klaus Quibeldey-Cirkel** und **Prof. Dr. Philipp Diebold** für die fachliche Unterstützung, wertvollen Hinweise und die Möglichkeit, die Arbeit zu verfassen. Ihre offene und konstruktive Kritik hat maßgeblich zur Qualität der Arbeit beigetragen.

Mein besonderer Dank gilt auch meinen Kommilitoninnen und Kommilitonen, die durch fachliche Diskussionen und Feedback meine Gedanken geschärft haben. Ihre Perspektiven haben mir geholfen, verschiedene Aspekte der Thematik tiefergehend zu beleuchten.

Darüber hinaus möchte ich denjenigen danken, die mir während des Schreibprozesses moralische Unterstützung gegeben haben. Ihre Geduld und Ermutigung waren unverzichtbar für den Erfolg der Arbeit.

Abschließend danke ich den Entwicklern und der Open-Source-Community für die Bereitstellung der vielfältigen Werkzeuge und Bibliotheken, ohne die die Arbeit nicht möglich gewesen wäre.

Hinweis zur geschlechtergerechten Sprache

In dieser Arbeit wird überwiegend eine geschlechtsneutrale Sprache verwendet. Aus Gründen der besseren Lesbarkeit wird in einzelnen Fällen das generische Maskulinum genutzt. Selbstverständlich sind damit Personen aller Geschlechter gleichermaßen gemeint.

Alle verwendeten Personenbezeichnungen gelten gleichermaßen für alle Geschlechter. Die Arbeit orientiert sich an den Empfehlungen des Leitfadens zur gendersensiblen und inklusiven Sprache der IU Internationale Hochschule.

Inhaltsverzeichnis

Kurzfassung	i
Abstract	ii
Danksagung	iii
Hinweis zur geschlechtergerechten Sprache	iv
Abbildungsverzeichnis	ix
Tabellenverzeichnis	x
Listings	xi
Abkürzungsverzeichnis	xii
1 Einleitung	1
1.1 Forschungsfragen	2
2 Theoretische Fundierung	3
2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen	3
2.2 Besonderheiten verteilter und cloud-nativer Anwendungen	4
2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces	5
2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung	7
2.5 Die Four Golden Signals als heuristischer Bezugsrahmen	7
2.6 SLIs, SLOs und Error Budgets	8
2.7 Alert Fatigue, Actionable Alerts und Runbooks	9
2.8 Zwischenfazit	10
3 Systemanalyse und Anforderungsdefinition: „frag.jetzt“	12
3.1 Fachlicher Kontext und Nutzungsszenario	12
3.2 Technische Architektur	12
3.3 Kritische User Journeys	14
3.3.1 Journey 1: Frage stellen.	14
3.3.2 Journey 2: Live-Voting.	14
3.3.3 Journey 3: KI-Zusammenfassung.	14
3.4 Risikopunkte und Zuordnung zu den Golden Signals	14
3.4.1 R1: Fehlende Timeouts und Resilienz.	14
3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.	15
3.4.3 R3: Backend als Kritischster Punkt ohne Observability.	15

3.4.4	R4: WebSocket-Verbindungsverlust.	15
3.4.5	R5: Fehlende Ressourcenlimits.	15
3.4.6	R6: Single-Host-Topologie als systemweiter Ausfallpfad.	15
3.5	Observability-Ist-Zustand	16
3.6	Ableitung der Observability-Anforderungen	16
3.6.1	A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.	16
3.6.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals.	17
3.6.3	A3: Strukturiertes, korrelierbares Logging.	17
3.6.4	A4: Handlungsfähige Alarmierung.	17
3.6.5	A5: Beobachtbarkeit der Infrastrukturschicht.	17
4	Methodisches Vorgehen und Bewertungsrahmen	18
4.1	Arbeitslogik der Arbeit	18
4.2	Vorgehen bei der Ableitung der Golden Signals pro Service	19
4.3	Kriterien für Stack-Auswahl und spätere Evaluation	20
4.3.1	Vergleichskriterien für die Stack-Auswahl	20
4.3.2	Bewertungskriterien für die Gesamtstrategie	21
5	Evaluation und Auswahl des Observability-Stacks	23
5.1	Festlegung der Vergleichskriterien	23
5.2	OpenTelemetry als verbindliche Instrumentierungsschicht	23
5.3	Vergleich der Zielarchitekturen	24
5.3.1	Elastic Observability	24
5.3.2	Grafana-Stack: Prometheus + Loki + Tempo + Grafana	25
5.3.3	Prometheus + Jaeger + Loki + Grafana	26
5.4	Vergleich und Synthese	26
5.5	Begründete Zielentscheidung für frag.jetzt	28
5.5.1	Entscheidungsbegründung im Kontext von frag.jetzt	28
5.5.2	Kompromisse und Einschränkungen	29
6	Konzeption der Observability-Strategie für frag.jetzt	30
6.1	Leitprinzipien der Strategie	30
6.2	Logische Zielarchitektur der Observability	31
6.2.1	Instrumentierungsschicht	32
6.2.2	Sammel- und Verarbeitungsschicht	33
6.2.3	Speicherschicht	34
6.2.4	Analyse- und Visualisierungsschicht	34
6.2.5	Einordnung im Kontext von frag.jetzt	35
6.3	Servicebezogene Operationalisierung der Four Golden Signals	36
6.3.1	PWA-Frontend	37

6.3.2	Spring-Boot-Backend	38
6.3.3	PostgreSQL	39
6.3.4	Externe KI-Dienste	40
6.4	Serviceübergreifende Signal-Matrix	40
6.5	Rollenbasiertes Informations- und Dashboard-Konzept	42
6.6	Alerting-Konzept und Runbook-Logik	43
6.6.1	Entwurfsgrundsätze	43
6.6.2	Alarmkategorien	44
6.6.3	Runbook-Verknüpfung	44
6.7	Anomalieerkennung als Erweiterungsperspektive	45
7	Prototypische Implementierung	46
7.1	Umfang und Abgrenzung der prototypischen Umsetzung	46
7.2	Instrumentierung ausgewählter Kernkomponenten	47
7.2.1	Observability-Stack und Telemetrie-Pipeline	47
7.2.2	Applikationsinstrumentierung über den OTel Java Agent	48
7.2.3	Infrastrukturexporter und Scrape-Konfiguration	48
7.2.4	Logging-Anbindung	49
7.2.5	Exemplarische Validierungsszenarien	50
7.3	Umsetzung des Golden-Signals-Dashboards	51
7.4	Alerting-Regeln und Beispiel-Runbooks	53
7.5	Herausforderungen und Lösungen bei der Umsetzung	55
7.5.1	Ressourcendimensionierung	55
7.5.2	Netzwerktopologie und Dienstkommunikation	55
7.5.3	Signalqualität und Rauschunterdrückung	56
7.6	Zwischenfazit	56
8	Evaluation der entwickelten Strategie	58
8.1	Evaluationsmethodik	58
8.2	Anforderungsbezogene Bewertung	58
8.2.1	A1: Ende-zu-Ende-Nachvollziehbarkeit	59
8.2.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals	60
8.2.3	A3: Strukturiertes, korrelierbares Logging	61
8.2.4	A4: Handlungsfähige Alarmierung	62
8.2.5	A5: Beobachtbarkeit der Infrastrukturschicht	63
8.3	Übergreifende Bewertungsdimensionen	64
8.4	Grenzen und methodische Einschränkungen	65
9	Diskussion	68
9.1	Einordnung der Ergebnisse im Verhältnis zur Literatur	68

9.2	Übertragbarkeit auf ähnliche cloud-native Plattformen	70
9.3	Wissenschaftlicher und praktischer Beitrag	71
	Literaturverzeichnis	72
	Index	75
	A Verzeichnis der KI-Nutzung	76
A.1	Copilot Prompts für die unterstützende Systemanalyse von frag.jetzt	77
A.2	C4-Container-Diagramm in Vollansicht	79

Abbildungsverzeichnis

2.1	Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing).	5
3.1	C4-Container-Diagramm der frag.jetzt-Architektur mit Applikationsdiensten, Infrastrukturkomponenten und Kommunikationswegen (Vollseitenansicht des Diagramms in Anhang A.2).	13
6.1	Logische Zielarchitektur der Observability-Pipeline fuer frag.jetzt. Das Schichtenmodell zeigt den Datenfluss von der Instrumentierung ueber Sammlung und Speicherung bis zur Auswertung. Gestrichelte Querverbindungen kennzeichnen Korrelationsmechanismen zwischen den Speicher-Backends.	36
7.1	Span-Waterfall eines Backend-Traces (<code>POST /livepoll/find</code>) mit 13 Spans und 15,4ms Gesamtdauer.	49
7.2	Service Graph aus Tempo mit den Kommunikationspfaden <code>user → fragjetzt-backend → fragjetzt-postgres</code> sowie <code>user → fragjetzt-ws-gateway</code>	50
7.3	Ausschnitt des Golden-Signals-Dashboards mit den Sektionen <i>Latency</i> und <i>Traffic</i>	53
7.4	Ausschnitt der <i>Errors</i> -Sektion mit Dummy-Fehlerrate zur Veranschaulichung der Fehlerindikatoren.	53
8.1	Trace-Waterfall in Tempo für den Endpunkt <code>GET /room/{id}/moderator</code> mit 11 Spans und zugeordneten Datenbankoperationen.	60
8.2	Log-to-Trace-Korrelation in Loki: Ein Logeintrag mit klickbarem TraceID-Link (links) navigiert zum zugehörigen Trace-Waterfall in Tempo (rechts).	61
8.3	Latenz-Alert im Zustand <i>Firing</i> nach 6 Minuten. Die Annotations enthalten Symptombeschreibung, Prüfpfad und Runbook-Link.	62
8.4	Infrastruktur-Sektion des Golden-Signals-Dashboards mit PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefe, Host-Ressourcen und Service Graph.	64
A.1	C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht	80

Tabellenverzeichnis

3.1	Applikationstragende Dienste und ihre Observability-Relevanz	13
3.2	Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung.	16
5.1	Vergleichende Bewertung der drei Observability-Zielarchitekturen	27
6.1	Serviceübergreifende Signal-Matrix mit Zuordnung von Dienst, Golden Signal, konkreter Messgröße, Erhebungszweck und primärer Stakeholder-Rolle.	41
7.1	Übersicht der instrumentierten Komponenten und beobachteten Telemetriesignale.	47
7.2	Zuordnung der prototypischen Umsetzung zu den Observability-Anforderungen.	57
8.1	Ergebnissynthese der anforderungsbezogenen Evaluation.	59
A.1	Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge	78

Listings

A.1 Copilot-Prompts zur Architekturanalyse	77
--	----

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLI	Service Level Indicator
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface
ARS	Audience Response System
FF	Forschungsfrage
TSDB	Time Series Database
REST	Representational State Transfer

1 Einleitung

Webbasierte Anwendungssysteme werden heute häufig als verteilte, cloud-native Anwendungen umgesetzt, die aus mehreren lose gekoppelten Services bestehen. Diese Architekturform erlaubt eine unabhängige Weiterentwicklung einzelner Komponenten sowie eine flexible Skalierung, erhöht jedoch zugleich den Aufwand für Betrieb, Überwachung und Fehleranalyse im Produkivsystem Faseeha et al., 2025; Sakinala, 2025. Insbesondere bei Microservices entstehen verteilte Fehlerzustände und komplexe Abhängigkeiten zwischen Diensten, wodurch Ursachen von Störungen nicht mehr eindeutig einzelnen Komponenten zugeordnet werden können Fischer et al., 2024.

Vor diesem Hintergrund gewinnt Observability als Erweiterung klassischer Monitoring-Konzepte an Bedeutung. Ziel von Observability ist es, den internen Zustand eines Systems anhand extern erfassbarer Anzeichen nachvollziehen zu können. Dazu werden Metriken, Logs und Traces kombiniert, um Zusammenhänge zwischen Systemverhalten, Fehlern und Leistungsmerkmalen sichtbar zu machen Faseeha et al., 2025. Empirische Untersuchungen aus dem Bereich cloud-nativer Anwendungen zeigen, dass ein strukturierter Einsatz von Observability-Praktiken die Analyse von Incidents unterstützt und die Reaktionszeit im Betrieb verkürzen kann Giamattei et al., 2024; Sakinala, 2025.

Ein häufig verwendeter Orientierungsrahmen zur Strukturierung servicebezogener Beobachtungsdaten sind die von Google im Kontext des Site Reliability Engineering (SRE) beschriebenen *Four Golden Signals*: Latenz, Traffic, Fehler und Sättigung Ewaschuk, 2017. Diese Signalarten erfassen grundlegende technische Symptome eines Dienstes und eignen sich als Ausgangspunkt zur Ableitung erster Monitoring-Indikatoren. Für den produktiven Betrieb liefern sie jedoch primär Rohinformationen, die erst durch Kontext, Interpretation und Bewertung für Menschen handlungsrelevant werden. Google SRE weist selbst darauf hin, dass die Wirksamkeit solcher Signale maßgeblich davon abhängt, wie sie in Alerting, Entscheidungsprozesse und betriebliche Abläufe eingebettet sind Ewaschuk, 2017.

Die Open-Source-Plattform *frag.jetzt* verfügt über eine solide Entwicklungs- und Deployment-Infrastruktur, jedoch ist die Erfassung und Auswertung von Betriebsdaten derzeit nicht als durchgängiges Observability-Konzept ausgelegt. Metriken, Logs, Traces und Alerting werden nicht konsistent entlang des gesamten Anfragepfads betrachtet, wodurch Zusammenhänge zwischen Nutzerverhalten, Backend-Verarbeitung und externen Abhängigkeiten nur eingeschränkt nachvollziehbar sind. Diese Einschätzung stützt sich auf die öffentlich verfügbare Projektdokumentation und Architekturübersicht des frag.jetzt-Repositories TH Mittelhessen und arsnova, 2025 sowie auf allgemeine Beobachtungen zu typischen Defiziten in vergleichbaren Systemen Giamattei et al., 2024.

Ziel dieser Arbeit ist die Entwicklung einer Observability-Strategie für *frag.jetzt*, bei der technische Messgrößen systematisch mit nutzungsbezogenen Fragestellungen verknüpft werden. Die Four Golden Signals dienen dabei als unterstützender Referenzrahmen zur Strukturierung der Beobachtungsdaten, nicht jedoch als alleinige Grundlage für betriebliche Entscheidungen. Die Signale werden servicespezifisch für das PWA-Frontend, das Spring-Boot-Backend, die PostgreSQL-Datenbank sowie angebundene KI-Dienste konkretisiert und in messbare Indikatoren überführt. Ergänzend wird untersucht, welche Open-Source-Observability-Werkzeuge die Anforderungen von *frag.jetzt* im Hinblick auf Erfassung und betriebliche Nutzbarkeit am besten erfüllen.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Ausgestaltung eines Alerting-Ansatzes, der Auffälligkeiten innerhalb der Plattform mit Fragen der Relevanz und Dringlichkeit verknüpft. Dabei steht nicht allein die Detektion von Abweichungen im Vordergrund, sondern insbesondere die Bewertung, ob eine Handlung erforderlich ist, wen ein Problem betrifft und welche Auswirkungen es auf die Nutzung der Plattform hat. Forschungsarbeiten zeigen, dass eine Kombination aus priorisiertem Alerting und datenbasierten Analyseverfahren dazu beitragen kann, irrelevante Alarmierungen zu reduzieren und menschliche Entscheidungsprozesse im Betrieb zu unterstützen Jalalvand et al., 2024. Aufbauend auf den definierten Alerts werden Runbooks konzipiert, die strukturierte Diagnose- und Handlungsschritte für wiederkehrende Incident-Szenarien bereitstellen.

1.1 Forschungsfragen

Die Forschungsfragen dieser Arbeit lauten:

1. FF1: Wie können die Four Golden Signals für die spezifischen Services von *frag.jetzt* (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und instrumentiert werden?
2. FF2: Welche Observability-Stack-Kombination (Prometheus/Grafana, ELK Stack, Jaeger) ist am besten geeignet für die Monitoring-Anforderungen von *frag.jetzt*?
3. FF3: Wie kann ein intelligentes Alert-System entwickelt werden, das False Positives minimiert und actionable insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

2 Theoretische Fundierung

2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen

Für die Entwicklung einer Observability-Strategie einer Microservices-Anwendung ist zunächst eine begriffliche Trennung von Monitoring und Observability erforderlich. Beide Konzepte zielen darauf ab, den Betrieb softwarebasierter Anwendungen transparent und steuerbar zu machen, unterscheiden sich jedoch in Umfang und diagnostischem Anspruch. Monitoring bezeichnet die kontinuierliche Erfassung und Analyse von Betriebsdaten wie Logs und Metriken, um den aktuellen Zustand eines Systems sichtbar zu machen (Usman et al., 2022, S. 86907). Es setzt voraus, dass bekannte Zustandsindikatoren und erwartbare Abweichungen vorab definiert wurden, und prüft, ob diese Grenzwerte eingehalten werden.

Observability geht über diese hypothesengeleitete Überwachung hinaus. Im Kern beschreibt der Begriff die Fähigkeit, aus den externen Ausgaben eines Systems auf dessen interne Zustände zu schließen (Usman et al., 2022, S. 86907). In verteilten Anwendungen sind diese Ausgaben vor allem Telemetriedaten wie Logs, Metriken und Traces. Entscheidend ist dabei nicht allein die Datenerhebung, sondern die Möglichkeit, auch neuartige und vorab nicht antizipierte Störungen nachvollziehen zu können. Solche unvorhergesehenen Fehlerbilder, die als *unknown unknowns* bezeichnet werden, entstehen gerade in verteilten Architekturen aus dem Zusammenspiel vieler parallel agierender Komponenten (Banerjee und Singh, 2024, S. 916). Die Interviewstudie von Niedermaier et al. (2019, S. 11) bestätigt diese Perspektive aus der Industriepraxis. Unternehmen wechseln zunehmend von einer komponentenisolierten Zustandsprüfung hin zu einer nutzungs- und geschäftsprozessorientierten Gesamtsicht. Nicht die einzelne Infrastrukturmetrik steht dann im Mittelpunkt, sondern die Frage, ob Anfragen aus Endnutzersicht zuverlässig und performant verarbeitet werden.

Monitoring bildet damit die notwendige Voraussetzung für Observability, wird jedoch nicht davon ersetzt (Usman et al., 2022, S. 86907). Für die vorliegende Arbeit bedeutet das, eine Strategie, die sich auf das reine Erfassen bekannter Grenzwerte beschränkt, reicht für ein verteiltes System wie frag.jetzt nicht aus. Benötigt wird vielmehr ein Ansatz, der technische Messdaten mit diagnostischer Tiefe und Nutzerperspektive verbindet.

2.2 Besonderheiten verteilter und cloud-nativer Anwendungen

Die begriffliche Abgrenzung gewinnt ihre praktische Bedeutung erst vor dem Hintergrund konkreter Systemarchitekturen. Verteilte und cloud-native Anwendungen werden heute häufig als Microservices-Architekturen umgesetzt, welche aus zahlreichen, oft unabhängig deploybaren Diensten bestehen, die über Netzwerkschnittstellen kommunizieren und in Containern oder vergleichbaren Laufzeitumgebungen betrieben werden (Sakinala, 2025, S. 102). Instanzen werden fortlaufend erzeugt und beendet, wodurch solche Umgebungen als operativ deutlich anspruchsvoller gelten als klassische Monolithen (Li et al., 2022, S. 1). Die resultierende Komplexität lässt sich laut Usman et al. (2022, S. 86908) auf vier Eigenschaften verdichten: Modularität, Verteilung, Dynamik und Flüchtigkeit. Frühere Überwachungsansätze, die auf einzelne Schichten oder Komponenten zugeschnitten waren, stoßen unter diesen Bedingungen an Grenzen, weil sie Interaktionen zwischen Diensten zur Laufzeit nur unzureichend abbilden (Usman et al., 2022, S. 86908).

Der Grund liegt in der starken Laufzeitkopplung trotz struktureller Entkopplung. Ein Leistungsproblem in einem Dienst kann durch Zustandsänderungen in einer ganz anderen Komponente verursacht werden, etwa durch verzögerte Datenbankzugriffe oder überlastete Downstream-Pfade. Isolierte Überwachungskonzepte ohne übergreifenden Kontext führen zu diagnostischen Lücken und erschweren die Fehlerbehebung in verteilten Architekturen erheblich (Niedermaier et al., 2019, S. 8). Auch die befragten Interviewteilnehmer berichten von der Schwierigkeit, aussagekräftige Schlüsse aus Alarmen zu ziehen, sobald eine Anfrage mehrere, von unterschiedlichen Teams verantwortete Komponenten durchläuft (Niedermaier et al., 2019, S. 8).

Daraus ergibt sich die Notwendigkeit einer Ende-zu-Ende-Sicht. Distributed Tracing gilt als das Verfahren, mit dem Serviceaufrufketten pro Anfrage nachvollzogen werden können (Li et al., 2022, S. 4). Ein Trace erfasst den kausal zusammenhängenden Ablauf einer Anfrage durch mehrere Stationen eines verteilten Systems und macht sichtbar, wo Verzögerungen oder Fehler auftreten. Die dafür erforderliche Weitergabe von Identifikatoren und Metadaten entlang des Anfragepfads ist eine entscheidende Voraussetzung, um Fehlerursachen in der Datenmenge moderner Anwendungen überhaupt eingrenzen zu können (Niedermaier et al., 2019, S. 11). Abbildung 2.1 verdeutlicht den Unterschied zwischen lokaler Komponentenüberwachung und einer durchgängigen Anfrageverfolgung über mehrere Dienste.

Für die nachfolgenden Kapitel folgt daraus, dass Klassisches, rein lokales Monitoring notwendig bleibt, für Microservices aber nicht hinreichend ist. Servicebezogene Messgrößen, korrelierte Telemetrie und handlungsorientierte Alarme setzen eine architekturübergreifende Beobachtbarkeit voraus.

Gegenüberstellung: Lokales Komponentenmonitoring vs. Durchgängige Anfrageverfolgung

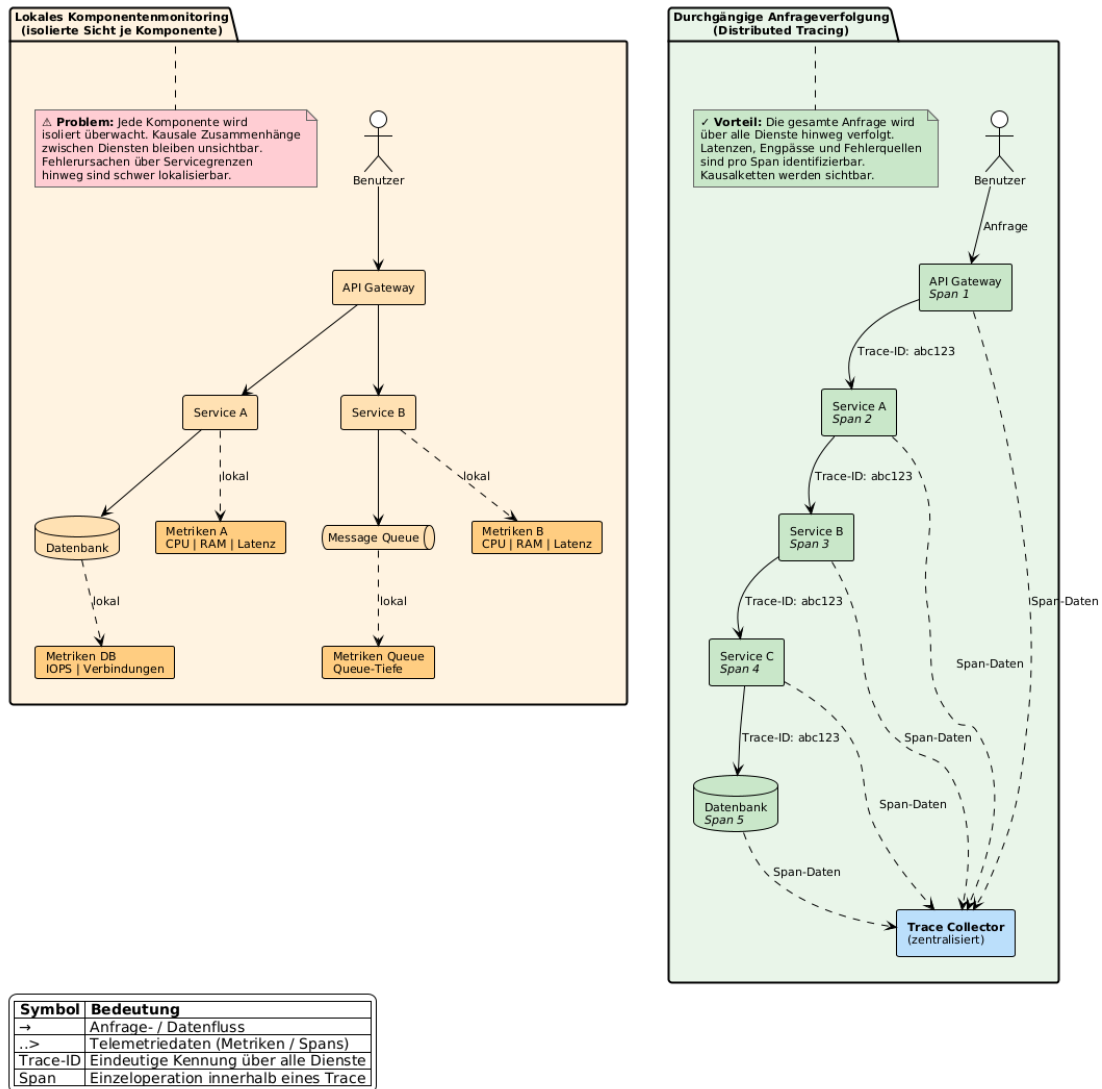


Abbildung 2.1: Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing). Quelle: Eigene Darstellung.

2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces

Für die Beobachtbarkeit verteilter Anwendungen sind Logs, Metrics und Traces die drei am weitesten verbreiteten Telemetrieformen. Die Literatur bezeichnet sie häufig als Grundpfeiler der Observability, betont jedoch zugleich, dass sie nicht die einzig denkbaren Datenarten darstellen (Li et al., 2022, S. 6; Usman et al., 2022, S. 86912–86913; Sakinala, 2025, S. 102). Keine dieser Formen liefert für sich genommen eine vollständige Betriebssicht. Ihr Nutzen entsteht erst aus den jeweils unterschiedlichen Perspektiven, die sie auf dasselbe Laufzeitverhalten eröffnen.

Logs dokumentieren diskrete Ereignisse in zeitlicher Abfolge und bieten damit eine sehr feingranulare Beschreibung des Systemverhaltens. Sie geben Einzelfälle mit hohem Detailgrad wieder und können dadurch Hinweise auf Ort, Zeitpunkt und technischen Kontext einer Störung liefern. Als zeitgestempelte Aufzeichnungen eignen sie sich insbesondere für die Analyse unerwarte-

ter Fehlerzustände (Albuquerque und Correia, 2025, S. 2). Dabei können Logs verschiedene Schweregrade (Info, Warnung, Fehler) abbilden und bieten deshalb oft den ersten Zugang zu konkreten Fehlersituationen (Usman et al., 2022, S. 86913). Allerdings erzeugen Microservices-Architekturen erhebliche Logmengen. Die zeitliche Zuordnung von Logereignissen über mehrere Dienste hinweg erfordert strukturierte Formate und Korrelationskennungen (Sakinala, 2025, S. 102). Zudem ist die Ableitung von Trends aus Logs vergleichsweise aufwendig, da die Verarbeitung großer Textmengen rechenintensiv ist und keine unmittelbare Echtzeitsicht bietet (Albuquerque und Correia, 2025, S. 11).

Metrics verfolgen eine andere Logik. Sie verdichten Laufzeitverhalten zu numerischen Messwerten, die über die Zeit aggregiert und ausgewertet werden können. Als Zahlendarstellungen des Betriebszustands lassen sie sich vergleichsweise einfach speichern, abfragen und korrelieren (Usman et al., 2022, S. 86913). Damit eignen sich Metrics besonders für Zustandsentwicklungen, Schwellenüberwachung und Alarmauslösung. Sowohl technische Größen wie Anfragerate oder Fehlerrate als auch fachlich orientierte Kennzahlen können abgebildet werden (Albuquerque und Correia, 2025, S. 12). Gleichzeitig abstrahieren Metrics stärker als Logs. Sie zeigen, dass eine Latenz steigt, erklären jedoch nicht automatisch den Grund. Reine Infrastrukturmetriken können für anwendungsbezogene Diagnosen zu grobkörnig ausfallen (Albuquerque und Correia, 2025, S. 11). Für eine Observability-Strategie bedeutet dies, dass Metrics unverzichtbar für Trendanalyse, Kapazitätsplanung und Alerting sind, eine tiefergehende Ursachenanalyse aber nicht ersetzen.

Während Metriken vor allem Zustandsverläufe aggregiert sichtbar machen, erlauben Traces die Rekonstruktion konkreter Anfragepfade über mehrere Komponenten hinweg. Damit ergänzen sie Metriken um eine prozessbezogene Perspektive, die insbesondere für die Lokalisierung von Verzögerungen und Übergabefehlern in verteilten Systemen relevant ist (Albuquerque und Correia, 2025, S. 2). Distributed Tracing schafft damit eine Ende-zu-Ende-Sicht auf komplexe Anfragepfade (Sakinala, 2025, S. 103). Allerdings ist Tracing aufwendiger als die Erhebung einzelner Metriken, da die Instrumentierung ressourcenintensiver und komplexer sein kann (Giamattei et al., 2024, S. 15), und Sampling-Strategien erforderlich werden, um den Overhead in ein vertretbares Verhältnis zum diagnostischen Nutzen zu bringen (Albuquerque und Correia, 2025, S. 10).

Aus diesen Unterschieden ergibt sich das eigentliche Zusammenspiel. Logs liefern ereignisnahe Details, Metrics verdichten Verhalten zu beobachtbaren Kennzahlen, und Traces rekonstruieren dienstübergreifende Abläufe. Der komplementäre Charakter dieser Datenarten ist in der Literatur breit anerkannt (Li et al., 2022, S. 6). Über Korrelationskennungen können Logs mit Traces verknüpft werden, sodass zeitliche Abläufe und Fehlermeldungen gemeinsam auswertbar werden (Albuquerque und Correia, 2025, S. 10). Eine leistungsfähige Observability-Plattform muss Anomalien daher nicht isoliert, sondern im Zusammenspiel aller drei Datenarten sichtbar machen (Usman et al., 2022, S. 86914).

2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung

Je stärker Observability in Microservices auf mehrere Datenarten, Sprachen und Werkzeuge angewiesen ist, desto dringender wird eine gemeinsame Instrumentierungsgrundlage. Heterogene Monitoringlandschaften leiden häufig unter proprietären Formaten und Integrationsengpässen (Sakinala, 2025, S. 105). Standardisierte Datenformate sind notwendig, um die Wartbarkeit der Überwachungsinfrastruktur langfristig zu sichern (Giamattei et al., 2024, S. 17). OpenTelemetry adressiert dieses vorgelagerte Problem der uneinheitlichen Telemetrieerzeugung.

Das Projekt wird in der Literatur als herstellerneutrale Instrumentierungsschicht eingeordnet, die aus der Zusammenführung von OpenCensus und OpenTracing hervorgegangen ist und die Erzeugung sowie Erfassung von Telemetriedaten in verteilten Anwendungen vereinheitlichen soll (Usman et al., 2022, S. 86911). Offene Spezifikationen wie OpenTelemetry erlauben es Entwicklungsteams, ihren Code zu instrumentieren, ohne sich an ein bestimmtes Analyse-Backend zu binden (Li et al., 2022, S. 22). Dieser Ansatz reduziert Herstellerabhängigkeiten und ermöglicht die Anbindung an unterschiedliche Plattformen (Sakinala, 2025, S. 106). Für polyglotte Systemlandschaften ist das besonders wertvoll, weil dieselben Instrumentierungsprinzipien über verschiedene Laufzeitumgebungen hinweg beibehalten werden können.

Zugleich darf OpenTelemetry nicht mit einem vollständigen Observability-Stack gleichgesetzt werden. Faseeha et al. (2025, S. 72030) unterscheiden explizit, dass OpenTelemetry die Instrumentierung übernimmt, während andere Werkzeuge für Speicherung, Darstellung und Alarmierung zuständig bleiben. Auch als kompatible Grundlage konzipiert, kann es sowohl mit offenen Werkzeugen als auch mit kommerziellen Plattformen zusammenarbeiten (Sakinala, 2025, S. 107). Dabei erzeugt Instrumentierung stets zusätzlichen Entwicklungs- und Pflegeaufwand und muss deshalb zur Entwicklungsgeschwindigkeit des jeweiligen Projekts passen (Giamattei et al., 2024, S. 15). Für die spätere Stack-Evaluation dieser Arbeit ist diese Differenzierung entscheidend. Die Wahl der Instrumentierungsschicht und die Wahl der Analyse-Backends sind zwei getrennte Architekturentscheidungen.

2.5 Die Four Golden Signals als heuristischer Bezugsrahmen

Die Four Golden Signals – *Latency*, *Traffic*, *Errors* und *Saturation* – bilden in der SRE-nahen Literatur einen bewusst reduzierten, aber praxistauglichen Rahmen zur Beobachtung nutzerorientierter Dienste. Ewaschuk (2017, S. 7) beschreibt sie als die vier Größen, auf die sich die Überwachung eines clientseitig wahrnehmbaren Dienstes mindestens konzentrieren sollte, wenn nur ein begrenzter Satz von Metriken erhoben werden kann.

Latency erfasst die Zeit, die ein Dienst zur Bearbeitung einer Anfrage benötigt. Dabei müssen erfolgreiche und fehlgeschlagene Anfragen getrennt betrachtet werden, weil schnell ausgelieferte Fehlantworten die Gesamtstatistik verzerren können (Ewaschuk, 2017, S. 7).

Traffic bezeichnet die auf einen Dienst wirkende Nachfrage und muss als dienstspezifische Lastgröße verstanden werden, etwa als HTTP-Anfragen pro Sekunde oder als Anzahl aktiver Sitzungen (Ewaschuk, 2017, S. 7).

Errors umfassen die Rate fehlgeschlagener Anfragen. Dabei geht es nicht nur um explizite Fehler wie HTTP-500-Antworten, sondern auch indirekte Fehlzustände, etwa sachlich falsche Ergebnisse oder Antwortzeiten jenseits zugesicherter Schwellen (Ewaschuk, 2017, S. 8).

Saturation beschreibt, wie stark ein Dienst seine limitierenden Ressourcen ausschöpft. Da Dienste häufig bereits vor Erreichen ihrer nominellen Volllast Leistungsabfälle zeigen, werden steigende Latenzen in der Praxis als Frühindikator für bevorstehende Kapazitätsengpässe interpretiert (Ewaschuk, 2017, S. 8).

Diese vier Größen werden auch von Usman et al. (2022, S. 86913–86914) als wesentliche, aus Telemetriedaten abgeleitete Messperspektiven eingeordnet, die als Grundlage für Alerting, Fehlersuche und Kapazitätsplanung dienen. Solche Metriken sind jedoch nur dann analytisch wertvoll, wenn sie an die konkrete Rolle des jeweiligen Dienstes angepasst werden (Albuquerque und Correia, 2025, S. 12). Ergänzend können fachliche Kennzahlen sinnvoll sein, wenn technische Indikatoren allein nicht ausreichen, um die Servicequalität zu bewerten.

Die Stärke dieses Rahmenwerks liegt nicht in Vollständigkeit, sondern in seiner Ordnungsfunktion. Es zwingt dazu, Beobachtungsgrößen nicht beliebig zu sammeln, sondern entlang weniger, aus Nutzersicht aussagekräftiger Dimensionen zu strukturieren. Für die vorliegende Arbeit bilden die Four Golden Signals den heuristischen Ausgangspunkt, um in den späteren Kapiteln servicebezogene Messgrößen für frag.jetzt abzuleiten. Sie ersetzen jedoch weder die kontextabhängige Auswahl konkreter Metriken noch die Verknüpfung mit Zielwerten und Alarmen.

2.6 SLIs, SLOs und Error Budgets

Während die Four Golden Signals einen Orientierungsrahmen für relevante Beobachtungsgrößen liefern, beantworten sie noch nicht, welches Niveau an Zuverlässigkeit als ausreichend gelten soll. An dieser Stelle werden Service Level Indicators (SLIs) und Service Level Objectives (SLOs) bedeutsam. In der Interviewstudie von Niedermaier et al. (2019, S. 11) werden SLOs als servicebezogene Zielgrößen beschrieben, die geschäftsrelevante Anforderungen aus der Nutzerperspektive fassbar machen, etwa in Form akzeptabler Nichtverfügbarkeit. SLIs binden konkrete Messgrößen an diese Ziele zurück. Damit entsteht eine Brücke zwischen technischer Messung und fachlicher Erwartung, die als gemeinsamer Referenzrahmen für die teamübergreifende Zusammenarbeit dient (Niedermaier et al., 2019, S. 12).

Gerade in verteilten Anwendungen ist diese Brücke wichtig, weil isolierte Infrastrukturmetriken wenig darüber aussagen, ob ein Dienst seine Aufgabe aus Nutzersicht erfüllt. Bei geschäftskritischen Diensten sollte die gewünschte Dienstqualität durch SLOs definiert und anhand geeigneter SLIs überwacht werden. Häufig herangezogene Messgrößen sind beispielsweise Verfügbarkeit, Latenz und Fehlerraten (Vinnakota und Kolla, 2025, S. 176). Alarme sollten nicht auf

beliebige Grenzwertverletzungen reagieren, sondern an solchen Schwellen orientiert sein, deren Überschreitung die Einhaltung definierter SLOs gefährdet.

In dieser Logik lässt sich auch das Konzept des Error Budgets verorten. Das Error Budget bezeichnet den rechnerisch aus einem SLO abgeleiteten Spielraum für tolerierte Unzuverlässigkeit innerhalb eines festgelegten Zeitfensters (Dasari, 2025, S. 993). Ein Dienst mit einem Verfügbarkeitsziel von 99,95 % darf demnach in einem Monat nur ein bestimmtes Maß an Ausfallzeit (21 min) aufweisen, ohne dass die Zielvorgabe als verletzt gilt. Die praktische Bedeutung dieses Konzepts liegt weniger in der reinen Berechnung als in der Steuerungsfunktion. Solange Budget vorhanden ist, können neue Releases oder Änderungen mit kontrolliertem Risiko ausgerollt werden. Ist das Budget jedoch aufgebraucht, erhält Stabilisierung Vorrang gegenüber Weiterentwicklung (Dasari, 2025, S. 993–994). Diese Priorisierungslogik findet sich in ähnlicher Form auch bei Gowda und Hameed (2022, S. 7), der im Kontext von Self-Healing-Frameworks beschreibt, wie SLO-Überwachung mit automatisierten Maßnahmen verknüpft werden kann.

Error Budgets formalisieren diesen Spielraum und machen ihn für betriebliche Entscheidungen nutzbar. Damit verändert sich auch die Funktion von Alerting. Ein Alarm, der lediglich auf lokale Ressourcenauslastung oder starre Einzelgrenzwerte reagiert, kann technisch korrekt und dennoch betrieblich wenig hilfreich sein. Ewaschuk (2017, S. 13) illustriert diese Logik am Beispiel von Googles Bigtable-Dienst, bei dem eine Neujustierung der SLOs und die Abschaltung zu häufig feuender E-Mail-Alarme dazu führten, dass das Team sich wieder auf die Behebung grundlegender Probleme konzentrieren konnte, anstatt ständig Symptome zu bearbeiten.

2.7 Alert Fatigue, Actionable Alerts und Runbooks

Die Literatur zu Monitoring und Alerting zeigt übereinstimmend, dass nicht die Menge an Alarmen, sondern deren geringe Handlungsqualität ein zentrales Betriebsproblem darstellt. Häufige Alarmierungen unterbrechen Mitarbeitende in ihrer Arbeit oder Erholungszeit. Bei zu hoher Frequenz werden Warnungen nur noch überflogen oder ignoriert, sodass echte Störungen im Rauschen (Noise) untergehen können (Ewaschuk, 2017, S. 4). Dieser Effekt wird als Alert Fatigue bezeichnet. Insbesondere nicht handlungsrelevante oder redundante Benachrichtigungen stumpfen das Bereitschaftspersonal ab (Vinnakota und Kolla, 2025, S. 173).

Dieses Muster ist nicht auf Infrastrukturmonitoring beschränkt. In einer strukturell vergleichbaren Problemlage zeigen Jalalvand et al. (2024, S. 2–3) für Security Operations Centres (Einrichtungen zur Überwachung, Analyse und Reaktion auf Cyberangriffe), dass hohe Alarmvolumina, zahlreiche False Positives und mangelnde Priorisierung Analyseteams überlasten. Obwohl der fachliche Kontext dort ein anderer ist, lässt sich die Grundstruktur übertragen. Zu viele schwach priorisierte Meldungen beeinträchtigen die Reaktionsfähigkeit auf tatsächlich kritische Ereignisse. Für die wissenschaftliche Einordnung von FF3 (vgl. Abschnitt 1.1) ist dieser domänenübergreifende Erkenntnis aufschlussreich, weil er verdeutlicht, dass Alarmmüdigkeit kein Randproblem einzelner Werkzeuge, sondern ein allgemeines Entwurfsproblem alarmintensiver

Betriebsumgebungen ist.

Vor diesem Hintergrund gewinnt der Begriff des *actionable alert* an Bedeutung. Ewaschuk (2017, S. 4) warnt davor, einen Menschen allein deshalb zu alarmieren, weil ein Zustand ungewöhnlich erscheint. Er argumentiert stattdessen, dass sich die Qualität von Alarmierungen daran beurteilen lässt, ob sie auf eine betrieblich relevante Störung mit erkennbarem Handlungsbedarf hinweisen. Alerts sollten daher nur dann ausgelöst werden, wenn eine tatsächliche oder unmittelbar drohende Beeinträchtigung für den Nutzer vorliegt (Ewaschuk, 2017, S. 11–12). Beim Eintreffen eines Alarms sollte der operative Kontext so aufbereitet sein, dass unmittelbar erkennbar wird, welches Problem vorliegt, welche Ursachen wahrscheinlich sind und welche Maßnahmen eingeleitet werden müssen sowie wer für die Bearbeitung zuständig ist (Vinnakota und Kolla, 2025, S. 174).

Wirksame Alerting-Strategien müssen zwischen handlungsrelevanten Hinweisen und bloßem Hintergrundrauschen differenzieren (Sakinala, 2025, S. 107). Als problematisch gelten insbesondere starre Schwellwerte in dynamischen Umgebungen, weil sie saisonale Schwankungen oder Abhängigkeiten zwischen Diensten unzureichend berücksichtigen. Dynamische Schwellen, mehrdimensionale Regeln und die laufende Überprüfung von False Positives werden als wesentliche Gestaltungsprinzipien genannt (Vinnakota und Kolla, 2025, S. 173, 176).

Eng mit handlungsfähiger Alarmierung ist die Rolle von Runbooks verbunden. Jede kritische Meldung sollte mit einem Runbook verknüpft sein, das den Bearbeitenden unabhängig vom Vorwissen in die Lage versetzt, die Störung einzuordnen und erste Gegenmaßnahmen einzuleiten. Dazu gehören Verweise auf relevante Observability-Werkzeuge, vordefinierte Behebungsschritte, klare Eskalationspfade und Angaben zur Zuständigkeit (Vinnakota und Kolla, 2025, S. 174). Der Nutzen liegt nicht nur in schnellerer Reaktion, sondern auch in der Wissenssicherung. Gleichzeitig sind veraltete Runbooks problematisch und müssen regelmäßig gepflegt werden. Für wiederkehrende, schematische Reaktionsmuster empfiehlt die SRE-Literatur eine konsequente Automatisierung. Alarmreaktionen, die rein algorithmischen Mustern folgen, sollten nicht dauerhaft menschliche Aufmerksamkeit binden, sondern Anlass geben, die zugrunde liegende Ursache zu beheben oder den Prozess zu automatisieren (Ewaschuk, 2017, S. 14).

2.8 Zwischenfazit

Die theoretische Fundierung dieses Kapitels zeigt, dass Beobachtbarkeit in verteilten und cloud-nativen Anwendungen über die isolierte Überwachung einzelner Komponenten hinausgehen muss. Fehlerursachen entstehen häufig dienstübergreifend und werden erst aus einer gemeinsamen Sicht auf Anfrageverläufe erkennbar (Niedermaier et al., 2019, S. 10–11). Logs, Metriken und Traces liefern jeweils unterschiedliche Perspektiven auf dasselbe Laufzeitverhalten und müssen in Beziehung gesetzt werden, um Latenzen, Fehlerursachen und Ressourceneffekte entlang realer Ablaufketten nachvollziehen zu können (Li et al., 2022, S. 6; Albuquerque und Correia, 2025, S. 10). OpenTelemetry wurde als standardisierte Instrumentierungsschicht eingeordnet, die eine herstellerübergreifende Erzeugung und Korrelation von Telemetriedaten unterstützt,

ohne selbst bereits Visualisierung oder Alerting bereitzustellen (Usman et al., 2022, S. 86911; Sakinala, 2025, S. 105–106). Die Four Golden Signals bieten einen heuristischen Ordnungsrahmen, um Beobachtungsgrößen entlang der Dimensionen Latency, Traffic, Errors und Saturation zu strukturieren (Ewaschuk, 2017, S. 7–8).

Ebenso wurde deutlich, dass eine Observability-Strategie erst dann betriebliche Steuerungswirkung entfaltet, wenn Messwerte mit Zielgrößen und Reaktionslogiken verknüpft werden. SLOs und SLIs überführen technische Kennzahlen in eine nutzungsbezogene Bewertung der Dienstqualität (Niedermaier et al., 2019, S. 11). Error Budgets formalisieren den daraus abgeleiteten Spielraum für tolerierte Abweichungen und ermöglichen eine datengestützte Priorisierung zwischen Weiterentwicklung und Stabilisierung (Dasari, 2025, S. 993–994). Für das Alerting bedeutet das, dass Warnungen nicht primär laut, sondern handlungsfähig sein müssen. Die SRE-Literatur betont symptomorientierte Alarmierung, weil Alarmer vor allem auf reale oder unmittelbar bevorstehende Nutzerbeeinträchtigungen reagieren sollen (Ewaschuk, 2017, S. 15). Ergänzend zeigen neuere Arbeiten, dass Alerts mit Kontext, Zuständigkeit und gepflegten Runbooks verbunden sein sollten, um Alarmmüdigkeit und unnötige Reibung im Incident-Handling zu begrenzen (Vinnakota und Kolla, 2025, S. 174).

Aus diesen Grundannahmen ergibt sich die Brücke zum weiteren Verlauf der Arbeit. Im nächsten Kapitel wird frag.jetzt als konkreter Anwendungskontext analysiert, damit aus Architektur, Nutzungspfaden und Risikopunkten jene Anforderungen abgeleitet werden können, die anschließend in eine servicebezogene Observability-Strategie überführt werden.

3 Systemanalyse und Anforderungsdefinition: „frag.jetzt“

Die folgende Systemanalyse basiert auf einer systematischen Auswertung des Quellcodes, der Deployment-Konfigurationen und der Abhängigkeitsstruktur der Repositories des frag.jetzt-Ökosystems. Ziel ist die Identifikation observability-relevanter Architekturmerkmale, Risikopunkte und bestehender Instrumentierungslücken als Grundlage für die Anforderungsdefinition. Zur Unterstützung der Analyse wurde ein KI-Werkzeug eingesetzt und anschließend manuell verifiziert (siehe Anhang A.1 und A.1).

3.1 Fachlicher Kontext und Nutzungsszenario

frag.jetzt ist ein webbasiertes Audience-Response-System für Lehrveranstaltungen. Teilnehmende können anonym Fragen stellen, auf Beiträge abstimmen und an Live-Umfragen teilnehmen. Ergänzend bietet das System KI-gestützte Funktionen wie Keyword-Extraktion und thematische Zusammenfassungen. Das Nutzungsmuster ist durch synchrone Spitzenlast geprägt. Innerhalb kurzer Veranstaltungsfenster können mehrere hundert Teilnehmende gleichzeitig aktiv sein, während die Last außerhalb nahe null liegt. Für die Observability-Strategie folgt daraus, dass Engpässe vor allem unter Spitzenlast erkannt werden müssen, nicht im Tagesdurchschnitt. Niedermaier et al. (2019, S. 8) identifizieren solche Dynamik als zentrale Herausforderung beim Monitoring verteilter Systeme, da konventionelle Schwellenwerte temporäre Lastspitzen nicht zuverlässig abbilden.

3.2 Technische Architektur

Das Ökosystem umfasst fünf Repositories, die über Docker Compose auf einem einzelnen Host zu zehn Containern orchestriert werden. Kubernetes kommt nicht zum Einsatz. Die Anwendung gliedert sich in vier applikationstragende Dienste (Tabelle 3.1) und mehrere Infrastrukturkomponenten.

Die Infrastruktur umfasst die PostgreSQL-Hauptdatenbank, eine dedizierte Keycloak-Datenbank, eine separate AI-Datenbank mit logischer Replikation aus der Hauptdatenbank, RabbitMQ als Message Broker, Keycloak für Authentifizierung (OIDC/SSO) und einen E-Mail-Dienst. Für die Observability-Strategie ist die Deployment-Topologie wesentlich. Alle Container laufen auf einem einzelnen Host ohne konfigurierte Ressourcenlimits (CPU, Speicher) und ohne Health-Check-Direktiven auf Container-Ebene.

Tabelle 3.1: Applikationstragende Dienste und ihre Observability-Relevanz. Quelle: Eigene Darstellung.

Dienst	Technologie	Observability-Relevanz
Frontend	Angular, Nginx	Einstiegspunkt; Nginx routet an Backend (/api), WS-Gateway (/gateway), Keycloak (/auth), AI Provider (/ai)
Backend	Spring Boot, WebFlux, R2DBC	Zentraler kritischer Dienst: 95 Endpunkte, 19 Controller
WS-Gateway	Kotlin, Netty, STOMP	Echtzeit-Updates via RabbitMQ; Session-Tracking in-memory
AI Provider	Python, FastAPI, Lang-Chain	19 LLM-Provider; externe Abhängigkeiten ohne Timeout

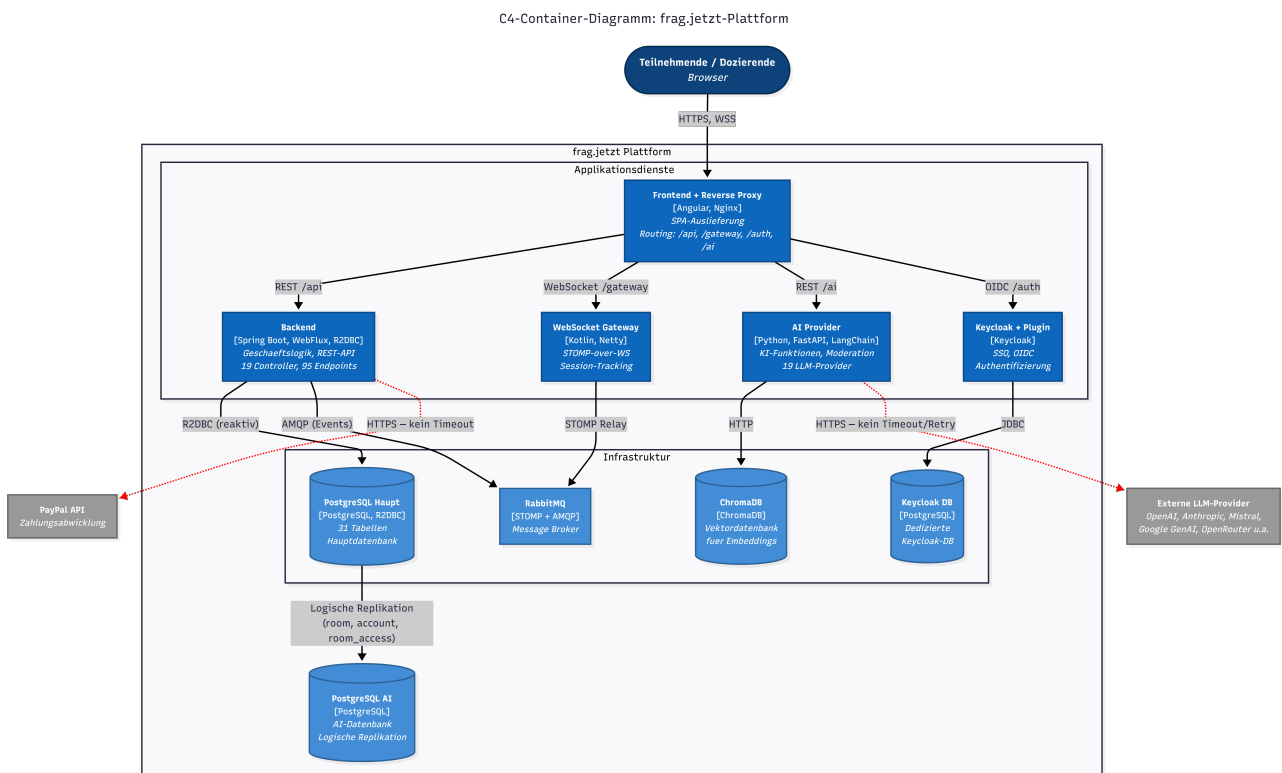


Abbildung 3.1: C4-Container-Diagramm der frag.jetzt-Architektur mit Applikationsdiensten, Infrastrukturkomponenten und Kommunikationswegen (Vollseitenansicht des Diagramms in Anhang A.2). Quelle: Eigene Darstellung.

3.3 Kritische User Journeys

Eine wirksame Observability-Strategie orientiert sich an den tatsächlichen Nutzungspfaden, nicht an abstrakten Systemkomponenten (Niedermaier et al., 2019, S. 10–11). Für frag.jetzt lassen sich drei geschäftskritische Abläufe identifizieren.

3.3.1 Journey 1: Frage stellen.

Die Frage wird vom Frontend per REST an das Backend übermittelt, dort validiert, in der Datenbank persistiert (wobei ein Trigger eine Kommentarnummer generiert) und über RabbitMQ als Echtzeit-Update an alle Raum-Clients weitergeleitet. Der Ablauf durchläuft vier Komponenten. Messgrößen sind Backend-Antwortzeit, Dauer des DB-Inserts und WebSocket-Zustelllatenz.

3.3.2 Journey 2: Live-Voting.

Votes werden per REST gespeichert. Ein Trigger aktualisiert synchron die Score-Spalte der zugehörigen Frage. Bei parallelen Abstimmungen kann es zu konkurrierenden Schreibzugriffen auf dieselbe Datenbankzeile kommen, wodurch Transaktionen aufgrund von Sperrmechanismen aufeinander warten müssen (Row-Level Contention). Unter Spitzenlast zu steigenden Antwortzeiten führen kann (vgl. Abschnitt 3.4, R2).

3.3.3 Journey 3: KI-Zusammenfassung.

Die Anfrage gelangt vom Frontend über das Backend an den AI Provider, der einen externen LLM-Dienst aufruft. Kein externer Aufruf im System hat einen konfigurierten Timeout. Ein nicht reagierender Upstream-Dienst kann daher Request-Laufzeiten unbegrenzt verlängern und die rechtzeitige Antwortlieferung an das Frontend gefährden.

Ergänzend kann synthetisches End-to-End-Monitoring diese Journeys von außen überwachen und Verfügbarkeit sowie Antwortverhalten aus Nutzersicht validieren (Niedermaier et al., 2019, S. 11).

3.4 Risikopunkte und Zuordnung zu den Golden Signals

Aus der Architekturanalyse lassen sich sechs Risikopunkte ableiten, die in Tabelle 3.2 den betroffenen Golden Signals und ihrer Nutzerwirkung zugeordnet werden.

3.4.1 R1: Fehlende Timeouts und Resilienz.

Weder Backend noch AI Provider konfigurieren Timeouts oder Retries auf ausgehenden HTTP-Aufrufen. Ein nicht reagierender externer Dienst kann Request-Laufzeiten unbegrenzt verlängern und die Ressourcenauslastung erhöhen. Niedermaier et al. (2019, S. 8–9) ordnen fehlende Trans-

parenz über Drittanbieter-Abhängigkeiten als zentrale betriebliche Herausforderung verteilter Systeme ein.

3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.

Die Vote-Trigger führen bei jedem Schreibvorgang einen zusätzlichen `SELECT` und `UPDATE` auf der Comment-Tabelle aus. Bei gleichzeitigen Abstimmungen entsteht hier ebenfalls Row-Level Contention. Die Kommentarnummern-Generierung skaliert zudem sublinear mit wachsender Kommentarzahl. Fehlende Indizes auf mehreren neueren Tabellen verschärfen das Problem.

3.4.3 R3: Backend als Kritischster Punkt ohne Observability.

Nahezu alle Nutzerinteraktionen laufen über das Backend, das gleichzeitig über keinerlei Observability-Infrastruktur verfügt. Es existieren weder anwendungsseitige Metriken noch Distributed Tracing oder strukturierte Logs. Fehler bleiben unsichtbar, bis Endnutzende Symptome wahrnehmen.

3.4.4 R4: WebSocket-Verbindungsverlust.

Für RabbitMQ existieren keine Health-Checks. Ein Broker-Ausfall unterbricht alle Echtzeit-Updates, bleibt aber ohne Überwachung der Queue-Tiefe und Subscription-Anzahl zunächst unbemerkt.

3.4.5 R5: Fehlende Ressourcenlimits.

Kein Container definiert CPU- oder Speicherlimits. Ein einzelner Dienst mit unkontrolliertem Ressourcenverbrauch kann durch Ressourcenverdrängung alle übrigen Dienste beeinträchtigen.

3.4.6 R6: Single-Host-Topologie als systemweiter Ausfallpfad.

Alle zehn Container laufen auf einem gemeinsamen Host. Ein Hardware- oder Betriebssystemausfall betrifft das Gesamtsystem. In Kombination mit R5 entsteht ein einzelner Sättigungspfad, über den ein Dienst alle übrigen destabilisieren kann.

Tabelle 3.2: Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung. Quelle: Eigene Darstellung.

Risiko	Dienst(e)	Signale	Nutzerwirkung
R1	Backend, AI Provider	Latency, Errors	Hängende Anfragen, Timeouts
R2	Backend, PostgreSQL	Latency, Saturation	Verzögerte Votes in Spitzenlast
R3	Backend	Alle	Fehler erst durch Nutzerbeschwerden sichtbar
R4	WS-Gateway, RabbitMQ	Errors, Traffic	Ausbleibende Echtzeit-Updates
R5	Alle Container	Saturation	Systemweite Instabilität
R6	Gesamtsystem	Saturation, Errors	Totalausfall bei Host-Ausfall

3.5 Observability-Ist-Zustand

Die Quellcode-Analyse ergibt, dass die bestehende Observability-Infrastruktur minimal ist. Das Backend besitzt weder Metriken, noch Tracing oder strukturierte Logs. Das WS-Gateway konfiguriert einen Health- und Prometheus-Endpoint, jedoch ist die benötigte Micrometer-Registry nicht als Abhängigkeit deklariert. RabbitMQ hat das Prometheus-Plugin aktiviert, der Port wird aber nicht exponiert. Systemweit fehlen Distributed Tracing, Korrelations-IDs, zentrale Log-Aggregation und ein Monitoring-Stack vollständig. Störungen werden gegenwärtig erst wahrgenommen, wenn Endnutzende direkt betroffen sind – ein Zustand, den Niedermaier et al. (2019, S. 9–10) als Folge einer rein reaktiven Monitoring-Implementierung einordnen.

3.6 Ableitung der Observability-Anforderungen

Aus Architektur, Kernabläufen, Risikopunkten und Ist-Zustand lassen sich fünf Anforderungen ableiten.

3.6.1 A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.

Die User Journeys durchlaufen jeweils mehrere Dienste. Kausale Zusammenhänge zwischen Störungen sind ohne dienstübergreifende Sichtbarkeit nicht erkennbar. Albuquerque und Correia (2025, S. 6) beschreiben *Distributed Tracing* als grundlegendes Entwurfsmuster für diese Anforderung. Jeder eingehenden Anfrage wird eine eindeutige ID zugewiesen, die über alle Dienste propagiert und in Logs sowie Spans mitgeführt wird, sodass der gesamte Ausführungspfad rekonstruierbar ist.

3.6.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals.

Die Instrumentierung muss auf drei Ebenen erfolgen: (1) Browser-Telemetrie im Frontend für Nutzererfahrungsmetriken, (2) Service-Metriken für Latenz, Durchsatz, Fehlerrate und Sättigung der vier Applikationsdienste und der Datenbank, sowie (3) Infrastruktur-Metriken für Container-Ressourcen, Verbindungspools und Queue-Tiefen. Diese mehrschichtige Betrachtung entspricht der von Albuquerque und Correia (2025, S. 10, 16, 22) beschriebenen Trennung zwischen *Application Metrics* und *Infrastructure Metrics*, deren Zusammenspiel eine ganzheitliche Sicht auf das System ermöglicht.

3.6.3 A3: Strukturiertes, korrelierbares Logging.

Das Logging muss auf strukturierte Formate (JSON) mit Korrelations-IDs umgestellt und zentral aggregiert werden, damit Logeinträge verschiedener Dienste einer gemeinsamen Anfrage zugeordnet werden können (Albuquerque & Correia, 2025, S. 9–10).

3.6.4 A4: Handlungsfähige Alarmierung.

Alarme sollen auf nutzerwirksame Zustände reagieren (etwa steigende Antwortzeiten, wachsende Fehlerraten oder nicht erreichbare Schnittstellen) und ausreichend Kontext für eine gezielte Erstreaktion enthalten. Formale SLOs existieren derzeit nicht und können als zukünftige Weiterentwicklung betrachtet werden.

3.6.5 A5: Beobachtbarkeit der Infrastrukturschicht.

Die Single-Host-Topologie (R6) und die fehlenden Ressourcenlimits (R5) erzeugen blinde Flecken, die nur durch Infrastruktur-Metriken sichtbar werden. Die Strategie muss Container-Ressourcenverbrauch, Datenbankverbindungspool-Auslastung, RabbitMQ-Queue-Tiefen und Dienstverfügbarkeit über Health-Endpunkte einbeziehen. Niedermaier et al. (2019, S. 10) formulieren eine solche ganzheitliche Sicht über System- und Infrastrukturebenen hinweg als Kernanforderung an Monitoring-Konzepte verteilter Systeme.

Diese Anforderungen bilden die Grundlage der folgenden Kapitel. A1–A3 adressieren primär Forschungsfrage 1 (Instrumentierung der Golden Signals), A2 und A5 liefern Evaluationskriterien für Forschungsfrage 2 (Stack-Auswahl), und A4 bildet die Grundlage für Forschungsfrage 3 (intelligentes Alert-System). Die Architektur, Nutzungspfade und Risikopunkte aus diesem Kapitel dienen dabei durchgängig als Referenzrahmen.

4 Methodisches Vorgehen und Bewertungsrahmen

Dieses Kapitel legt das methodische Vorgehen offen, mit dem aus den theoretischen Grundlagen (Kapitel 2) und der Systemanalyse (Kapitel 3) eine konkrete Observability-Strategie für frag.jetzt entsteht. Es beschreibt zunächst die Arbeitslogik der Arbeit, dann das Verfahren zur servicebezogenen Ableitung der Golden Signals und abschließend die Kriterien für Stack-Auswahl und Evaluation.

4.1 Arbeitslogik der Arbeit

Die Arbeit folgt einer konstruktiv-artefaktorientierten Arbeitslogik. Auf Basis der theoretischen Fundierung und der Systemanalyse werden Observability-Anforderungen abgeleitet. Darauf aufbauend wird eine Observability-Strategie entwickelt, in ausgewählten Komponenten prototypisch umgesetzt und anhand zuvor definierter Kriterien bewertet. Der Schwerpunkt liegt auf der systematischen Entwicklung und Beurteilung eines konkreten Lösungskonzepts für einen realen Anwendungskontext, nicht auf einem empirischen Forschungsdesign im engeren Sinne.

Die Arbeitsschritte gliedern sich in fünf aufeinander aufbauende Phasen. Die *theoretische Fundierung* (Kapitel 2) erarbeitet die begrifflichen Grundlagen. Die *Systemanalyse* (Kapitel 3) überführt diese in den Kontext von frag.jetzt und leitet fünf Observability-Anforderungen (A1–A5) ab. Die *Stack-Evaluation* (Kapitel 5) bewertet verfügbare Technologiekombinationen anhand der in Abschnitt 4.3 definierten Vergleichskriterien. Die *Strategiekonzeption mit prototypischer Umsetzung* (Kapitel 6 und 7) bildet den Kern der Eigenleistung: Hier werden die Four Golden Signals dienstspezifisch operationalisiert, eine Telemetrie-Pipeline entworfen sowie Dashboards und Alerting-Regeln konzipiert. Abschließend prüft die *Evaluation* (Kapitel 8) die Strategie gegen die Bewertungskriterien und die Anforderungen aus Kapitel 3.

Die Architekturhebung in Kapitel 3 stützt sich ergänzend auf eine KI-gestützte Quellcodeexploration mithilfe von GitHub Copilot. Das Werkzeug diente als Explorationshilfe, um Architekturübersichten, Controller-Strukturen und bestehende Konfigurationen aus der umfangreichen Codebasis systematisch zu extrahieren. Sämtliche architekturelevanten Befunde, insbesondere Serviceabhängigkeiten, Datenbankzugriffe, externe Anbindungen und vorhandene Monitoring-Konfigurationen, wurden anschließend durch manuelle Prüfung gegen Quellcode, Docker-Compose-Dateien und Konfigurationsartefakte validiert. GitHub Copilot fungierte dabei ausschließlich als unterstützendes Analysewerkzeug, nicht als eigenständiger Erkenntnisträger. Das Verfahren erfasst implizite Laufzeitabhängigkeiten und Konfigurationen außerhalb des versionierten Quellcodes nur eingeschränkt. Diese Grenzen werden bei der Anforderungsableitung berücksichtigt.

4.2 Vorgehen bei der Ableitung der Golden Signals pro Service

Die Four Golden Signals (Latency, Traffic, Errors und Saturation) bilden den heuristischen Rahmen für die servicebezogene Instrumentierung (Ewaschuk, 2017, Kap. *The Four Golden Signals*). Die Herausforderung liegt weniger in der Kenntnis der vier Kategorien als in ihrer konkreten Übersetzung auf heterogene Dienste mit unterschiedlichen Rollen und Kommunikationsmustern. Damit diese Übersetzung nachvollziehbar erfolgt, folgt die Arbeit einer fünfstufigen Ableitungslogik.

Im ersten Schritt wird die *Rolle des Dienstes im Gesamtsystem* bestimmt. Die funktionale Einordnung – ob Einstiegspunkt für Nutzerinteraktionen, zentrale Geschäftslogik, Datenhaltung oder Anbindung externer Ressourcen – beeinflusst maßgeblich, welche Beobachtungsgrößen aussagekräftig sind.

Der zweite Schritt identifiziert die *kritischen Interaktionen* des Dienstes. Hierzu dienen die User Journeys und Risikopunkte aus Kapitel 3 als Eingangsgröße. Metriken sollten an realen Nutzungspfaden ausgerichtet sein, nicht an abstrakten Komponentengrenzen (Niedermaier et al., 2019, S. 10–11).

Auf dieser Grundlage werden im dritten Schritt *geeignete Messgrößen* abgeleitet. Wo fachlich sinnvoll, wird pro Golden Signal mindestens eine primäre Messgröße bestimmt. Ergänzend können Logs, Traces oder Black-Box-Indikatoren herangezogen werden. Nicht jedes Signal lässt sich bei jedem Dienst gleichermaßen als White-Box-Metrik erfassen. Bei externen KI-Services etwa ist Saturation nur indirekt über Antwortzeiten und Timeout-Raten beobachtbar. Ewaschuk (2017, Kap. *Symptoms Versus Causes*) betont die Unterscheidung zwischen symptomorientierten und ursachenorientierten Beobachtungsgrößen. Für die Alarmierung sind Symptome entscheidend, während Ursachenindikatoren primär als Debugging-Kontext dienen. Albuquerque und Correia (2025, S. 10, 16) differenzieren zudem zwischen *Application Metrics* und *Infrastructure Metrics* als komplementären Entwurfsmustern, deren Zusammenspiel erst eine vollständige Sicht auf das Verhalten eines Dienstes ermöglicht (Albuquerque & Correia, 2025, S. 22).

Der vierte Schritt begründet *initiale Ziel- und Schwellenwerte*. Da für frag.jetzt keine historischen Produktivdaten vorliegen, werden zunächst operative Startschwellen auf Basis von Lasttests und Erfahrungswerten festgelegt. Diese dienen als Ausgangspunkt für begründete Zielwerte, die zunächst in einer Staging-Umgebung anhand definierter Last- und Störungsszenarien überprüft und bei Bedarf angepasst werden. Formale Service Level Objectives, wie sie im SRE-Kontext als quantifizierbare Zielvorgaben die Instrumentierung und Alarmierung strukturieren (Hallur, 2024, S. 605), existieren für frag.jetzt derzeit nicht und werden als zukünftige Weiterentwicklung betrachtet. Niedermaier et al. (2019, S. 9) bestätigen, dass unklare nichtfunktionale Anforderungen und eine reaktive Monitoring-Entwicklung zu den häufigsten Herausforderungen verteilter Systeme gehören. Ein Ausgangszustand, der auch auf frag.jetzt zutrifft.

Im fünften Schritt wird bestimmt, welche *Visualisierung und Alarmierung* aus den Messgrößen

folgen. Nicht jede Metrik rechtfertigt einen eigenen Alarm. Alarme sollten auf nutzerseitig wahrnehmbare Symptome ausgerichtet sein, während interne Ursachenindikatoren als Debugging-Kontext bereitstehen (Vinnakota & Kolla, 2025, S. 175). Ebenso bestimmt die Zielgruppe die Aufbereitung. Eine p99-Latenzverteilung eignet sich für ein DevOps-Dashboard, während ein Product Owner eher aggregierte Verfügbarkeitsindikatoren benötigt.

Diese Logik stellt sicher, dass die Signal-Ableitung in Kapitel 6 nachvollziehbar an die Systemanalyse rückgebunden bleibt.

4.3 Kriterien für Stack-Auswahl und spätere Evaluation

Die Arbeit verwendet zwei aufeinander abgestimmte, aber bewusst getrennte Kriterienebenen. Die Stack-Auswahl (Kapitel 5) beantwortet die Werkzeugfrage, welche Technologiekombination die Anforderungen am besten adressiert. Die Evaluation (Kapitel 8) beantwortet die Ergebnisfrage, ob die entwickelte Strategie tatsächlich die identifizierten Probleme löst.

4.3.1 Vergleichskriterien für die Stack-Auswahl

Eine systematische Stack-Auswahl erfordert mehr als bloße Feature-Listen. Bei der Wahl von Observability-Werkzeugen müssen Faktoren wie Datenaufnahmeraten, Abfrageleistung, Speicherkosten und Integrationsfähigkeit berücksichtigt werden (Sakinala, 2025, S. 104). Faseeha et al. (2025, S. 72015) schlagen eine thematische Taxonomie vor, die Werkzeuge nach Einsatzzweck, erfassten Parametern und Architekturmustern vergleichbar macht. Giamattei et al. (2024, S. 3) operationalisieren einen ähnlichen Ansatz, indem sie funktionale und technologische Merkmale systematisch gegenüberstellen.

Aufbauend auf diesen Vorarbeiten und den Anforderungen A1–A5 werden folgende Vergleichskriterien verwendet:

- **Abdeckung der Telemetriepfeiler.** Unterstützung von Metriken, Logs und Traces in integrierter Form.
- **Korrelationsfähigkeit.** Verknüpfung verschiedener Telemetriearten über gemeinsame Bezeichner (Trace-IDs, Korrelations-IDs).
- **OpenTelemetry-Kompatibilität.** Unterstützung des herstellerneutralen CNCF-Standards für Instrumentierung und Datenexport, um Vendor Lock-in zu vermeiden.
- **Dashboarding und Alerting.** Eignung des Stacks für die Erstellung rollenspezifischer Dashboards sowie für die Definition und Verwaltung differenzierter Alarmregeln.
- **Integrationsaufwand.** Technischer Aufwand für die Einbindung in die bestehende Docker-Compose-Topologie und die vorhandenen Technologien (Spring Boot, Kotlin/Netty, Python/FastAPI).
- **Ressourcenbedarf.** Zusätzlicher Speicher-, CPU- und Festplattenbedarf auf einem einzelnen Host, besonders relevant angesichts der Single-Host-Topologie von frag.jetzt (vgl. R6).

- **Erweiterbarkeit.** Anpassungsfähigkeit bei steigenden Anforderungen, etwa wachsendem Telemetriedatenvolumen oder zusätzlichen Diensten.
- **Community und Ökosystem.** Breite der Nutzerbasis, Dokumentationsreife und Integrationen mit gängigen Frameworks.

Die Gesamtbewertung eines Stacks wird abschließend unter dem Gesichtspunkt der *Eignung für frag.jetzt* zusammengeführt. Dabei fließen die Einzelkriterien gewichtet in eine Synthese ein, die den spezifischen Kontext (Single-Host-Betrieb, begrenztes Betriebsteam, heterogene Technologielandschaft) explizit berücksichtigt.

4.3.2 Bewertungskriterien für die Gesamtstrategie

Die Gesamtstrategie wird an einem übergeordneten Maßstab gemessen, der direkt auf die Anforderungen A1–A5 aus Kapitel 3 zurückgreift. Niedermaier et al. (2019, S. 9–10) formulieren eine ganzheitliche Sicht über System- und Infrastrukturebenen hinweg als Kernanforderung an Monitoring-Konzepte verteilter Systeme. Diese Perspektive wird hier aufgegriffen und in prüfbare Evaluationsfragen überführt.

Für jede Anforderung wird eine konkrete Prüffrage formuliert: Lässt sich ein Request Ende-zu-Ende rekonstruieren (A1)? Deckt die Instrumentierung alle drei Messebenen ab (A2)? Können Logeinträge über Korrelations-IDs verknüpft werden (A3)? Reagieren Alarmer auf nutzerwirksame Zustände und enthalten sie ausreichend Kontext für eine Erstreaktion (A4)? Werden Container-Ressourcen, Datenbankpool-Auslastung und Queue-Tiefen erfasst (A5)? Für die Alarmierungsqualität betonen Gowda und Hameed (2022, S. 3), dass ein Alarm nur dann operativen Wert besitzt, wenn er Problem, Schweregrad, betroffene Komponenten und empfohlene Gegenmaßnahmen klar benennt.

Die Beantwortung dieser Fragen stützt sich auf drei Evidenzquellen: erstens die *Artefaktanalyse*, bei der Instrumentierungskonfigurationen, Dashboard-Definitionen und Alarmregeln gegen die Anforderungen geprüft werden. Zweitens die *Konfigurations- und Instrumentierungsprüfung*, die sicherstellt, dass die definierten Metriken, Logs und Traces tatsächlich erhoben und korrekt korreliert werden. Drittens ausgewählte *Last- und Störungsszenarien*, die in einer produktionsnahen Testumgebung gezielt Lastspitzen und Fehlerbilder simulieren.

Ergänzend fließen drei übergreifende Bewertungsdimensionen ein: die *Stakeholder-Tauglichkeit*, also ob Dashboards den unterschiedlichen Rollen die jeweils benötigten Informationen liefern, die *Abdeckung der Risikopunkte* R1–R6 aus Kapitel 3 und der *Umsetzungsrealismus*, also ob die Strategie mit den verfügbaren Mitteln und der bestehenden Infrastruktur implementierbar ist.

Die Evaluation in Kapitel 8 gleicht diese Kriterien mit den Ergebnissen der prototypischen Umsetzung ab. Zu diesem Zweck werden in einer produktionsnahen Staging-Umgebung gezielt Lastspitzen und Fehlerszenarien erzeugt, um die Instrumentierung, Korrelation und Alarmierungslogik kontrolliert zu prüfen. Die Aussagekraft dieser Validierung bleibt gegenüber Beobach-

tungen aus einem langfristigen Produktivbetrieb begrenzt und wird in Kapitel 8 entsprechend eingeordnet.

5 Evaluation und Auswahl des Observability-Stacks

5.1 Festlegung der Vergleichskriterien

Die acht Vergleichskriterien für die Stack-Auswahl wurden in Abschnitt 4.3 aus den Anforderungen A1–A5 und der einschlägigen Literatur hergeleitet. Sie umfassen die Abdeckung der drei Telemetriepfeiler, Korrelationsfähigkeit, OpenTelemetry-Kompatibilität, Dashboarding und Alerting, Integrationsaufwand, Ressourcenbedarf, Erweiterbarkeit sowie Community und Ökosystem. Im Kern adressieren die Anforderungen drei Kernbedürfnisse, nämlich eine durchgängige Nachvollziehbarkeit dienstübergreifender Abläufe (A1), eine mehrstufige Messbarkeit vom Infrastruktur- bis zum Anwendungsniveau (A2) sowie ein korrelierbares, strukturiertes Logging (A3). Hinzu kommen die Betriebsrestriktionen einer Single-Host-Topologie (A4) und das Erfordernis, alle Stakeholder-Rollen mit handlungsorientierten Informationen zu versorgen (A5). Das vorliegende Kapitel wendet diese Kriterien auf drei vollständige Observability-Architekturen an, um Forschungsfrage 2 zu beantworten: Welche Observability-Stack-Kombination eignet sich am besten für die Überwachungsanforderungen von frag.jetzt?

5.2 OpenTelemetry als verbindliche Instrumentierungsschicht

Bevor ein Vergleich der Analyse- und Visualisierungs-Backends sinnvoll ist, muss die vorgelagerte Instrumentierungsschicht festgelegt werden. Bereits in Kapitel 2 wurde OpenTelemetry als herstellernerutraler CNCF-Standard eingeordnet, der die Erzeugung und Erfassung von Logs, Metriken und Traces vereinheitlicht, ohne selbst Speicherung oder Darstellung zu übernehmen (Faseeha et al., 2025, S. 72030). Für die vorliegende Arbeit wird OpenTelemetry als verbindliche Instrumentierungsgrundlage festgelegt. Die Begründung folgt aus drei Überlegungen.

Erstens entkoppelt eine standardisierte Instrumentierung die Entscheidung über den Applikationscode von der Entscheidung über das Backend. Sakinala (2025, S. 107) beschreiben den OpenTelemetry Collector als flexiblen, herstellerunabhängigen Verarbeitungsbaustein, der Telemetriedaten aus verschiedenen Quellen entgegennimmt, transformiert und an beliebige Zielsysteme weiterleitet. Sollte sich ein Backend künftig als ungeeignet erweisen, lässt sich die Analyse-Ebene austauschen, ohne die Instrumentierung im Anwendungscode anzupassen. Diese Austauschbarkeit mindert das Risiko einer Herstellerbindung. Ein Vorteil, den Sakinala (2025, S. 107) der herstellernerutralen Architektur von OpenTelemetry explizit zuschreiben.

Zweitens profitiert frag.jetzt als polyglotte Anwendung besonders von einem sprachübergreifenden Instrumentierungsansatz. Backend und WS-Gateway nutzen Java beziehungsweise Kotlin, der AI Provider basiert auf Python, und das Frontend ist in TypeScript geschrieben. OpenTelemetry stellt für alle vier Laufzeitumgebungen SDKs bereit, die einheitliche Semantikkonventionen und Datenmodelle verwenden (Sakinala, 2025, S. 106). Ohne einen solchen gemeinsamen Rahmen

müssten für jede Sprache separate Bibliotheken gepflegt werden. Ein Aufwand, der bei begrenzten Betriebsressourcen schnell unwirtschaftlich wird. Giamattei et al. (2024, S. 15) empfehlen, Werkzeuge zu bevorzugen, deren Instrumentierung zum Entwicklungstempo des Projekts passt; die automatischen Instrumentierungsbibliotheken, die OpenTelemetry für gängige Frameworks bereitstellt, kommen dieser Anforderung entgegen.

Drittens sichert die Trägerschaft durch die Cloud Native Computing Foundation eine langfristige Weiterentwicklung und breite Herstellerunterstützung (Sakinala, 2025, S. 107). Der folgende Vergleich setzt OpenTelemetry daher als gegebene Instrumentierungsschicht voraus und bewertet die Kandidaten-Stacks ausschließlich in ihrer Rolle als Empfangs-, Speicher-, Visualisierungs- und Alarmierungsebene.

5.3 Vergleich der Zielarchitekturen

Einzelne Werkzeuge wie Prometheus allein oder der klassische ELK-Stack ohne APM decken jeweils nur einen der drei Telemetriepfeiler vollwertig ab und kommen daher als alleinstehende Endkandidaten für Forschungsfrage 2 nicht in Betracht, was auch die Fachliteratur bestätigt. Giamattei et al. (2024, S. 1, 5) führen Prometheus, Jaeger und Elasticsearch als komplementäre Werkzeuge, die typischerweise kombiniert werden; Garimilla (2024, S. 158–159) ordnen diese Werkzeuge explizit den Rollen Metrikerfassung, Tracing und Loganalyse zu, während Grafana die übergreifende Visualisierung übernimmt. Für den Vergleich werden deshalb ausschließlich Architekturen herangezogen, die Logs, Metriken und Traces vollständig adressieren und über eine gemeinsame Auswertungsebene verfügen. Alle drei Kandidaten basieren auf quelloffenen Kernkomponenten und werden in der Fachliteratur zur Microservices-Beobachtbarkeit regelmäßig herangezogen (Faseeha et al., 2025, S. 72019–72021).

5.3.1 Elastic Observability

Elastic Observability erweitert den klassischen ELK-Stack um die APM-Komponente und OTel-Integration und adressiert damit alle drei Telemetriepfeiler innerhalb eines Hersteller-ökosystems. Der APM-Server unterstützt den Empfang von Traces, Metriken und Logs über das OpenTelemetry-Protokoll (OTLP) nativ (Elastic, n. d.). Elasticsearch bildet als verteilte Such- und Analysemaschine den Kern und indiziert sämtliche eingehenden Dokumente im Volltext (Banerjee & Singh, 2024, S. 918). Kibana stellt die Visualisierungs- und Analyse-schnittstelle bereit; die Trace-Log-Korrelation erfolgt innerhalb der Kibana-Oberfläche, während Metrik-Auswertungen über Lens-Visualisierungen angebunden werden (Elastic, n. d.).

Die Stärke des Ansatzes liegt in der tiefgreifenden Loganalyse. Unstrukturierte Textdaten lassen sich mit hoher Geschwindigkeit durchsuchen, filtern und korrelieren (Banerjee & Singh, 2024, S. 920). Für den Kontext von frag.jetzt ergeben sich allerdings gewichtige Nachteile. Der Ansatz, sämtliche Logzeilen vollständig zu indizieren, verursacht erheblichen Speicher- und Rechenaufwand (Banerjee & Singh, 2024, S. 918). Auf der Single-Host-Topologie von

frag.jetzt konkurriert ein Elasticsearch-Knoten mit seinem JVM-Heap unmittelbar mit den Anwendungscontainern um CPU und Arbeitsspeicher. Hinzu kommt die Lizenzentwicklung. Elasticsearch und Kibana wurden 2021 von Apache 2.0 auf die Elastic License v2 und SSPL umgestellt; seit 2024 steht der Quellcode zusätzlich unter AGPL zur Verfügung (Elastic, 2024). Aus Sicht des Autors schränkt die mehrfache Lizenzänderung die langfristige Planungssicherheit im Vergleich zu CNCF-getragenen Projekten ein, auch wenn die aktuelle AGPL-Option den Zugang wieder erleichtert.

5.3.2 Grafana-Stack: Prometheus + Loki + Tempo + Grafana

Die zweite Zielarchitektur kombiniert Prometheus für Metriken, Loki für Logs und Tempo für Distributed Traces mit Grafana als gemeinsamer Visualisierungs- und Alarmierungsplattform. Gudimetla (2025, S. 501) beschreiben eine vergleichbare Referenzarchitektur, in der ein OpenTelemetry Collector als zentrale Routing-Schicht Telemetriedaten an Prometheus, Loki und einen Trace-Speicher weiterleitet, während Grafana die integrierte Visualisierung aller drei Datenquellen übernimmt.

Loki verfolgt bei der Log-Aggregation einen ressourcenschonenden Ansatz. Statt den vollständigen Inhalt jeder Logzeile zu indizieren, indiziert Loki ausschließlich Labels, Metadaten wie Dienstname, Umgebung oder Log-Level (Faseeha et al., 2025, S. 72019–72020). Dieser Mechanismus, konzeptionell an das Label-Modell von Prometheus angelehnt (Usman et al., 2022, S. 86911), reduziert den Speicher- und Verarbeitungsbedarf erheblich. Gudimetla (2025, S. 502) beziffern die Speicherersparnis auf 50–80 % gegenüber volltextbasierter Indizierung, weisen zugleich aber darauf hin, dass die Abfrageflexibilität eingeschränkt ist. Suchanfragen ohne Label-Eingrenzung durchlaufen sämtliche zugehörigen Chunks und sind entsprechend langsamer. Die Grafana-Loki-Dokumentation ergänzt, dass zu viele dynamische Labels die Zahl der Streams erhöhen und zu erheblichen Leistungseinbußen führen können (Grafana Labs, n. d. c). Für frag.jetzt, dessen Logvolumen moderat ausfällt und dessen Label-Struktur durch die begrenzte Zahl von Diensten überschaubar bleibt, überwiegen die Vorteile gegenüber der Vollindizierung.

Tempo empfängt Trace-Daten über standardisierte Protokolle, darunter OTLP sowie Jaeger- und Zipkin-Formate (Tzanettis et al., 2022, S. 8). Die Grafana-Tempo-Dokumentation beschreibt Tempo als Backend, das Trace-Daten speichert und deren Verknüpfung mit Logs und Metriken innerhalb von Grafana ermöglicht (Grafana Labs, n. d. b). Sundström (2025, S. 16) bestätigen diese Architektur in einer prototypischen Implementierung und beschreiben Grafana als Oberfläche, die alle Observability-Signale vereint und die Navigation zwischen Metriken, Traces und Logs erlaubt.

Prometheus übernimmt die Metrikerfassung und -speicherung. Auch ohne ein zusätzliches Langzeitspeicher-Backend wie Mimir ergibt sich eine zentrale Auswertungsoberfläche für alle drei Signaltypen, da Grafana Prometheus-, Loki- und Tempo-Datenquellen nativ unterstützt (Sundström, 2025, S. 16–17). Mimir ist daher nicht Voraussetzung für eine einheitliche Oberflä-

che, sondern ein Prometheus-kompatibles, skalierbares Langzeitspeicher- und Abfragebackend (Grafana Labs, n. d. a). Gudimetla (2025, S. 502) beschreiben Prometheus als Lösung für Datenerfassung und Kurzzeitspeicherung, während Mimir, Thanos oder Cortex die horizontale Skalierung und Langzeitvorhaltung übernehmen. Für die aktuelle Skalierungsstufe von frag.jetzt ist die lokale Prometheus-TSDB für den Prototyp ausreichend. Mimir bleibt als evolutive Ausbauoption bestehen.

5.3.3 Prometheus + Jaeger + Loki + Grafana

Die dritte Zielarchitektur verfolgt einen Bausteinansatz aus jeweils eigenständigen, ausgereiften Open-Source-Werkzeugen. Prometheus übernimmt die Metriken, Jaeger das Distributed Tracing und Loki die Log-Aggregation, visualisiert über Grafana. Alle drei Backend-Komponenten lassen sich in eine OpenTelemetry-basierte Pipeline integrieren. Der OTel Collector leitet die jeweiligen Telemetriedaten an das zuständige Backend weiter (Gudimetla, 2025, S. 501). Prometheus und Jaeger sind graduated CNCF-Projekte. Giamattei et al. (2024, S. 15) heben die aktive Open-Source-Community beider Werkzeuge als langfristigen Stabilitätsfaktor hervor. Jaeger bietet eine dedizierte Trace-Oberfläche, die in der Literatur häufig als Referenz herangezogen wird (Faseeha et al., 2025, S. 72020). Faseeha et al. (2025, S. 72019) führen Prometheus, Loki und Jaeger als verbreitete Werkzeuge für Microservices-Observability auf, was zeigt, dass diese Kombination keine exotische Eigenkonstruktion darstellt.

Dem Vorteil der Komponentenreife steht eine erhöhte Integrationskomplexität gegenüber. Die drei Werkzeuge folgen unterschiedlichen Konfigurationsformaten und Storage-Modellen. Prometheus nutzt eine lokale TSDB, Jaeger benötigt ein separates Speicher-Backend wie Elasticsearch, Cassandra oder Badger (Gudimetla, 2025, S. 501), und Loki folgt einem eigenen Chunk-Storage-Modell. Die Korrelation zwischen den Telemetriearten ist über Grafana grundsätzlich möglich. Sundström (2025, S. 17) weisen jedoch darauf hin, dass Jaeger und Zipkin typischerweise als separate Trace-Backends dienen, die über zusätzliche Konfiguration in Grafana eingebunden werden. Die Verknüpfung bleibt damit nach Einschätzung des Autors weniger eng integriert als in einer Architektur, deren Komponenten von vornherein aufeinander abgestimmt sind.

5.4 Vergleich und Synthese

Tabelle 5.1 fasst die Bewertung der drei Zielarchitekturen entlang der acht Vergleichskriterien zusammen. Die Skala reicht von 1 (eingeschränkt) über 2 (befriedigend) bis 3 (sehr gut). Die Einstufungen beruhen auf der qualitativen Analyse in den vorherigen Abschnitten und den in Kapitel 4.3 formulierten Kriterienausprägungen. Sie stellen eine argumentativ begründete Einschätzung des Autors dar, keine empirisch erhobenen Messwerte.

Bei den Telemetriepfeilern und der OTel-Kompatibilität schneiden alle drei Kandidaten gleichwertig ab, was eine direkte Folge der Konstruktionsentscheidung ist, nur vollständige Architekturen zu vergleichen. Die Differenzierung ergibt sich in den übrigen Kriterien. Elastic Observability

Tabelle 5.1: Vergleichende Bewertung der drei Observability-Zielarchitekturen (3 = sehr gut, 2 = befriedigend, 1 = eingeschränkt). Alle drei Kandidaten adressieren Logs, Metriken und Traces. OpenTelemetry wird als gemeinsame Instrumentierungsschicht vorausgesetzt. Quelle: Eigene Darstellung.

Kriterium		Elastic Observability (Elastic APM, ES, Kibana)		Grafana-Stack (Prom., Loki, Tempo, Grafana)		Prom./Jaeger/Loki (Prom., Jaeger, Loki, Grafana)
Telemetrie- pfeiler	3	Logs, Metriken und Traces über APM/OTel abgedeckt	3	Alle drei Pfeiler durch Prometheus, Loki und Tempo	3	Alle drei Pfeiler durch Einzelkomponenten
Korrelation	2	Trace-Log-Korrelation in Kibana; Metrik-Anbindung über Lens	3	Trace-Log-Metrik-Navigation nativ in Grafana	2	Via Grafana-Plugins, weniger eng integriert
OTel- Kompa- tibilität	3	OTLP-Empfang nativ, EDOT-Distributionen verfügbar	3	Alle Komponenten OTel-nativ	3	Alle Komponenten OTel-nativ
Dashboard. & Alerting	2	Kibana + Watcher/Rules; weniger flexibel als Grafana	3	Grafana Unified Alerting + Alertmanager	3	Grafana + Alertmanager
Integrations- aufwand	1	Logstash/Beats-Pipeline, APM-Server, JVM-Tuning	2	Mehr Container, aber homogene Konfiguration via OTel-Collector	1	Drei Konfig.-Formate und Storage-Backends
Ressourcen- bedarf	1	Elasticsearch speicherintensiv, JVM-Heap konkurriert mit Anwendung	2	Höher als Prom. allein, deutlich geringer als Elastic	2	Jaeger benötigt separates Speicher-Backend
Erweiter- barkeit	2	Nativ skalierbar, aber Cluster-Management komplex	3	Prometheus via Mimir erweiterbar; Loki/Tempo modular	3	Jede Komponente einzeln skalierbar
Community & Ökosys- tem	2	Große Community, aber Lizenzänderungen (ELv2/SSPL, seit 2024 teils AGPL)	2	Grafana Labs getragen, wachsend, Tempo jünger als Jaeger	3	Prom. u. Jaeger graduated CNCF, Loki etabliert
Summe		16		21		20

erzielt die niedrigste Gesamtbewertung, weil der hohe Ressourcenbedarf von Elasticsearch und der komplexe Integrationspfad auf einer Single-Host-Topologie besonders ins Gewicht fallen. Die beiden Grafana-basierten Architekturen liegen eng beieinander. Der Unterschied manifestiert sich in den Kriterien Korrelation und Integrationsaufwand.

Beide Grafana-basierten Architekturen ermöglichen eine durchgängige Sicht auf das Laufzeitverhalten, indem sie anwendungsbezogene Entwurfsmuster und infrastrukturnahe Messgrößen als komplementäre Perspektiven verbinden (Albuquerque & Correia, 2025, S. 22). Bissig (2024, S. 57) bestätigen, dass eine einheitliche, auf OpenTelemetry und Distributed Tracing gestützte Analyseperspektive in heterogenen verteilten Systemen die Voraussetzung für umfassende Observability darstellt. Die Entscheidung zwischen den beiden Grafana-basierten Varianten wird im folgenden Abschnitt anhand des konkreten Betriebskontexts begründet.

5.5 Begründete Zielentscheidung für frag.jetzt

5.5.1 Entscheidungsbegründung im Kontext von frag.jetzt

Das erste und schwerwiegendste Argument betrifft die *Integrationskomplexität bei begrenzten Betriebsressourcen*. Niedermaier et al. (2019, S. 9) identifizieren mangelnde Erfahrung, knappe zeitliche Kapazitäten und fehlende Ressourcen als eigenständige Herausforderung beim Aufbau von Monitoring-Konzepten in verteilten Systemen (C7). frag.jetzt wird von einem sehr kleinen Entwicklungsteam betrieben, das Observability erstmals einführt. Die Alternative aus Prometheus, Jaeger und Loki erfordert die Pflege dreier unterschiedlicher Konfigurationsformate und Storage-Backends (Gudimetla, 2025, S. 501). Der Grafana-Stack reduziert diese Heterogenität, da Loki und Tempo aus demselben Ökosystem stammen und vergleichbare Konfigurationsmuster verwenden. Sakinala (2025, S. 105) unterstreichen, dass Integrationskomplexität zwischen verschiedenen Werkzeugen eine der häufigsten Ursachen für Verzögerungen bei der Umsetzung von Observability-Vorhaben darstellt.

Das zweite Argument adressiert die *Korrelierbarkeit innerhalb einer einzigen Benutzeroberfläche*. Die Interviewstudie von Niedermaier et al. (2019, S. 8) dokumentiert das Fehlen einer zentralen, navigierbaren Gesamtsicht als wiederkehrendes Defizit (C4). Sundström (2025, S. 16) beschreiben Grafana im Kontext einer prototypischen Tracing-Implementierung als die Oberfläche, die alle Observability-Signale vereint und eine Navigation zwischen Metriken, Traces und Logs ermöglicht. Usman et al. (2022, S. 86914) ordnen eine solche einheitliche Sicht, die Logs, Traces und Metriken nahezu in Echtzeit zusammenführt, als Schlüsseleigenschaft einer leistungsfähigen Observability-Plattform ein. In der alternativen Werkzeugkombination wäre Jaeger als separates Trace-Backend über die Grafana-Oberfläche zwar einbindbar (Sundström, 2025, S. 17), die Verknüpfung bliebe jedoch weniger eng integriert, und die Jaeger-eigene Oberfläche würde als paralleles Werkzeug bestehen bleiben. Sakinala (2025, S. 109) betonen, dass kontextbezogene Dashboard-Implementierungen, die Telemetriedaten aus verschiedenen Quellen korrelieren, den Zugriff auf separate Werkzeuge während der Störungsbehandlung überflüssig machen und dadurch die Diagnosezeit verkürzen.

Das dritte Argument betrifft die *Austauschbarkeit bei stabilem Instrumentierungskern*. OpenTelemetry entkoppelt die Instrumentierung von der Backend-Wahl (vgl. Abschnitt 5.2). Sollte sich eine Komponente (z.B. Tempo) im Betrieb als unzureichend erweisen, ließe sich der Trace-Export im OTel Collector auf Jaeger umleiten, ohne Applikationscode anzupassen. Ebenso lässt sich die Metrikspeicherung bei wachsendem Datenvolumen von der lokalen Prometheus-TSDB auf Mimir als skalierbares Langzeit-Backend umstellen (Gudimetla, 2025, S. 502). Diese Flexibilität ist für Organisationen mit begrenzter Monitoring-Erfahrung besonders relevant, deren nichtfunktionale Anforderungen sich häufig erst im laufenden Betrieb konkretisieren (Niedermaier et al., 2019, S. 9–10).

5.5.2 Kompromisse und Einschränkungen

Die Entscheidung für den Grafana-Stack geht mit bewussten Kompromissen einher. Tempo ist ein jüngeres Projekt als Jaeger, das als graduated CNCF-Projekt über einen breiteren Reifegrad verfügt (Giamattei et al., 2024, S. 15). Die Community um Tempo ist nach Einschätzung des Autors kleiner und die öffentlich dokumentierte Lösungsbasis für Randprobleme weniger umfangreich.

Ebenso verzichtet die gewählte Architektur bewusst auf Mimir als Metrik-Backend. Prometheus speichert Zeitreihen lokal mit begrenzter Vorhaltezeit (Banerjee & Singh, 2024, S. 919). Für den aktuellen Umfang von frag.jetzt, wo moderates Telemetrevolumen, Single-Host-Betrieb und prototypische Umsetzung im Rahmen einer Bachelorarbeit gegeben sind, ist die lokale TSDB von Prometheus jedoch ausreichend. Gudimetla (2025, S. 502) beschreiben Mimir als Alternative, die horizontale Skalierung und Langzeitvorhaltung auf Objektspeicher ermöglicht. Diese Option bleibt als evolutiver Ausbaupfad bestehen.

Lokis labelbasierte Indexierung erfordert eine sorgfältige Auswahl der Labels, um eine unkontrollierte Vermehrung von Streams zu vermeiden (Grafana Labs, n.d. c). Da frag.jetzt aus einer überschaubaren Zahl von Diensten besteht, lässt sich dieses Risiko durch klare Konventionen beherrschen.

Diese Nachteile werden durch drei Faktoren relativiert. Erstens ist die offizielle Grafana-Labs-Dokumentation für die hier genutzten Standardkonfigurationen nach Einschätzung des Autors ausreichend. Zweitens überwiegen für ein sehr kleines Team die Vorteile einer homogenen Komponentenfamilie gegenüber der breiteren Dokumentationsbasis verstreuter Einzelwerkzeuge. Drittens stellt OpenTelemetry als Instrumentierungsschicht sicher, dass ein späterer Wechsel einzelner Backend-Komponenten mit vertretbarem Aufwand möglich bleibt.

Damit beantwortet die gewählte Kombination aus Prometheus, Loki, Tempo und Grafana (instrumentiert über OpenTelemetry) Forschungsfrage 2. Im weiteren Verlauf bildet sie die technische Grundlage für die Konzeption der Telemetrie-Pipeline (Kapitel 6) sowie die prototypische Umsetzung (Kapitel 7).

6 Konzeption der Observability-Strategie für frag.jetzt

Die vorangegangenen Kapitel haben den theoretischen Bezugsrahmen gelegt, den Ist-Zustand von frag.jetzt analysiert und den Observability-Stack begründet ausgewählt. Das vorliegende Kapitel überführt diese Vorarbeiten in eine zusammenhängende Strategie. Es beantwortet damit die Frage, *was* beobachtet werden soll, *warum* gerade diese Größen betrieblich bedeutsam sind und *nach welchen Prinzipien* die Lösung aufgebaut wird. Die technische Konfiguration, also das *Wie* im Detail, folgt in Kapitel 7.

Abschnitt 6.1 formuliert die Leitprinzipien, die alle weiteren Entwurfsentscheidungen strukturieren. Abschnitt 6.2 beschreibt die logische Zielarchitektur als Schichtenmodell. In den nachfolgenden Abschnitten werden dann die Four Golden Signals servicebezogen operationalisiert (6.3), in einer übergreifenden Signal-Matrix verdichtet (6.4), das rollenbasierte Informationsdesign erarbeitet (6.5), das Alerting-Konzept formuliert (6.6) und anomaliebasierte Erweiterungsperspektiven eingeordnet (6.7).

6.1 Leitprinzipien der Strategie

Eine Observability-Strategie, die lediglich eine Sammlung von Metriken und Dashboards bereitstellt, ohne ein erkennbares Ordnungsprinzip zu verfolgen, bleibt operativ wenig steuerbar. Die nachfolgend formulierten vier Prinzipien greifen die Anforderungen A1–A5 aus Kapitel 3 auf und übersetzen sie in Gestaltungsleitlinien, die für alle weiteren Abschnitte dieses Kapitels verbindlich gelten.

P1: Servicebezogene Messbarkeit. Jeder Dienst des frag.jetzt-Ökosystems, vom Spring-Boot-Backend über den AI Provider bis zur PostgreSQL-Datenbank, wird als eigenständige Beobachtungseinheit behandelt. Für jeden Dienst werden Messgrößen entlang der vier Golden Signals abgeleitet, anstatt sich auf globale Aggregationen zu beschränken. Dieses Vorgehen folgt der Empfehlung von Ewaschuk (2017, S. 7–8), Beobachtungsgrößen an den Schnittstellen der Dienste zu erheben, weil sich dort Symptome manifestieren, bevor tieferliegende Ursachen sichtbar werden. Zugleich trägt es der Forderung von Niedermaier et al. (2019, S. 11) Rechnung, Monitoring konsequent auf Dienstebene und nicht nur auf Infrastrukturebene zu verankern. Anforderung A2 wird damit unmittelbar adressiert.

P2: Ende-zu-Ende-Nachvollziehbarkeit. Die Systemanalyse hat gezeigt, dass ein Großteil der kritischen Nutzungspfade mehrere Dienste durchläuft und kein Distributed Tracing existiert (Risikopunkt R6). Die Literatur beschreibt die Übergabepunkte zwischen Diensten, die sogenannten *Edges*, als die Stellen, an denen Trace-Kontext propagiert werden muss, um kausal zusammenhängende Aktivitäten über Dienstgrenzen hinweg korrelieren zu können (Bissig, 2024, S. 32). Ohne diese kontexttragende Verknüpfung bleiben Logs und Metriken isolierte Daten-

punkte, aus denen sich keine Ende-zu-Ende-Diagnose ableiten lässt (Sundström, 2025, S. 14–15). Für frag.jetzt bedeutet das konkret, dass an jedem der in Abschnitt 3.2 identifizierten Dienstübergänge, etwa zwischen Backend und Datenbank oder zwischen Backend und AI Provider, ein Propagationsmechanismus vorgesehen werden muss. Das Prinzip greift Anforderung A1 und den Korrelationsaspekt von A3 auf.

P3: Rollenorientierte Aufbereitung. Telemetriedaten adressieren nicht nur Entwicklerinnen und Entwickler, die einen Stacktrace analysieren, sondern auch Betriebsteams, die den Host überwachen, und Verantwortliche, die den Dienststatus aus Produktsicht bewerten. Bissig (2024, S. 54) hält fest, dass unterschiedliche Stakeholder unterschiedliche Fragen an dasselbe Laufzeitverhalten stellen und die Aufbereitung der Daten diesen Bedürfnissen folgen muss. Die gemeinsame Basisfrage *“Funktioniert es?”* teilen dabei alle Rollen. Die Granularität der Antwort variiert. Für frag.jetzt wird dieses Prinzip in Abschnitt 6.5 in ein konkretes Informationsdesign mit drei Perspektiven überführt, die aus den in Abschnitt 3.4 identifizierten Stakeholder-Gruppen abgeleitet werden.

P4: Schrittweise Erweiterbarkeit. frag.jetzt wird von einem kleinen Entwicklungsteam betrieben, das Observability erstmals einführt. Niedermaier et al. (2019, S. 9–10) identifizieren knappe Ressourcen und fehlende Vorerfahrung als eigenständige Herausforderung beim Aufbau von Monitoring-Konzepten. Daraus folgt, dass die Strategie nicht als monolithisches Zielkonzept, sondern als erweiterbarer Ausgangspunkt angelegt sein muss. Konkret bedeutet das, dass die Instrumentierung über OpenTelemetry standardisiert wird, sodass Backend-Komponenten austauschbar bleiben (Gudimetla, 2025, S. 502). Exportpfade und Samplingstrategien lassen sich anpassen, ohne Anwendungscode zu verändern. Dieses Prinzip setzt die in Abschnitt 5.5 begründete Flexibilität des gewählten Stacks auf Strategieebene fort.

Die vier Prinzipien sind keine unabhängigen Einzelforderungen, sondern greifen ineinander. Servicebezogene Messbarkeit (P1) erzeugt erst dann eine belastbare Gesamtsicht, wenn die erhobenen Daten dienstübergreifend korrelierbar sind (P2). Die gewonnenen Erkenntnisse entfalten ihre betriebliche Wirkung, wenn sie rollenbezogen aufbereitet werden (P3). Und die dauerhafte Pflege des gesamten Aufbaus setzt voraus, dass er inkrementell weiterentwickelt werden kann (P4). Im folgenden Abschnitt wird die logische Architektur beschrieben, die diese Prinzipien in eine technische Struktur übersetzt.

6.2 Logische Zielarchitektur der Observability

Die Observability-Architektur für frag.jetzt folgt einem vierstufigen Schichtenmodell, das den Datenfluss von der Erzeugung bis zur Auswertung abbildet. Kosińska et al. (2023, S. 73037–73038) beschreiben eine vergleichbare Gliederung als grundlegendes Muster für cloud-native Anwendungen. Telemetriedaten werden in der Anwendung erzeugt, von einer Sammelschicht entgegengenommen, in spezialisierten Backends gespeichert und schließlich über Analyse- und

Visualisierungswerkzeuge ausgewertet. Die folgenden Unterabschnitte übertragen dieses Muster auf den konkreten Kontext von frag.jetzt und benennen für jede Schicht die strategische Funktion. Die werkzeugspezifische Konfiguration wird in Kapitel 7 dokumentiert.

6.2.1 Instrumentierungsschicht

Die Erzeugung von Telemetriedaten bestimmt, an welchen Stellen im System Metriken, Spans und Logeinträge entstehen. Sie bildet die Grundlage aller nachgelagerten Verarbeitungsschritte und zugleich die Schicht, deren Gestaltung den größten Einfluss auf die diagnostische Tiefe der gesamten Lösung hat.

Grundsätzlich lassen sich zwei Beobachtungsperspektiven unterscheiden. Bei der *White-Box-Instrumentierung* gibt die Anwendung ihren internen Zustand aktiv als Telemetriedaten preis, etwa durch Spans, strukturierte Logeinträge oder anwendungsspezifische Metriken (Ewaschuk, 2017, S. 6). Bei der *Black-Box-Beobachtung* wird eine Komponente ausschließlich von außen betrachtet, indem ein- und ausgehende Datenströme gemessen werden, ohne den Quellcode zu verändern (Bissig, 2024, S. 30). White-Box-Instrumentierung liefert entscheidende Vorteile, darunter die Anreicherung von Telemetriedaten mit internem Kontext und die Weitergabe von Trace-Identifikatoren für die dienstübergreifende Korrelation (Bissig, 2024, S. 31).

Aus der in Kapitel 3 rekonstruierten Architektur ergibt sich für frag.jetzt eine differenzierte Instrumentierungsstrategie. Backend, WebSocket Gateway und AI Provider sind als Eigenentwicklungen White-Box-instrumentierbar. Für diese intern kontrollierbaren Laufzeitkomponenten wird eine standardisierte Auto-Instrumentierung mit selektiver manueller Ergänzung vorgesehen. Die Auto-Instrumentierung erfasst gängige Einstiegspunkte wie eingehende Anfragen und ausgehende Aufrufe, während manuelle Ergänzungen dort greifen, wo fachliche Kontrollflüsse über die Standardpunkte hinausgehen. Die Quellcodeanalyse in Kapitel 3 hat gezeigt, dass keine dieser Komponenten bislang Telemetrie emittiert (R4). Demgegenüber agieren die externen LLM-Provider als Black-Box-Abhängigkeiten, deren Quellcode nicht verfügbar ist. In solchen Fällen empfiehlt die Literatur, zumindest die Ein- und Ausgangsseite jedes Aufrufs zu instrumentieren, um Antwortzeiten, Fehlerraten und Timeouts zu erfassen (Bissig, 2024, S. 31). Sundström (2025, S. 13) ordnet die Instrumentierung als erste von drei Phasen einer Tracing-Architektur ein und unterscheidet zwischen automatischer und manueller Instrumentierung, die je nach Systemanforderung kombiniert werden können.

OpenTelemetry übernimmt in der geplanten Architektur die Rolle der standardisierten Instrumentierungsschicht (vgl. Abschnitt 5.2). Ein wesentlicher Aspekt ist dabei die Kontextpropagation an den Übergabepunkten zwischen Diensten. Bissig (2024, S. 32) definiert diese Übergabepunkte als *Edges*, an denen Arbeit von einer Komponente an eine andere übergeben wird, und fordert, dass an jeder solchen Stelle Trace-Kontext in-band weitergereicht wird. OpenTelemetry unterstützt dies über den W3C-Trace-Context-Standard (Sundström, 2025, S. 15). Für frag.jetzt sind die relevanten Edges die Verbindungen zwischen Frontend und Reverse-Proxy, zwischen

Reverse-Proxy und Backend, zwischen Backend und Datenbank sowie zwischen Backend und AI Provider. Eine besondere Herausforderung stellt die Echtzeitkommunikation über den Message Broker dar, da dort die automatische Kontextpropagation nicht ohne Weiteres greift und eine manuelle Injektion des Trace-Kontexts in das Nachrichtenformat erforderlich wird (Sundström, 2025, S. 25). Die konkrete Umsetzung dieser Propagation wird in Kapitel 7 dokumentiert.

Neben der Anwendungsinstrumentierung ist die Infrastrukturebene zu berücksichtigen. Container-Metriken wie CPU-Auslastung und Speicherverbrauch sowie datenbankspezifische Indikatoren werden über spezialisierte Exporter erhoben, die Prometheus-kompatible Endpunkte bereitstellen (vgl. Abschnitt 5.5). Die Zusammenführung von Anwendungs- und Infrastrukturtelemetrie stellt in der Praxis nach wie vor eine offene Herausforderung dar (Bissig, 2024, S. 38). Für frag.jetzt wird diese Zusammenführung dadurch erleichtert, dass alle Container auf einem einzelnen Host laufen und über gemeinsame Labels identifizierbar sind.

6.2.2 Sammel- und Verarbeitungsschicht

Nach der Erzeugung müssen die erfassten Betriebs- und Diagnosedaten gesammelt, bei Bedarf transformiert und an die zuständigen Speicher-Backends weitergeleitet werden. Der OpenTelemetry Collector fungiert hier als zentrale Routing-Komponente (Sakinala, 2025, S. 107). Seine Architektur basiert auf drei Bausteinen. *Receivers* nehmen eingehende Telemetriedaten entgegen, *Processors* transformieren oder filtern sie, und *Exporters* leiten das Ergebnis an die nachgelagerten Backends weiter (Kosińska et al., 2023, S. 73038).

Gudimetla (2025, S. 502) beschreiben den Collector als die Schicht, die Instrumentierung und Backend entkoppelt. Diese Entkopplung ist strategisch bedeutsam, weil sie einen Backend-Wechsel ermöglicht, ohne die Instrumentierung in den Anwendungen anpassen zu müssen. Damit wird Leitprinzip P4 auf der Ebene der Datensammlung umgesetzt. Für frag.jetzt wird der Collector als pushbasierte Empfangskomponente für anwendungsseitige Traces und Metriken konfiguriert. Bissig (2024, S. 39) begründet die Bevorzugung eines Push-Modells gegenüber Pull-Mechanismen. In Umgebungen mit dynamischen Lebenszyklen von Containern kann eine rein pullbasierte Abfrage Telemetriedaten verpassen. Obwohl frag.jetzt als Docker-Compose-Deployment einen relativ stabilen Containersatz aufweist, wird dieses Muster beibehalten, weil es bei einer zukünftigen Erweiterung keine Umstellung erfordert. Infrastrukturmetriken folgen dagegen einem Pull-Modell. Prometheus fragt die Exporter-Endpunkte aktiv in konfigurierbaren Intervallen ab, was für Infrastrukturmetriken etabliert ist, weil deren Erzeugungsrate vom Scrape-Intervall und nicht von der Anwendungslast abhängt (Bissig, 2024, S. 33).

Eine besondere Entwurfsentscheidung betrifft das *Sampling*. Bei geringem Datenvolumen, wie es für frag.jetzt im Normalbetrieb zu erwarten ist, kann zunächst auf Sampling verzichtet und jede Anfrage vollständig aufgezeichnet werden. Bissig (2024, S. 40–41) unterscheidet kopfbasierte und endbasierte Samplingstrategien und ordnet sie nach ihren Trade-offs ein. Die Strategie für frag.jetzt sieht vor, im Prototyp ohne Sampling zu beginnen und eine Anpassung erst dann

vorzunehmen, wenn das Telemetrievolumen die Speicherkapazität des Hosts belastet.

6.2.3 Speicherschicht

Die aufbereiteten Signale werden in drei spezialisierten Backends abgelegt, die jeweils für einen Datentyp optimiert sind. Diese Trennung folgt der in Kapitel 5 begründeten Architekturentscheidung für den LGTM-Stack. Metriken werden in der lokalen Time-Series-Datenbank von Prometheus gespeichert. Für das moderate Telemetrievolumen von frag.jetzt ist die lokale Vorhaltung mit einer begrenzten Retention ausreichend (vgl. Abschnitt 5.5). Logs werden in Loki abgelegt, das ausschließlich Metadaten-Labels indexiert und nicht den vollständigen Loginhalt, ein Ansatz, der die Speicherkosten gegenüber volltextindexierten Alternativen erheblich reduziert, bei dem Abfragen aber stets über Labels eingegrenzt werden müssen (Gudimetla, 2025, S. 502). Traces werden in Tempo persistiert, das für die Aufnahme von Span-Daten konzipiert ist.

Entscheidend für die diagnostische Wirksamkeit ist jedoch nicht die Wahl des einzelnen Speicher-Backends, sondern die Korrelierbarkeit über die drei Speicher hinweg. Bissig (2024, S. 44) diskutiert die Frage, ob Telemetriedaten in getrennten oder in einem gemeinsamen Speicher abgelegt werden sollten, und stellt fest, dass eine getrennte Speicherung die Optimierung pro Datentyp ermöglicht, die Korrelation aber nicht automatisch mitliefert, sodass sie durch zusätzliche Integrationsarbeit bewusst hergestellt werden muss. Für den gewählten Stack geschieht dies über drei Mechanismen, die zusammen die diagnostischen Brücken zwischen den Speichern bilden. Erstens transportiert die Trace-ID als gemeinsamer Identifikator den Kontext über alle drei Pfeiler. Zweitens erlaubt die Injektion der Trace-ID in den Mapped Diagnostic Context (MDC) des Logging-Frameworks, dass jeder Logeintrag automatisch die zugehörige Trace-ID enthält (Sundström, 2025, S. 20). Drittens ermöglichen Metriken-Exemplars, also Einzelverweise von einem aggregierten Metrikwert auf einen konkreten Trace, den direkten Übergang von einer auffälligen Metrik zum zugehörigen Trace (Bissig, 2024, S. 25); (Albuquerque & Correia, 2025, S. 12).¹ Für frag.jetzt folgt daraus, dass die Speichertrennung nur dann diagnostisch tragfähig ist, wenn diese Verknüpfungspfade von Beginn an eingeplant und in der Instrumentierung verankert werden.

6.2.4 Analyse- und Visualisierungsschicht

Grafana bildet die einheitliche Oberfläche, über die alle gespeicherten Telemetriedaten abgefragt, visualisiert und in Alarme überführt werden. Sundström (2025, S. 16) beschreibt Grafana als die Plattform, die Metriken, Traces und Logs in einer navigierbaren Oberfläche zusammenführt. Die Interviewstudie von Niedermaier et al. (2019, S. 8) identifiziert das Fehlen einer solchen

¹ Die konkrete Umsetzung dieser Korrelationsmechanismen, insbesondere die Konfiguration von Trace-to-Logs, Trace-to-Metrics und Exemplars in Grafana, basiert auf den Funktionalitäten des LGTM-Stacks und wird in Kapitel 7 anhand der offiziellen Grafana-, Tempo- und Loki-Dokumentation dokumentiert.

navigierbaren Gesamtsicht als wiederkehrendes Defizit in der Praxis. Für frag.jetzt stellt Grafana die zentrale Auswertungsschicht dar, in der die servicebezogenen Dashboards (Abschnitt 6.5) und die Alerting-Regeln (Abschnitt 6.6) verortet werden.

Die Korrelation wird in Grafana über Datenquellen-übergreifende Navigation hergestellt. Von einem auffälligen Metrikpunkt lässt sich über ein Exemplar direkt zum zugehörigen Trace springen. Aus einem Trace heraus führen verknüpfte Logabfragen zum relevanten Kontext. Diese Navigationspfade konkretisieren die in Abschnitt 5.5 begründete Entscheidung für einen eng integrierten Stack und setzen Leitprinzip P2 auf der Auswertungsebene um.

6.2.5 Einordnung im Kontext von frag.jetzt

Die beschriebene Vier-Schichten-Architektur orientiert sich an Referenzmodellen, die vorrangig für Kubernetes-basierte Umgebungen mit horizontaler Skalierung konzipiert sind (Gudimetla, 2025, S. 501). Im Unterschied zu solchen generischen Referenzarchitekturen weicht frag.jetzt in drei wesentlichen Punkten davon ab, was eigenständige Entwurfsentscheidungen erfordert.

Erstens läuft das gesamte Ökosystem auf einem einzelnen Host ohne Orchestrierung durch Kubernetes. Die Collector-Instanz wird daher nicht als Sidecar pro Container, sondern als eigenständiger Container im selben Docker-Compose-Verbund betrieben.

Zweitens fehlt ein Service Mesh, das in Kubernetes-Umgebungen häufig die automatische Kontextpropagation und Metrikerhebung auf Netzwerkebene übernimmt. Für frag.jetzt bedeutet das, dass die Telemetrieerzeugung vollständig auf Anwendungsebene erfolgen muss, über die in Abschnitt 6.2.1 beschriebene Instrumentierungsstrategie.

Drittens ist das erwartete Telemetrevolumen im Normalbetrieb gering genug, um auf Sampling zu verzichten und alle drei Speicher-Backends ohne vorgelagerte Aggregation zu betreiben. Diese Eigenschaft vereinfacht den initialen Betrieb erheblich, erfordert aber eine bewusste Kapazitätsbeobachtung, sobald frag.jetzt in größerem Umfang eingesetzt wird.

Borges et al. (2024, S. 1–2) argumentieren, dass Instrumentierung und Konfiguration einer Observability-Lösung werkzeugabhängig und an Kosten gebunden sind und Architekten deshalb die zugehörigen Entwurfs-Trade-offs verstehen müssen. Die Literatur liefert dafür den Rahmen. Die konkrete Priorisierung erfolgt jedoch anhand der in Abschnitt 3.4 identifizierten Risiken. Die vorliegende Architektur hält den Einstieg bewusst schlank und lässt Erweiterungspfade, etwa das Hinzufügen von Mimir für Langzeitmetriken oder die Aktivierung von Tail-Based Sampling im Collector, offen, ohne die Grundstruktur zu verändern.

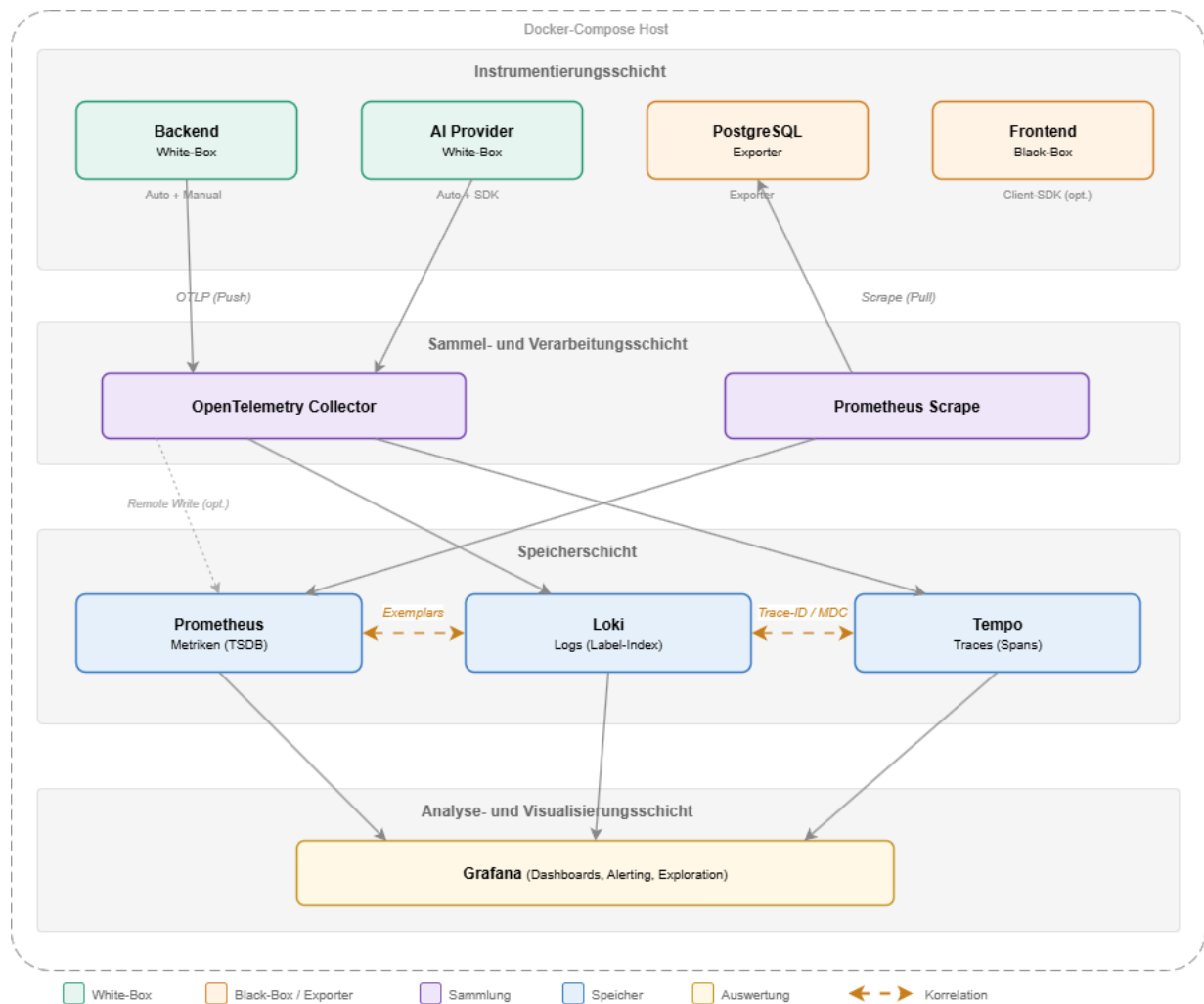


Abbildung 6.1: Logische Zielarchitektur der Observability-Pipeline fuer frag.jetzt. Das Schichtenmodell zeigt den Datenfluss von der Instrumentierung ueber Sammlung und Speicherung bis zur Auswertung. Gestrichelte Querverbindungen kennzeichnen Korrelationsmechanismen zwischen den Speicher-Backends. Quelle: Eigene Darstellung.

6.3 Servicebezogene Operationalisierung der Four Golden Signals

Die Ableitung der Beobachtungsgrößen folgt dem in Abschnitt 4.2 beschriebenen Vorgehen. Zunächst wird die Rolle des Dienstes im Gesamtsystem bestimmt, dann werden die kritischen Interaktionspunkte identifiziert, und schließlich werden geeignete Messgrößen entlang der vier Dimensionen Latency, Traffic, Errors und Saturation abgeleitet (Ewaschuk, 2017, S. 7–8). Die servicebezogene Operationalisierung konzentriert sich auf die vier für die Forschungsfragen zentralen Beobachtungseinheiten Frontend, Backend, Datenbank und externe KI-Dienste. Nginx, RabbitMQ und das WebSocket Gateway werden dabei entweder als Infrastruktur- bzw. Integrationskomponenten über die Infrastrukturmetriken mitbeobachtet oder im Kontext des Backends mitgeführt, erhalten jedoch im Rahmen dieser Arbeit keine eigenständige Signal-Matrix, da ihre betriebliche Rolle über die angrenzenden Dienste abgedeckt wird.

Dabei ist zu beachten, dass Ewaschuk (2017, S. 7) die Wahl des Traffic-Indikators ausdrücklich

als *high-level system-specific metric* bezeichnet. Ein Webdienst misst Anfragen pro Sekunde, ein Streaming-Dienst Netzwerkdurchsatz und ein Schlüsselspeicher Transaktionsraten. Diese Kontextabhängigkeit bedeutet, dass die Übertragung der vier Dimensionen auf jeden Dienst von frag.jetzt eine eigenständige Entwurfsleistung erfordert. Albuquerque und Correia (2025, S. 12) empfehlen, die Four Golden Signals als Ausgangspunkt zu verwenden und bei Bedarf um servicespezifische Kennzahlen zu ergänzen. Da für frag.jetzt keine historischen Produktivdaten vorliegen, dienen die nachfolgend definierten Schwellenwerte als operative Startwerte, die anhand von Lasttests kalibriert und im Betrieb angepasst werden müssen (vgl. Abschnitt 4.2).

6.3.1 PWA-Frontend

Das Angular-Frontend bildet den ersten Berührungspunkt der Nutzenden mit dem System. Seine Beobachtbarkeit unterscheidet sich grundlegend von den serverseitigen Komponenten, weil die Ausführung im Browser der Endgeräte stattfindet und die Infrastruktur nicht direkt kontrollierbar ist.

Latency. Aus Nutzersicht manifestiert sich Latenz im Frontend als wahrgenommene Ladezeit. Neben der Server-Antwortzeit tragen clientseitige Faktoren wie Netzwerkqualität, Rendering und JavaScript-Ausführung zur Gesamtlatenz bei, eine Unterscheidung, die bereits die Systemanalyse in Abschnitt 3.2 für die Architektur von frag.jetzt aufgezeigt hat. Aussagekräftige Basisindikatoren sind die Dauer von API-Aufrufen an das Backend (gemessen als Zeitspanne zwischen Absenden und Empfang der HTTP-Antwort) und die Ende-zu-Ende-Ladezeit beim Betreten eines Raums. Ergänzend können browserbasierte Metriken wie First Contentful Paint herangezogen werden, sofern eine clientseitige Instrumentierung über das OpenTelemetry-Browser-SDK erfolgt. Diese sind jedoch als optionale Erweiterung zu verstehen, da sie eine zusätzliche Integrationsleistung auf der Clientseite erfordern.

Traffic. Die Nachfrageintensität lässt sich anhand der Anzahl aktiver WebSocket-Verbindungen sowie der HTTP-Anfragen pro Zeiteinheit messen. Für frag.jetzt ist die Zahl gleichzeitig geöffneter Raumsitzungen ein besonders aufschlussreicher Indikator, weil sie den tatsächlichen Nutzungsgrad während einer Lehrveranstaltung widerspiegelt.

Errors. Relevante Fehlerindikatoren sind fehlgeschlagene API-Aufrufe (HTTP-Statuscodes 4xx und 5xx), unbehandelte JavaScript-Exceptions sowie Verbindungsabbrüche auf der WebSocket-Ebene. Da das Frontend den Reverse-Proxy-Endpunkt anspricht, spiegeln clientseitige Fehlerraten indirekt die Verfügbarkeit aller dahinterliegenden Dienste wider.

Saturation. Klassische Ressourcenmetriken wie CPU oder Arbeitsspeicher beziehen sich im Frontend auf das Endgerät und liegen außerhalb des Einflussbereichs des Betriebsteams. Die Sättigung kann daher nur über indirekte Proxy-Indikatoren abgeschätzt werden, die eine entsprechende Instrumentierung auf der Clientseite voraussetzen, etwa steigende Antwortzeiten

der API-Aufrufe oder eine zunehmende Anzahl von Retry-Versuchen im HTTP-Client. Diese Indikatoren leiten sich aus der Black-Box-Einschränkung des Frontends ab (vgl. Abschnitt 6.2.1) und bilden die ressourcenseitige Sättigung der dahinterliegenden Dienste indirekt ab. Da die Grenzen der Frontend-Beobachtbarkeit eng gesteckt sind, liegt der instrumentierungstechnische Schwerpunkt auf den serverseitigen Komponenten, die das Nutzungserlebnis bestimmen.

6.3.2 Spring-Boot-Backend

Das Backend ist die zentrale Verarbeitungskomponente und der in Kapitel 3 identifizierte Single Point of Failure. Es verarbeitet alle REST-Anfragen, steuert die Geschäftslogik und kommuniziert mit Datenbank, Message Broker und AI Provider.

Latency. Der primäre Indikator ist die Antwortdauer je REST-Endpunkt, gemessen als Histogramm, um neben Mittelwerten auch Perzentile (p50, p95, p99) darstellen zu können. Ewaschuk (2017, S. 7) betont die Notwendigkeit, Latenzverteilungen zu betrachten, weil Durchschnittswerte die Erfahrung einer kleinen, aber betroffenen Nutzergruppe mit hohen Antwortzeiten verbergen können. Ebenso ist die Unterscheidung zwischen der Latenz erfolgreicher und fehlgeschlagener Anfragen wesentlich, da schnell zurückgegebene Fehler den Gesamtdurchschnitt nach unten verzerren können (Ewaschuk, 2017, S. 7). Ergänzend sind die Latenzen der ausgehenden Aufrufe an PostgreSQL, den Message Broker und den AI Provider separat zu erfassen, um bei erhöhter Gesamtlatenz die verursachende Downstream-Abhängigkeit eingrenzen zu können. Das reaktive Programmiermodell auf Basis von WebFlux erfordert dabei besondere Aufmerksamkeit. Da keine blockierenden Threads im klassischen Sinne vorliegen, muss die Messung auf Span-Ebene innerhalb der reaktiven Kette erfolgen, analog zur manuellen Instrumentierung, die Sundström (2025, S. 25) für asynchrone Kommunikationspfade beschreibt.

Traffic. Die Anfragerate pro Endpunkt, aufgeschlüsselt nach HTTP-Methode und Pfad, bildet den zentralen Verkehrsindikator. Für die geschäftskritischen Abläufe aus Abschnitt 3.3, also Frage stellen, Abstimmung durchführen und KI-Zusammenfassung abrufen, ist eine gesonderte Erfassung sinnvoll, damit Laständerungen auf diesen Pfaden sofort erkennbar werden. Zusätzlich bildet die Anzahl publizierter Nachrichten an den Message Broker den Echtzeit-Traffic ab.

Errors. Fehler werden auf mehreren Ebenen erfasst, darunter HTTP-Statuscodes 5xx an den REST-Endpunkten, unbehandelte Exceptions in der Anwendungslogik (erkennbar an Trace-Spans mit Fehlerstatus) und fehlgeschlagene Downstream-Aufrufe. Die Systemanalyse hat gezeigt, dass kein einziger externer Aufruf einen konfigurierten Timeout besitzt (Abschnitt 3.2). Solange diese Lücke nicht geschlossen wird, äußern sich Timeout-bedingte Fehler nicht als explizite Fehlermeldungen, sondern als hängende Anfragen, ein Befund, der die Latenzüberwachung umso dringlicher macht.

Saturation. Der Ressourcendruck auf das Backend wird über Container-Metriken (CPU-Auslastung und Speicherverbrauch) sowie über anwendungsinterne Indikatoren gemessen. Dazu zählt insbesondere der aktive Verbindungspool zur Datenbank (R2DBC-Connection-Pool-Nutzung). Als optionaler Vertiefungsindikator kann die Event-Loop-Auslastung des zugrundeliegenden Servers herangezogen werden, sofern die Instrumentierung diese Metrik exponiert. Ewaschuk (2017, S. 8) empfiehlt, die Sättigung indirekt über steigende Latenz am 99. Perzentil über ein kurzes Zeitfenster zu beobachten, weil Latenzzunahmen häufig ein frühes Anzeichen für Ressourcenerschöpfung sind.

6.3.3 PostgreSQL

Die Datenbank ist keine Randkomponente, sondern an allen drei geschäftskritischen Nutzungspfaden beteiligt. Die in Abschnitt 3.3 beschriebenen Vote-Trigger und die Comment-Nummern-Generierung erzeugen potenzielle Engpässe, die gezielt beobachtet werden müssen.

Latency. Die durchschnittliche und maximale Abfrageausführungszeit, aufgeschlüsselt nach Datenbank, ist der zentrale Latenzindikator. Besondere Aufmerksamkeit verdienen lang laufende Abfragen, die auf fehlende Indizes oder Row-Level Contention hindeuten. Optional kann auch die Reaktionszeit der Flyway-Migrationen beim Container-Start als einmaliger, aber betrieblich relevanter Latenzwert erfasst werden.

Traffic. Geeignete Messgrößen sind die Transaktionsrate (Commits und Rollbacks pro Sekunde) und die Anzahl aktiver Verbindungen im Verhältnis zum konfigurierten Maximum. Ein plötzlicher Anstieg der Transaktionsrate korreliert mit Massenabstimmungen in Lehrveranstaltungen und ist ein direkter Indikator für die Nutzungsintensität. Sofern die Replikation zur `postgres_langchain`-Instanz im Produktivbetrieb relevant wird, kann deren Rate als ergänzender Traffic-Indikator aufgenommen werden.

Errors. Relevante Fehlersignale umfassen fehlgeschlagene Transaktionen, Deadlocks und Verbindungsablehnungen (weil das Pool-Maximum erreicht ist). Die in Abschnitt 3.3 beschriebene Praxis, Trigger zur Laufzeit über DDL-Statements zu deaktivieren, stellt ein zusätzliches Fehlerrisiko dar, das über die Überwachung von Lock-Wait-Ereignissen erkannt werden kann.

Saturation. Die Sättigung wird anhand des Verhältnisses genutzter zu maximal verfügbarer Verbindungen, der I/O-Auslastung (gelesen/geschrieben pro Sekunde), der Buffer-Cache-Hit-Rate und des verfügbaren Festplattenspeichers gemessen. Ein sinkender Cache-Hit-Anteil deutet darauf hin, dass die Arbeitsspeicherkonfiguration von PostgreSQL nicht mehr zum aktuellen Datenvolumen passt und die Datenbank verstärkt auf den Plattenspeicher zugreifen muss.

6.3.4 Externe KI-Dienste

Die Beobachtung der externen KI-Dienste folgt einer grundsätzlich anderen Logik als die der selbst betriebenen Komponenten. Der Quellcode der LLM-Provider ist nicht verfügbar. Die Beobachtbarkeit beschränkt sich auf die Schnittstelle zwischen AI Provider und den angebundenen Modellen. Bissig (2024, S. 31) ordnet dieses Muster als Black-Box-Beobachtung ein, bei der mindestens die Ein- und Ausgangsseite jedes Aufrufs instrumentiert wird.

Latency. Die Antwortzeit je LLM-Aufruf variiert erheblich. Je nach Modell, Prompt-Länge und Provider-Auslastung können einzelne Aufrufe mehrere Sekunden bis zweistellige Sekundenwerte erreichen. Die Latenz wird als Histogramm pro Provider und Modell erfasst, um sowohl die typische als auch die Worst-Case-Dauer sichtbar zu machen. Da kein Timeout konfiguriert ist (R1), kann ein einzelner hängender Aufruf den AI Provider blockieren und über die Aufrufkette das Backend beeinflussen.

Traffic. Die Anzahl der LLM-Aufrufe pro Zeiteinheit, aufgeschlüsselt nach Funktionstyp (Keyword-Extraktion, Moderation, Konversationsthread), bildet den Traffic ab. Da die Nutzung der KI-Funktionen von der Konfiguration des jeweiligen Raums und dem verfügbaren API-Kontingent abhängt, unterliegt der Traffic stärkeren Schwankungen als bei den anderen Komponenten.

Errors. Fehlersignale sind HTTP-Fehlerantworten der Provider-APIs (insbesondere 429 Rate Limit Exceeded und 5xx), Timeouts (sobald konfiguriert) sowie Parsing-Fehler bei unerwarteten Antwortformaten. Die Systemanalyse hat gezeigt, dass neun von 19 LLM-Providern explizit mit `max_retries=0` konfiguriert sind – ein fehlgeschlagener Aufruf wird also nicht automatisch wiederholt, sondern schlägt unmittelbar durch.

Saturation. Da die LLM-Provider als externe Dienste keinen direkten Einblick in ihre Ressourcenauslastung gewähren, wird Sättigung über Proxy-Indikatoren wie steigende Antwortzeiten, zunehmende Rate-Limit-Fehler und wachsende Aufrufwarteschlangen im AI Provider abgebildet. Auf Seiten des eigenen AI-Provider-Containers sind CPU- und Speicherverbrauch zu erfassen, weil die Vorverarbeitung (Prompt-Aufbau, Embedding-Generierung) lokal stattfindet und unter Last Ressourcen beansprucht.

6.4 Serviceübergreifende Signal-Matrix

Die vorangehenden Abschnitte haben die Golden Signals je Komponente beschrieben. Für den operativen Einsatz in Dashboards und Alerting ist eine kompakte Gesamtübersicht erforderlich, die auf einen Blick zeigt, welche Messgröße an welcher Stelle erhoben wird, welchem Zweck sie dient und welche Rolle primär von ihr profitiert. Tabelle 6.1 fasst diese Informationen in einer Signal-Matrix zusammen.

Tabelle 6.1: Serviceübergreifende Signal-Matrix mit Zuordnung von Dienst, Golden Signal, konkreter Messgröße, Erhebungszweck und primärer Stakeholder-Rolle. Quelle: Eigene Darstellung.

Dienst	Signal	Messgröße	Zweck	Rolle
Backend	Latency	Antwortdauer je Endpunkt (p50, p95, p99)	Nutzererfahrung, Engpasserkennung	Entwicklung
Backend	Traffic	Anfragen/s je Pfad, Broker-Nachrichten/s	Lastbild, Kapazitätsplanung	DevOps
Backend	Errors	HTTP-5xx-Rate, Exception-Rate, Downstream-Fehler	Fehlererkennung, Trendanalyse	Entwicklung
Backend	Saturation	CPU, Speicher, R2DBC-Pool-Nutzung	Ressourcenwarnung	DevOps
PostgreSQL	Latency	Abfrageausführungszeit (Durchschnitt, Maximum)	Indexbedarf, Contention-Erkennung	Entwicklung
PostgreSQL	Traffic	Transaktionen/s, aktive Verbindungen	Nutzungsintensität	DevOps
PostgreSQL	Errors	Deadlocks, fehlgeschl. Transaktionen, Lock-Waits	Stabilitätsüberwachung	Entwicklung
PostgreSQL	Saturation	Verbindungspool-Quote, Cache-Hit-Rate, Disk-I/O	Kapazitätswarnung	DevOps
AI Prov.	Latency	LLM-Antwortzeit je Provider/Modell (Histogramm)	SLA-Proxy, Kaskadenerkennung	Entwicklung
AI Prov.	Traffic	LLM-Aufrufe/Zeiteinheit je Funktionstyp	Nutzungsmuster, Kontingentplanung	Product Owner
AI Prov.	Errors	Provider-Fehlerrate, 429er-Rate, Parsing-Fehler	Verfügbarkeit ext. Abhängigkeiten	Entwicklung
AI Prov.	Saturation	Antwortzeit-Trend, Rate-Limit-Häufigkeit, lokale CPU	Kapazitätswarnung	DevOps
Frontend	Latency	API-Aufruf-Dauer, Raum-Ladezeit	Nutzererfahrung	Product Owner
Frontend	Traffic	Aktive WS-Verbindungen, HTTP-Anfragen/s	Nutzungsgrad	Product Owner
Frontend	Errors	HTTP-Fehler, JS-Exceptions, WS-Abbrüche	Fehlererkennung clientseitig	Entwicklung
Frontend	Saturation	Antwortzeit-Trend, Retry-Häufigkeit	Überlastindikation	DevOps

Die Matrix dient drei Zwecken. Erstens macht sie die Zuordnung zwischen technischen Messgrößen und fachlichen Fragestellungen explizit, sodass jede Messgröße einen erkennbaren Beobachtungszweck besitzt. Zweitens bildet sie die Grundlage für das rollenbasierte Dashboard-Design in Abschnitt 6.5, weil sich aus der Spalte *Rolle* direkt ableiten lässt, welche Kennzahlen in welcher Sicht erscheinen sollen. Drittens definiert sie den Rahmen für das Alerting-Konzept in Abschnitt 6.6. Nicht jede Messgröße eignet sich als Alarmauslöser. Die Matrix hilft, die Auswahl auf betrieblich relevante Indikatoren zu fokussieren. Damit adressiert die Matrix die Anforderungen A1 (Observability aller Kernkomponenten), A2 (servicebezogene Messbarkeit) und A3 (Korrelierbarkeit über Dienstgrenzen) aus Abschnitt 3.4.

6.5 Rollenbasiertes Informations- und Dashboard-Konzept

Kapitel 2 hat theoretisch begründet, dass Beobachtbarkeit eine nutzungsbezogene Sichtbarkeit erfordert, nicht bloß eine Datensammlung. Abschnitt 6.1 hat dies als Leitprinzip P3 verankert. Der vorliegende Abschnitt konkretisiert, welche Rollen welche Informationsverdichtung benötigen, und definiert die konzeptionellen Dashboard-Ebenen. Die technische Umsetzung, konkrete Panel-Konfigurationen, Abfragesprachen und Drill-down-Pfade, wird in Kapitel 7 dokumentiert.

Sakinala (2025, S. 108) beschreiben eine hierarchische Informationsarchitektur als bewährtes Muster, das von einer systemweiten Übersicht über dienstspezifische Ansichten bis hin zu komponentenbezogenen Detailmetriken reicht. Für frag.jetzt wird dieses Muster in drei Sichtebenen übersetzt, deren Zuschnitt sich aus den in Abschnitt 3.4 identifizierten Stakeholder-Gruppen und der Signal-Matrix aus Abschnitt 6.4 ableitet.

Übersichtssicht (Product Owner / Management). Diese Ebene beantwortet die Frage, ob frag.jetzt aus Nutzersicht funktioniert. Sie zeigt verdichtete Indikatoren wie aktive Raumsitzungen, globale Fehlerrate, Gesamtlatenz der Kernabläufe und Verfügbarkeit der KI-Funktionen. Einzelne Metriken werden hier bewusst aggregiert. Detailinformationen sind nur über Verlinkung erreichbar.

Dienst- und Infrastruktursicht (DevOps / Betrieb). Diese Ebene beantwortet die Frage, wo das Problem liegt. Sie stellt Ressourcenmetriken (CPU, Speicher, Disk I/O je Container), Datenbankauslastung (Verbindungspool, Cache-Hit-Rate), Message-Broker-Kennzahlen und den Host-Zustand dar. Eine Service-Map, die aus den Trace-Daten gewonnen wird, zeigt die Abhängigkeiten zwischen den Komponenten und ermöglicht es, von einem auffälligen Knoten direkt zur Detailsicht zu navigieren.

Diagnose- und Entwicklungssicht (Entwicklung). Diese Ebene beantwortet die Frage, was genau im Code passiert. Sie bietet endpunktbezogene Latenzhistogramme, Fehleraufschlüsselungen nach Exception-Typ, Trace-Ansichten für einzelne Anfrageketten und die Möglichkeit, aus einem Trace in die zugehörigen Logeinträge zu springen. Diese Sicht nutzt die in Abschnitt 6.2.3 beschriebenen Korrelationsmechanismen, also Trace-ID-Propagation, MDC-Injektion und Exem-

plars, und stellt damit die diagnostische Tiefe bereit, die für die Ursachenanalyse erforderlich ist.

Die Trennung in drei Ebenen verhindert, dass ein einzelnes Dashboard alle Nutzungsgruppen zu bedienen versucht und dabei entweder zu detailliert oder zu oberflächlich wird. Niedermaier et al. (2019, S. 8) dokumentieren das Fehlen einer navigierbaren Gesamtsicht als wiederkehrendes Praxisproblem. Die hierarchische Struktur adressiert diesen Befund, indem jede Ebene als Einstiegspunkt dienen kann und Drill-down-Pfade zur jeweils nächsttiefer Ebene führen.

6.6 Alerting-Konzept und Runbook-Logik

Die theoretischen Grundlagen zur Alarmierungsqualität wurden in Abschnitt 2.7 erarbeitet. Der vorliegende Abschnitt überführt diese Prinzipien in ein konkretes Alerting-Konzept für frag.jetzt. Die technische Konfiguration der Alarmregeln, also Schwellenwerte, Routing-Pfade und Benachrichtigungskanäle, folgt in Kapitel 7.

6.6.1 Entwurfsgrundsätze

Das Alerting-Konzept folgt drei Grundsätzen, die direkt aus den Erkenntnissen von Kapitel 2 abgeleitet sind. Alle drei zielen darauf ab, die Spannung zwischen der Fülle technisch messbarer Warnsignale und der begrenzten Aufmerksamkeit eines kleinen Betriebsteams zugunsten betrieblich relevanter Alarme aufzulösen.

Erstens, symptomorientiert statt ursachenorientiert. Alarme sollen auf Zustände reagieren, die eine tatsächliche oder unmittelbar bevorstehende Beeinträchtigung für Nutzende darstellen (Ewaschuk, 2017, S. 11–12). Eine erhöhte CPU-Auslastung allein rechtfertigt keinen Alarm, solange die Antwortzeiten und Fehlerraten im akzeptablen Bereich bleiben. Erst wenn sich die Ressourcenerschöpfung in einer für Nutzende spürbaren Verschlechterung niederschlägt, etwa in steigender Latenz oder zunehmenden Fehlern, wird alarmiert. Symptomorientiertes Alerting schützt damit vor Alarmmüdigkeit und priorisiert Störungen, die die wahrgenommene Dienstqualität tatsächlich beeinträchtigen (Ewaschuk, 2017, S. 12).

Zweitens, jeder Alarm muss handlungsfähig sein. Das bedeutet, dass beim Eintreffen einer Meldung erkennbar sein muss, welches Problem vorliegt, welche Komponente betroffen ist und welche Erstmaßnahme empfohlen wird (Vinnakota & Kolla, 2025, S. 174). Alarme ohne klaren Handlungsbezug erzeugen Rauschen und tragen zur Alarmmüdigkeit bei (Sakinala, 2025, S. 107). Für frag.jetzt ist dies besonders relevant, weil das kleine Betriebsteam nicht über dedizierte On-Call-Kapazitäten verfügt und jeder Alarm einen realen Aufmerksamkeitsaufwand erzeugt.

Drittens, Schweregrade orientieren sich an der Nutzerwirkung. Für frag.jetzt werden zwei Stufen definiert, *Critical* für akute Ausfälle geschäftskritischer Funktionen und *Warning* für sich abzeichnende Verschlechterungen, die noch keinen unmittelbaren Ausfall darstellen. Diese Priorisierung nach Nutzerwirkung statt nach technischer Ursache folgt der Empfehlung von

Gowda und Hameed (2022, S. 3), dass ein Alarm seinen operativen Wert erst dann entfaltet, wenn er Problem, Schweregrad und empfohlene Gegenmaßnahme klar benennt.

6.6.2 Alarmkategorien

Aus der Signal-Matrix lassen sich vier Kategorien ableiten, für die jeweils exemplarische Alarmregeln skizziert werden. Die konkreten Schwellenwerte und Zeitfenster werden in Kapitel 7 festgelegt.

Verfügbarkeitsalarme (Critical). Ein Alarm wird ausgelöst, wenn eine geschäftskritische Funktion nicht mehr erreichbar ist, etwa wenn das Backend über einen definierten Zeitraum keine erfolgreichen Antworten mehr liefert oder die Datenbankverbindung dauerhaft fehlschlägt. Da das Backend als Single Point of Failure identifiziert wurde (R4), kommt seiner Erreichbarkeit die höchste Priorität zu.

Latenzalarme (Warning / Critical). Steigt die Antwortzeit eines Kernendpunkts über einen definierten Grenzwert, etwa das 95. Perzentil über ein gleitendes Fenster, wird zunächst eine Warnung und bei weiterer Verschlechterung ein kritischer Alarm erzeugt. Für die KI-Dienste, deren Antwortzeiten naturgemäß höher liegen, gelten angepasste Grenzwerte.

Fehleralarme (Warning / Critical). Eine Fehlerrate oberhalb eines definierten Anteils eingehender Anfragen löst eine Warnung aus. Persistierende Fehlerspitzen oder ein Totalausfall eines Dienstes eskalieren die Meldung zu Critical. Die explizite Trennung von client- (4xx) und serverseitigen (5xx) Fehlern verhindert, dass ungültige Eingaben denselben Alarm wie interne Serverfehler erzeugen.

Sättigungsalarme (Warning). Ressourcenmetriken wie CPU-Auslastung, Speicherdruck und Datenbankverbindungspool-Nutzung werden als vorausschauende Warnung konfiguriert. Sie lösen keine sofortige Eskalation aus, sondern signalisieren, dass eine Kapazitätsgrenze näher rückt. Ewaschuk (2017, S. 8) empfiehlt, Sättigung über steigende Latenz am 99. Perzentil indirekt zu erkennen, weil dieses Maß häufig vor dem vollständigen Ressourcenverbrauch anschlägt.

6.6.3 Runbook-Verknüpfung

Jeder Critical-Alarm wird mit einem Runbook verknüpft, das mindestens folgende Elemente enthält. Dazu gehören eine Problembeschreibung, die betroffene Komponente, empfohlene Diagnoseschritte (mit Verweis auf das relevante Dashboard und die zugehörige Datenquelle), Erstmaßnahmen und ein Eskalationspfad. Kapitel 2 hat die Rolle von Runbooks als Wissenssicherung und Reaktionsbeschleuniger begründet (Vinnakota & Kolla, 2025, S. 174). Für den Prototyp werden exemplarische Runbooks für zwei bis drei der häufigsten Störungsszenarien aus Abschnitt 3.4 erstellt, insbesondere für den Kaskadeneffekt durch KI-Service-Timeouts (R1)

und für Datenbankengpässe bei Massenabstimmungen (R2). Die konkrete Runbook-Struktur und deren Anbindung an die Alarmbenachrichtigung werden in Kapitel 7 dokumentiert.

6.7 Anomalieerkennung als Erweiterungsperspektive

Die bisher beschriebene Alerting-Logik basiert auf regelbasierten Schwellenwerten. Für einen Ausgangszustand ohne historische Betriebsdaten und mit begrenztem Teamumfang ist dieser Ansatz angemessen und gut wartbar. Regelbasierte Alarme stoßen jedoch an Grenzen, wenn sich die typischen Lastmuster im Tages- oder Semesterverlauf verändern und starre Schwellenwerte entweder zu viele False Positives oder zu späte Warnungen produzieren (Vinnakota & Kolla, 2025, S. 173).

Verfahren der automatisierten Anomalieerkennung, ob statistisch oder auf Basis maschinellen Lernens, setzen voraus, dass ein Modell des regulären Systemverhaltens existiert, gegen das Abweichungen gemessen werden können. Soldani und Brogi (2022, S. 5–6) zeigen in ihrem Survey, dass die gängigen Verfahren zunächst eine Lernphase mit fehlerfreien Betriebsdaten durchlaufen, in der sie ein Basismodell der erwarteten Metrik- oder Logmuster aufbauen. Erst gegen dieses Basismodell werden anschließend im Produktivbetrieb Abweichungen detektiert. Dabei weisen Soldani und Brogi (2022, S. 6) auf eine grundlegende Einschränkung hin. Verändern sich die Laufzeitbedingungen einer Anwendung etwa durch neue Dienste, veränderte Lastmuster oder Architekturadaptierungen, sinkt die Genauigkeit trainierter Modelle, weil das gelernte Normalverhalten nicht mehr dem aktuellen Systemzustand entspricht. Dies kann zu erhöhten Raten von False Positives und False Negatives führen. Niedermaier et al. (2019, S. 13) bestätigen diesen Befund aus der Praxis, indem sie darauf hinweisen, dass die Voraussetzungen für den Einsatz KI-gestützter Anomalieerkennung, insbesondere eine hinreichende Datenqualität, eine zentrale Datenhaltung und propagierter Kontext, in vielen Organisationen noch nicht erfüllt sind.

Für frag.jetzt liegen zum Zeitpunkt der Strategieentwicklung keine historischen Produktivdaten vor. Ein Anomalieerkennungsmodell hätte keine Basis, gegen die es Abweichungen messen könnte. Darüber hinaus befindet sich die Architektur in einer Phase aktiver Weiterentwicklung, in der neue Instrumentierung und veränderte Dienstkonfigurationen das Laufzeitverhalten fortlaufend beeinflussen. Beide Faktoren sprechen dafür, die Anomalieerkennung bewusst als *zukünftigen Ausbauschritt* zu positionieren, nicht als Bestandteil der prototypischen Implementierung.

Der konzeptionelle Platz innerhalb der Architektur ist jedoch vorbereitet. Die standardisierte Metrikerhebung über Prometheus und die offene Abfrageschnittstelle über PromQL erlauben es, nachgelagerte Analysewerkzeuge, ob statistische Trendalarme, externe Zeitreihenanalyse oder lernbasierte Modelle, ohne Änderung der Instrumentierung anzubinden, sobald ein hinreichender Betriebsdatenbestand aufgebaut ist.

Dieser Abschnitt vervollständigt die konzeptionelle Strategie. In Kapitel 7 wird der hier formulierte Entwurf in einer prototypischen Implementierung konkretisiert.

7 Prototypische Implementierung

Dieses Kapitel dokumentiert die praktische Umsetzung der in Kapitel 6 konzipierten Observability-Strategie. Es beschreibt den Umfang des Prototyps, die Instrumentierung der Kernkomponenten, die beobachtete Funktionsfähigkeit in exemplarischen Szenarien, die Umsetzung eines Golden-Signals-Dashboards sowie die Konfiguration exemplarischer Alerting-Regeln mit zugehörigen Runbooks. Den Abschluss bildet eine Reflexion über Herausforderungen, die während der Implementierung aufgetreten sind.

7.1 Umfang und Abgrenzung der prototypischen Umsetzung

Die Umsetzung erfolgte auf dem Staging-Server von frag.jetzt, auf dem der vollständige Anwendungsstack in einer produktionsnahen Docker-Compose-Konfiguration betrieben wird. Der Observability-Stack wurde als eigenständiges Compose-Projekt neben der bestehenden Orchestrierung deployt und über ein gemeinsames Docker-Netzwerk an die Applikationscontainer angebunden.

Die Implementierung umfasst drei Schwerpunkte: erstens das Deployment des LGTM-Stacks mit allen Speicher-Backends und Infrastruktur-Exportern, zweitens die Instrumentierung von Backend und WS-Gateway über den OpenTelemetry Java Agent und drittens die Erstellung eines Golden-Signals-Dashboards in Grafana mit zugehörigen Alerting-Regeln. Bewusst ausgeklammert wurden die Instrumentierung des AI Providers (Python/FastAPI) und des Frontends (Angular/Nginx), die clientseitige Browser-Telemetrie sowie die Produktivhärtung mit Hochverfügbarkeit und Langzeitspeicherung. Ebenso beschränkt sich das Dashboard-Design auf ein einzelnes, technisch orientiertes Golden-Signals-Dashboard. Die in Kapitel 6.5 konzipierte rollenspezifische Aufbereitung für Product Owner und Entwicklung verbleibt als Ausbauschritt, weil sie auf denselben Datenquellen aufsetzt und primär eine Frage der Informationsverdichtung ist.

Diese Abgrenzungen begrenzen die Vollständigkeit der prototypischen Validierung: Eine durchgängige Ende-zu-Ende-Beobachtbarkeit aller Systemgrenzen kann ohne Frontend- und AI-Provider-Instrumentierung nicht nachgewiesen werden. Die Teilumsetzung erlaubt jedoch die Prüfung der zentralen Mechanismen, insbesondere der Trace-Erfassung über Dienstgrenzen, der Metrik-Aggregation entlang der Golden Signals und der Korrelation zwischen Telemetriesignalen. Bissig (2024, S. 52) beobachten in einer vergleichbaren Implementierungsstudie, dass der Hauptaufwand bei der Einführung von Observability nicht in der theoretischen Konzeption, sondern in den Infrastrukturentscheidungen und der Konfiguration der Observability-Komponenten liegt.

Die folgenden Abschnitte dokumentieren die konkreten Konfigurationen und ihre beobachteten Ergebnisse. Umfangreichere Konfigurationsdateien wie die vollständige OTel-Collector-Konfiguration und das Dashboard-JSON befinden sich im Anhang.

7.2 Instrumentierung ausgewählter Kernkomponenten

Die Instrumentierung folgt dem in Kapitel 6.2 beschriebenen Schichtenmodell: Applikationsdienste senden Telemetriedaten über OTLP an den Collector, Infrastrukturkomponenten werden über dedizierte Exporter beobachtet. Tabelle 7.1 gibt eine Übersicht über die instrumentierten Komponenten, die jeweils erzeugten Signale und den im Prototyp beobachteten Nachweis.

Tabelle 7.1: Übersicht der instrumentierten Komponenten und beobachteten Telemetriesignale.
Quelle: Eigene Darstellung.

Komponente	Instrumentierung	Erzeugte Signale	Nachweis im Prototyp
Backend (Spring Boot)	OTel Java Agent (Auto-Instr.)	HTTP-Spans, DB-Spans, Logback-Logs mit Trace-ID, JVM-Metriken	Span-Waterfall mit DB-Operationen in Grafana sichtbar
WS-Gateway (Netty)	OTel Java Agent (Auto-Instr.)	WebSocket-Spans, RabbitMQ-Spans, JVM-Metriken	Service Graph zeigt Knoten mit Anfragerate
PostgreSQL	PostgreSQL Exporter	Verbindungszähler, Transaktionsraten, Lock-Statistiken	<code>pg_stat_activity_count</code> in Prometheus abfragbar
RabbitMQ	Prometheus-Plugin	Queue-Tiefen, Nachrichtenraten	<code>rabbitmq_queue_messages</code> in Dashboard-Panel sichtbar
Host	Node Exporter, cAdvisor	CPU, Speicher, Netzwerk, Container-Metriken	Gauge-Panels im Dashboard mit Schwellenwertfärbung

7.2.1 Observability-Stack und Telemetrie-Pipeline

Das Deployment umfasst acht Container, die über ein separates Docker-Compose-File orchestriert werden. Der OpenTelemetry Collector empfängt sämtliche Applikationstelemetrie über OTLP (gRPC auf Port 4317, HTTP auf Port 4318) und verteilt sie an die drei Speicher-Backends: Metriken an Prometheus, Logs an Loki und Traces an Tempo. Für die Infrastrukturschicht erfassen cAdvisor Container-Ressourcenmetriken, der Node Exporter Host-Kennzahlen und der PostgreSQL Exporter Datenbankmetriken. Die Metriken des RabbitMQ-Prometheus-Plugins werden von Prometheus direkt über das Docker-interne Netzwerk abgerufen, ohne dass ein zusätzlicher Port auf dem Host freigegeben werden muss. Damit implementiert die Pipeline das in Kapitel 6.2 entworfene Architekturschema mit den dort festgelegten Komponentenzuordnungen.

Alle Observability-Container sind mit expliziten Speicherlimits konfiguriert, um das in Kapitel 3 beschriebene Risiko einer unkontrollierten Ressourcenverdrängung (R5) nicht im Observability-Stack selbst zu reproduzieren. Grafana wird mit provisionierten Datenquellen gestartet, sodass Prometheus, Loki und Tempo beim ersten Aufruf bereits konfiguriert sind. Die Datenquellen-Provisioning stellt insbesondere die Verknüpfungen zwischen den Backends her: Tempo-Traces können auf korrespondierende Loki-Logs verweisen und umgekehrt. Diese Korrelationsfähigkeit

ist eine Voraussetzung für die in A1 geforderte Ende-zu-Ende-Nachvollziehbarkeit (Sundström, 2025, S. 16).

7.2.2 Applikationsinstrumentierung über den OTel Java Agent

Für die JVM-basierten Dienste (Backend und WS-Gateway) wurde der OpenTelemetry Java Agent in Version 2.26.0 eingesetzt. Der Agent wird nicht in das Docker-Image integriert, sondern als Read-Only-Volume in den Container gemountet und über die Umgebungsvariable `JAVA_TOOL_OPTIONS` als JVM-Argument aktiviert. Dieser Ansatz vermeidet Änderungen am Applikationscode und an den bestehenden Docker-Images. Bissig (2024, S. 51–52) bestätigen, dass bei Standard-Java-Diensten lediglich das Einbinden des Agenten als Volume und die Konfiguration weniger Umgebungsvariablen erforderlich sind, um funktionsfähige Telemetrie zu erzeugen. Die Konfiguration erfolgt vollständig über Umgebungsvariablen im Docker-Compose-Override-File: Dienstname (`OTEL_SERVICE_NAME`), Collector-Adresse (`OTEL_EXPORTER_OTLP_ENDPOINT`), Exportformat und Umgebungsattribute.

Der Agent instrumentiert automatisch HTTP-Anfragen (Spring WebFlux, Netty), R2DBC-Datenbankzugriffe, RabbitMQ-Interaktionen und Logback-Logausgaben. Im Ergebnis erzeugt jeder eingehende HTTP-Request einen Root-Span, der untergeordnete Spans für Datenbankoperationen enthält. Die Trace-Ansicht in Grafana zeigt diese Hierarchie als Span-Waterfall, in der beispielsweise ein `POST /livepoll/find`-Request mit seinen zugehörigen `SELECT`- und `UPDATE`-Operationen auf der PostgreSQL-Datenbank sichtbar wird. Für die instrumentierten Dienste ist damit der Verarbeitungspfad vom HTTP-Eingang über die Geschäftslogik bis zur Datenpersistierung in einem zusammenhängenden Trace rekonstruierbar, was Anforderung A1 in Teilen adressiert.

Der Service Graph in Grafana, der aus den von Tempo generierten Span-Metriken gespeist wird, visualisiert die Kommunikationsbeziehungen zwischen den instrumentierten Diensten. Im Prototyp zeigt er drei Knoten (`fragjetzt-backend`, `fragjetzt-ws-gateway` und `fragjetzt-postgres`) mit ihren jeweiligen Anfrageraten und durchschnittlichen Antwortzeiten. Die automatische Erzeugung dieses Graphen aus Trace-Daten bestätigt, dass die gewählte Kombination aus OTel Agent und Tempo die in Kapitel 5 als Vorteil beschriebene enge Grafana-Integration im Prototyp einlöst (Sundström, 2025, S. 16).

7.2.3 Infrastrukturexporter und Scrape-Konfiguration

Prometheus scrapet sechs Targets im 15-Sekunden-Intervall: sich selbst, den OTel Collector, cAdvisor, den Node Exporter, den PostgreSQL Exporter und das RabbitMQ-Prometheus-Plugin. Die Infrastruktur-Exporter liefern Metriken, die direkt auf die Sättigungsindikatoren der Signal-Matrix aus Kapitel 6.4 abbilden: `pg_stat_activity_count` für die Datenbankverbindungsauslastung, `rabbitmq_queue_messages` für Queue-Tiefen, `node_memory_MemAvailable_bytes` für die Host-Speicherauslastung und JVM-Metriken

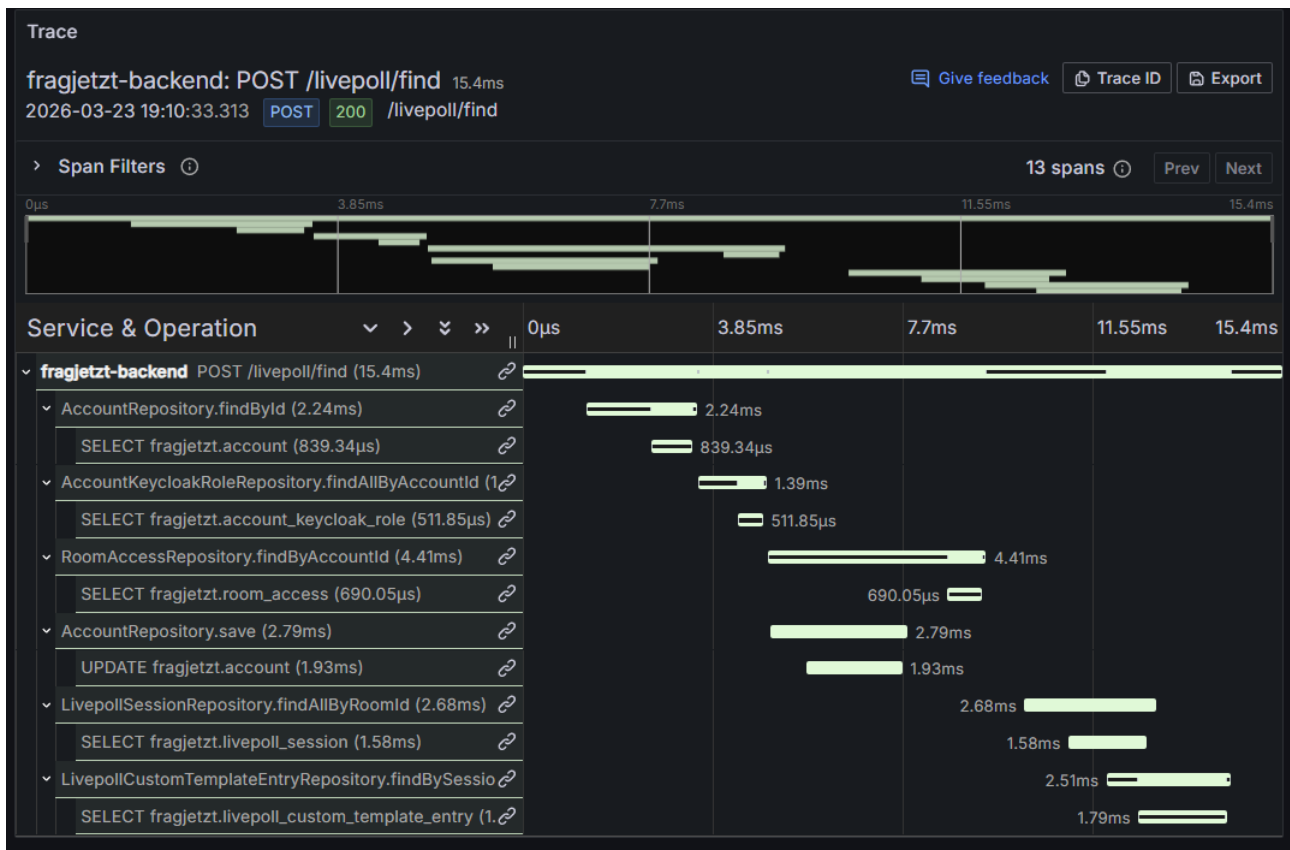


Abbildung 7.1: Span-Waterfall eines Backend-Traces (`POST /livepoll/find`) mit 13 Spans und einer Gesamtdauer von 15,4 ms. Quelle: Eigene Darstellung.

(`jvm_memory_used_bytes`, `jvm_cpu_recent_utilization_ratio`) für die Anwendungsressourcen. Damit wird Anforderung A5 auf Metrikebene im Prototyp abgebildet.

Ergänzend zu den Infrastrukturmetriken empfängt Prometheus über den OTLP-Receiver Applikationsmetriken vom OTEL Collector, darunter HTTP-Request-Dauer-Histogramme (`http_server_request_duration_seconds`). Tempo generiert aus den eingehenden Traces automatisch Span-Metriken (`traces_spanmetrics_calls_total`, `traces_spanmetrics_latency`) und schreibt diese per Remote Write an Prometheus zurück. Aus diesen Span-Metriken werden der Service Graph und die Traffic- sowie Fehlerratenpanels im Dashboard gespeist. Dieses Zusammenspiel aus Infrastruktur- und Applikationsmetriken entspricht den von Albuquerque und Correia (2025, S. 10, 16) beschriebenen komplementären Entwurfsmustern *Application Metrics* und *Infrastructure Metrics*.

7.2.4 Logging-Anbindung

Das Backend erzeugte im Standardbetrieb nahezu keine Logausgaben, weil Spring Boot WebFlux eingehende HTTP-Requests nicht automatisch protokolliert. Die Logs, die über den OTEL Agent an Loki weitergeleitet werden, beschränkten sich auf Startmeldungen. Für den Prototyp wurde Debug-Level-Logging für die HTTP-Verarbeitungsschicht (`org.springframework.web`) selektiv aktiviert, sodass auch Request-bezogene Einträge in Loki verfügbar sind und über die vom OTEL

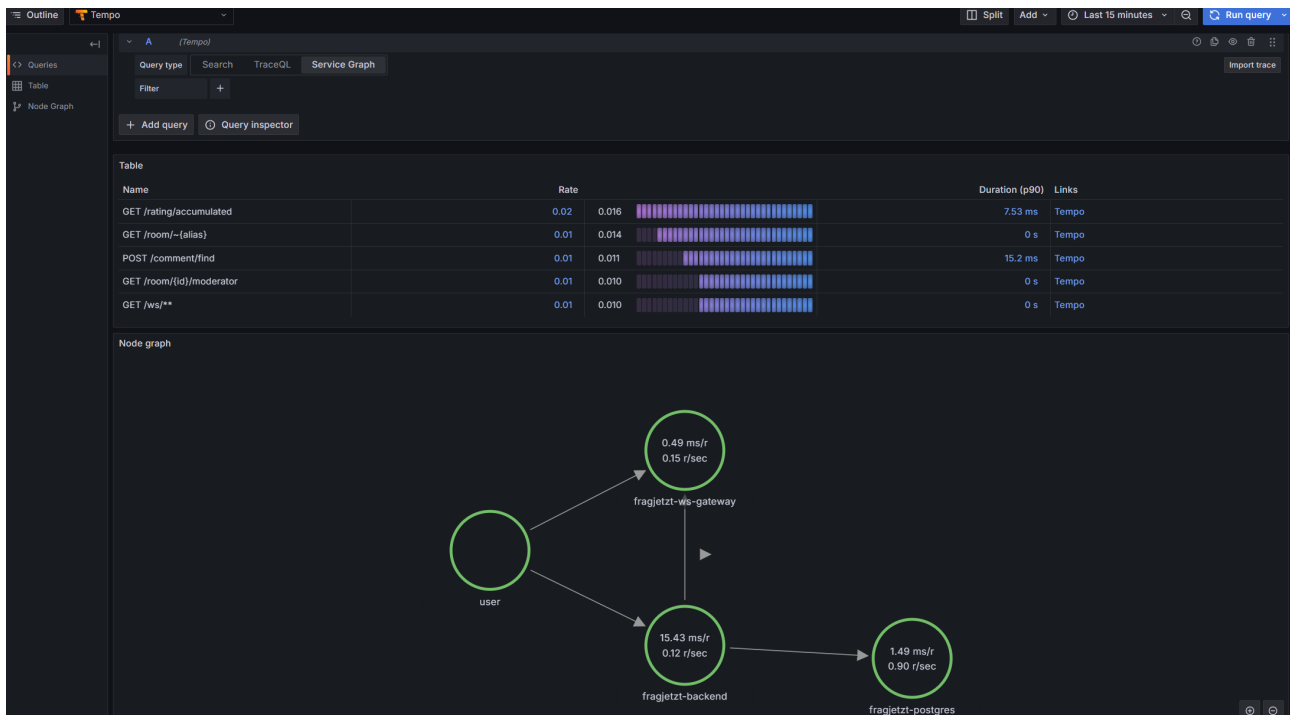


Abbildung 7.2: Service Graph aus Tempo mit den Kommunikationspfaden user → fragjetzt-backend → fragjetzt-postgres sowie user → fragjetzt-ws-gateway. Quelle: Eigene Darstellung.

Agent automatisch injizierten Trace-IDs mit den zugehörigen Spans korreliert werden können. Diese Korrelierbarkeit über Trace-IDs adressiert Anforderung A3 in Grundzügen.

Einschränkend ist festzuhalten, dass diese Konfiguration eine Demonstrationslösung für den Staging-Prototyp darstellt. Debug-Level-Logging erzeugt ein hohes Ausgabevolumen, das im Produktivbetrieb nicht vertretbar wäre. Für einen späteren Produktiveinsatz wäre stattdessen strukturierte, selektiv priorisierte JSON-Protokollierung vorzuziehen, bei der Logeinträge je nach Schweregrad gefiltert und nur geschäftsrelevante Ereignisse dauerhaft an Loki weitergeleitet werden.

7.2.5 Exemplarische Validierungsszenarien

Um die Funktionsfähigkeit der Telemetrie-Pipeline über die reine Konfigurationsbeschreibung hinaus zu belegen, wurden drei Szenarien auf dem Staging-Server exemplarisch durchgeführt. Die Szenarien folgen jeweils dem Muster: Ausgangsaktion, erwartete Telemetriesignale, beobachtetes Ergebnis. Sie erheben keinen Anspruch auf systematische Testabdeckung, sondern dienen als Plausibilitätsnachweis für die Kernmechanismen.

Szenario 1: Normaler Nutzerrequest (Frage stellen). Ein Nutzer erstellte über die Weboberfläche eine neue Frage in einem aktiven Raum. Erwartet wurde ein Root-Span für den POST /comment/-Endpunkt mit untergeordneten Spans für den Datenbank-Insert sowie ein Anstieg des Latenz- und Traffic-Panels im Dashboard. In Grafana war der entsprechende Trace mit 8 Spans sichtbar, darunter der INSERT-Span auf der PostgreSQL-Datenbank mit einer Dauer

von 3,2ms. Das Latenz-Panel im Dashboard zeigte den Request im p50-Bereich bei circa 12ms. Dieses Ergebnis bestätigt, dass die in A1 geforderte Nachvollziehbarkeit für die instrumentierten Dienste funktional gegeben ist.

Szenario 2: Erhöhte Last auf Voting-Endpunkt. Durch wiederholte manuelle Abstimmungen auf mehrere Fragen wurde eine erhöhte Anfragerate auf dem `POST /vote/`-Endpunkt erzeugt. Erwartet wurde ein sichtbarer Anstieg im Traffic-Panel und ein messbarer Effekt auf die Datenbankverbindungsauslastung. Im Dashboard stieg die Anfragerate des Backends im Beobachtungszeitraum erkennbar an. Der PostgreSQL-Verbindungszähler (`pg_stat_activity_count`) blieb im unkritischen Bereich, was bei manueller Last erwartbar ist. Das Szenario zeigt, dass Sättigungsindikatoren (A5) unter Last prinzipiell sichtbar werden, auch wenn ein belastbarer Stresstest ein automatisiertes Lastwerkzeug erfordern würde.

Szenario 3: Provozierter Fehlerfall. Ein HTTP-Request an einen nicht existierenden Endpunkt (`GET /api/nonexistent`) erzeugte einen 404-Statuscode. In Tempo war der zugehörige Trace mit einem Fehler-Tag (`http.status_code=404`) sichtbar. Im Errors-Panel des Dashboards erschien der Endpunkt in der Tabelle der fehlerhäufigsten Span-Operationen. Der Critical-Alert für die HTTP-5xx-Fehlerrate wurde durch dieses Szenario nicht ausgelöst, weil der Prototyp ausschließlich 5xx-Fehler als kritisch wertet. Ein gezielter Test des 5xx-Alerts, etwa durch temporäres Abschalten der Datenbank, wurde im Rahmen der prototypischen Umsetzung nicht durchgeführt und wäre für eine weitergehende Validierung empfehlenswert.

Die drei Szenarien demonstrieren, dass Traces, Metriken und Logs im Prototyp tatsächlich erzeugt, gespeichert und im Dashboard dargestellt werden. Sie belegen die grundlegende Funktionsfähigkeit der Pipeline, ersetzen jedoch keine systematische Last- oder Fehlerinjektionsprüfung, wie sie für eine Produktivfreigabe erforderlich wäre.

7.3 Umsetzung des Golden-Signals-Dashboards

Das Golden-Signals-Dashboard wurde als Grafana-JSON-Modell erstellt und über den Import-Mechanismus in die Grafana-Instanz geladen. Es enthält 15 Panels in fünf Sektionen, die den vier Golden Signals und einer Infrastrukturektion entsprechen. Die Panelstruktur bildet die in Kapitel 6.4 definierte Signal-Matrix ab und übersetzt die dort festgelegten Messgrößen in konkrete Visualisierungen. Im Folgenden wird für jede Sektion erläutert, welches Diagnoseziel sie verfolgt und welche Prometheus-Abfragefamilien die Panels speisen.

Die Sektion *Latency* zeigt Backend-Antwortzeiten als Perzentil-Zeitreihen (p50, p95, p99) und eine aufgeschlüsselte Latenzansicht pro HTTP-Endpunkt. Die Panels basieren auf dem Histogramm `http_server_request_duration_seconds_bucket` und nutzen die Prometheus-Funktion `histogram_quantile`. Die Perzentildarstellung folgt dem Prinzip, dass eine rein arithmetisch gemittelte Latenz langsame Anfragen verbergen kann (Ewaschuk, 2017, S. 7). Die endpunktspezifische Aufschlüsselung ermöglicht es, Pfade wie `/livepoll/find` oder `/comment/`

gezielt zu beobachten. Das Diagnoseziel dieser Sektion besteht darin, zwischen einer globalen Verlangsamung und endpunktspezifischen Ausreißern zu unterscheiden, was für die Nachverfolgung der User Journeys aus Kapitel 3.3 nützlich ist.

Die Sektion *Traffic* stellt die Anfragerate pro Dienst aus Span-Metriken (`traces_spanmetrics_calls_total`) sowie die meistfrequentierten Endpunkte als gestapeltes Balkendiagramm dar. Das Diagnoseziel ist die Sichtbarkeit von Lastverteilung und Lastspitzen. Im Kontext von `frag.jetzt` macht diese Darstellung sichtbar, ob etwa Voting-Endpunkte unter Spitzenlast dominieren, was auf die in R2 beschriebene Row-Level Contention hindeuten könnte.

Die Sektion *Errors* kombiniert eine Span-basierte Fehlerrate pro Service mit einem Stat-Panel für die HTTP-5xx-Quote und einer Tabelle der fehlerhäufigsten Span-Operationen. Diese Kombination verfolgt ein gestuftes Diagnoseziel: Das Stat-Panel zeigt, *ob* ein Fehlerproblem vorliegt, die Tabelle zeigt, *wo* es auftritt, und über den Trace-Link lässt sich der Kontext des fehlerhaften Requests in Tempo aufrufen.

Die Sektion *Saturation* umfasst JVM-Heap-Auslastung, Thread-Zähler und CPU-Nutzung als Gauge-Panel mit Schwellenwertfärbung. Die Schwellenwerte orientieren sich an den in Kapitel 6.4 festgelegten initialen Startwerten (beispielsweise 80 % Heap-Auslastung als Warngrenze). Das Diagnoseziel ist die frühzeitige Erkennung von Ressourcenengpässen, bevor sie sich als Latenzanstieg oder Fehlerrate in den anderen Sektionen manifestieren.

Die *Infrastruktur*-Sektion zeigt PostgreSQL-Verbindungen (`pg_stat_activity_count`), RabbitMQ-Queue-Tiefen (`rabbitmq_queue_messages`), Host-Ressourcen und GC-Pausen sowie den Service Graph aus Tempo. Die PostgreSQL-Panels adressieren den Sättigungsindikator der Datenbankkomponente, die in der Signal-Matrix als besonders kritisch bewertet wurde.

Sämtliche Panels verwenden `$_rate_interval` als dynamisches Zeitfenster für Rate-Berechnungen, sodass die Darstellung sich automatisch an den gewählten Betrachtungszeitraum anpasst. Das Dashboard greift ausschließlich auf die in Abschnitt 7.2 beschriebenen Datenquellen zu und erfordert keine zusätzliche Konfiguration. Die vollständige JSON-Definition befindet sich in Anhang [VERWEIS].

Das vorliegende Dashboard ist ein technisches Übersichtsdashboard, das primär DevOps-Aufgaben unterstützt. Die in Kapitel 6.5 konzipierten rollenspezifischen Sichten für Entwicklung und Product Owner wurden im Prototyp nicht umgesetzt. Sie ließen sich aus denselben Prometheus- und Loki-Datenquellen ableiten, indem die Informationsdichte und der Abstraktionsgrad der Panels angepasst werden.

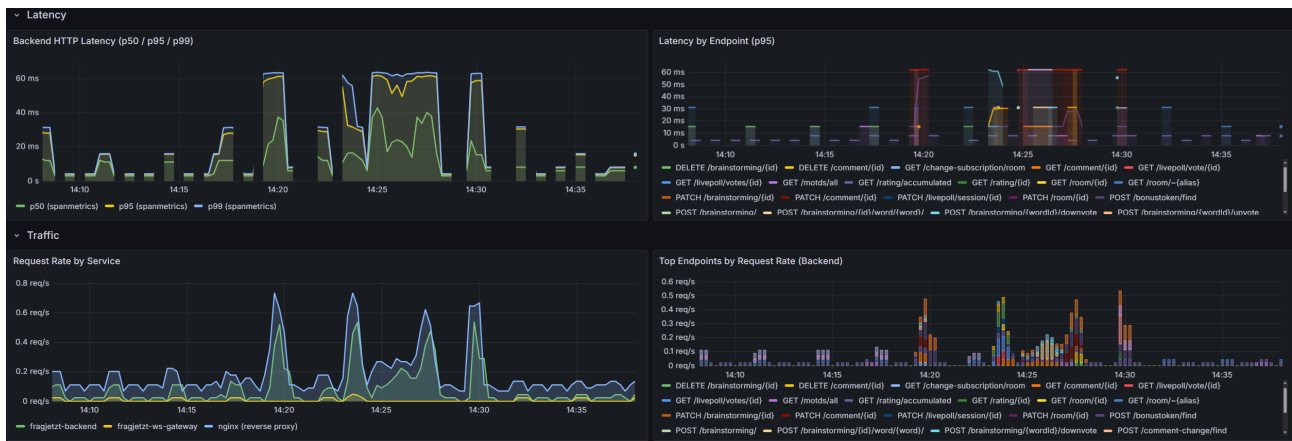


Abbildung 7.3: Ausschnitt des Golden-Signals-Dashboards mit den Sektionen *Latency* und *Traffic*. Quelle: Eigene Darstellung.

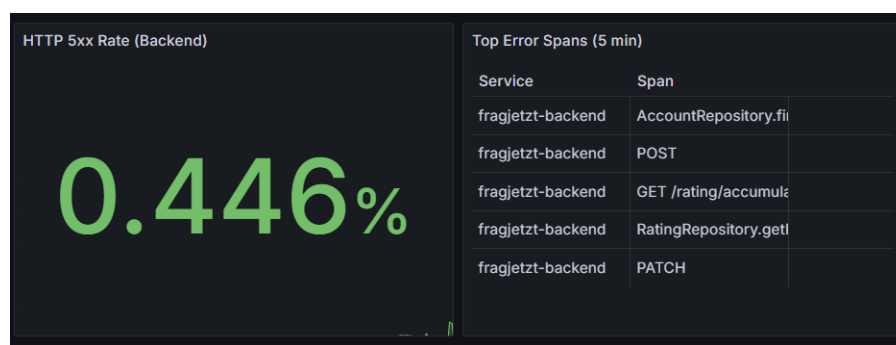


Abbildung 7.4: Ausschnitt der *Errors*-Sektion mit Dummy-Fehlerrate zur Veranschaulichung der Fehlerindikatoren. Quelle: Eigene Darstellung.

7.4 Alerting-Regeln und Beispiel-Runbooks

Die in Kapitel 6.6 konzipierte Alerting-Logik wird exemplarisch in konkrete Grafana-Alerting-Regeln überführt. Die Regeln nutzen Grafana Unified Alerting, das Schwellenwertbedingungen auf Prometheus-Abfragen auswertet und bei Überschreitung Benachrichtigungen über konfigurierbare Kanäle auslöst. Im Prototyp wurde ein E-Mail-Kanal konfiguriert; für den Produktivbetrieb wären zusätzlich Integrationen in Messaging-Dienste oder PagerDuty vorgesehen. Die Umsetzung adressiert Anforderung A4 (handlungsfähige Alarmierung).

Drei exemplarische Alerting-Regeln illustrieren die drei Schweregrade aus Kapitel 6.6:

Die erste Regel (*Critical*) löst aus, wenn die HTTP-5xx-Fehlerrate des Backends über fünf Minuten den Schwellenwert von 5 % überschreitet. Die Prometheus-Abfrage berechnet den Anteil der 5xx-Antworten an allen Antworten über ein gleitendes Fenster. Ein Alarm auf dieser Ebene signalisiert eine nutzerwirksame Beeinträchtigung und erfordert sofortiges Eingreifen.

Die zweite Regel (*Warning*) reagiert auf eine PostgreSQL-Verbindungsauslastung über 70 % des konfigurierten Limits von 100 Verbindungen. Der Schwellenwert basiert auf dem Indikator `pg_stat_activity_count` im Verhältnis zur konfigurierten Maximalgröße. Ein wachsender Verbindungspool deutet auf einen bevorstehenden Engpass hin, der rechtzeitig adressiert werden

kann.

Die dritte Regel (*Warning*) überwacht die Backend-Latenz im p95-Bereich und alarmiert bei einer anhaltenden Überschreitung von 500 ms über drei Minuten.

Alle drei Regeln folgen dem symptomorientierten Ansatz, der in der SRE-Literatur als Kern wirksamer Alerting-Konfigurationen beschrieben wird: Sie reagieren auf nutzerwirksame Zustände, nicht auf interne Ursachenindikatoren (Ewaschuk, 2017, S. 11–12). Die Evaluationsfenster von drei bis fünf Minuten fungieren als Debounce-Mechanismus, der transiente Spitzen toleriert und dadurch die Zahl falsch-positiver Alarmer reduzieren soll (Vinnakota & Kolla, 2025, S. 176). Jede Regel enthält in ihrer Annotation-Konfiguration einen Link zum zugehörigen Runbook sowie zum relevanten Dashboard-Panel, sodass beim Eintreffen des Alarms der operative Kontext unmittelbar verfügbar ist. Vinnakota und Kolla (2025, S. 174) betonen, dass jeder Alarm vier Fragen beantworten muss: Was ist betroffen? Warum vermutlich? Welche Sofortmaßnahme? Wer ist zuständig? Die Annotation-Links adressieren die ersten drei Fragen; die Zuständigkeitsfrage bleibt im Prototyp offen, da frag.jetzt derzeit kein formales On-Call-Modell betreibt.

Einschränkend ist zu vermerken, dass die drei Regeln im Prototyp angelegt und konfiguriert wurden und eine erste Störungsinjektion den 5xx-Alert auslöste. Die kurzfristige Fehlerdiagnostik blieb dennoch eingeschränkt, weil die zugehörigen Fehler-Metriken in kurzen Zeitfenstern keine konsistent aktuellen Werte lieferten. Eine weitergehende Validierung durch systematische Störungsinjektionen, etwa mit kontrollierten Lastprofilen und reproduzierbaren Zeitfenstern, wäre erforderlich.

Im Vergleich zu den in Kapitel 6.7 skizzierten anomaliebasierten Verfahren bleibt der Ansatz statisch: Die Schwellenwerte passen sich nicht automatisch an Lastschwankungen an, und es findet keine automatische Korrelation zwischen verschiedenen Alarmsignalen statt. Vinnakota und Kolla (2025, S. 173) weisen darauf hin, dass starre Schwellenwerte in dynamischen Umgebungen saisonale Muster und Abhängigkeiten zwischen Diensten unzureichend berücksichtigen. Für frag.jetzt mit seinen kalendergebundenen Lastspitzen während Vorlesungszeiten wäre eine spätere Ergänzung um dynamische Schwellen sinnvoll. Die konkreten Alerting-Definitionen im Grafana-JSON-Format befinden sich in Anhang [VERWEIS].

Ergänzend wurden zwei Runbooks als strukturierte Markdown-Dokumente erstellt, die der in Kapitel 6.6 definierten Fünf-Felder-Struktur (Symptom, Kontext, Diagnose, Behebung, Eskalation) folgen. Das erste Runbook behandelt den Alarm „Backend-Fehlerrate über Schwellenwert“ und verweist auf das Golden-Signals-Dashboard, die Trace-Suche in Tempo und die Logabfrage in Loki als Diagnoseressourcen. Es beschreibt als Behebungsschritte die Inspektion der jüngsten Traces auf fehlerhafte Spans, die Prüfung der Datenbankverbindungen und den Neustart des Backend-Containers als Sofortmaßnahme. Das zweite Runbook adressiert den Alarm „PostgreSQL-Verbindungspool nahe Limit“ und leitet durch die Prüfung der aktiven Verbindungen über den PostgreSQL Exporter, die Identifikation langlaufender Queries und die Erhöhung der `max_connections` als Kapazitätsmaßnahme. Beide Runbooks sind bewusst so formuliert,

dass sie auch von Personen ohne tiefe frag.jetzt-Kenntnis nachvollzogen werden können. Die vollständigen Runbooks befinden sich im Anhang [VERWEIS].

7.5 Herausforderungen und Lösungen bei der Umsetzung

Die prototypische Implementierung verlief nicht friktionsfrei. Drei Kategorien von Herausforderungen traten auf, die in vergleichbaren Deployments ebenfalls zu erwarten wären und daher hier dokumentiert werden.

7.5.1 Ressourcendimensionierung

Das Trace-Backend Tempo wurde initial mit einem Speicherlimit von 256 MB konfiguriert. Unter der Trace-Last, die Backend und WS-Gateway über den OTel Agent erzeugten, erschöpfte der Container seinen verfügbaren Speicher und wurde wiederholt durch den Kernel beendet (OOM-Kill, Exit Code 137). Jeder erzwungene Neustart hinterließ unvollständige WAL-Dateien (Write-Ahead Log), die beim nächsten Start erneut Fehler verursachten. Die Lösung bestand aus zwei Schritten: der Erhöhung des Speicherlimits auf 512 MB und der einmaligen Bereinigung des WAL-Volumes.

Bissig (2024, S. 52) berichten, dass Konfigurationsfehler bei Observability-Komponenten zu Datenverlust führen können und der operative Aufwand oft unterschätzt wird. Dieses Problem veranschaulicht zudem, dass die Ressourcendimensionierung von Observability-Komponenten auf einem Single-Host-System eine sorgfältige Abstimmung erfordert, insbesondere wenn die Anwendungscontainer bereits ohne Ressourcenlimits betrieben werden (vgl. R5). Die Auto-Instrumentierung selbst erzeugt einen nicht trivialen Ressourcen-Overhead: Bissig (2024, S. 50–51) messen in einem Benchmark mit Java-Spring-Boot-Diensten einen CPU-Anstieg von rund 30 % und einen Speicherzuwachs von über 100 MiB pro instrumentiertem Service. Für die Staging-Umgebung von frag.jetzt war dieser Overhead vertretbar, müsste aber bei einer Produktiveinführung in die Kapazitätsplanung einfließen.

7.5.2 Netzwerktopologie und Dienstkommunikation

Tempo generiert aus eingehenden Traces automatisch Span-Metriken, die über Remote Write an Prometheus übermittelt werden. Diese Metriken bilden die Grundlage für den Service Graph in Grafana. Damit dieser Mechanismus funktioniert, müssen Tempo und Prometheus innerhalb desselben Docker-Netzwerks erreichbar sein. Im initialen Deployment waren die Observability-Dienste in einem eigenen Netzwerk isoliert und Prometheus war zwar zusätzlich im Applikationsnetzwerk eingebunden, Tempo jedoch nicht. Der Remote-Write-Export scheiterte dadurch an der Netzwerktrennung, und der Service Graph blieb leer. Die Lösung bestand darin, Prometheus in beiden Netzwerken verfügbar zu machen, sodass es sowohl von Tempo als auch von den Applikations-Exportern erreichbar ist.

Dieses Problem verdeutlicht eine Besonderheit Docker-Compose-basierter Deployments: Anders als in Kubernetes, wo alle Pods standardmäßig über ein flaches Netzwerk kommunizieren, erfordern separate Compose-Projekte explizite Netzwerkkonfigurationen. Für die Übertragbarkeit der Strategie auf andere Plattformen ist dieser Unterschied wesentlich.

7.5.3 Signalqualität und Rauschunterdrückung

Die Auto-Instrumentierung über den OTel Java Agent erfasst grundsätzlich alle erkennbaren Operationen, ohne zwischen geschäftsrelevanten und internen Abläufen zu unterscheiden. Im Prototyp führte dies dazu, dass periodische Hintergrundprozesse wie der `ScheduledMessageSender` des WS-Gateways (Sendeintervall 50 ms) und der `MailScheduler` des Backends die Trace-Liste in Grafana dominierten. Diagnostisch relevante Traces, etwa Nutzeranfragen zum Fragestellen oder Abstimmen, waren in dieser Masse kaum auffindbar.

Albuquerque und Correia (2025, S. 10) weisen darauf hin, dass Sampling-Strategien erforderlich werden, um den Overhead der Tracing-Instrumentierung in ein vertretbares Verhältnis zu setzen. Der hier gewählte Ansatz geht einen Schritt weiter: Statt alle Spans gleichmäßig zu sampeln und dabei auch relevante zu verlieren, wurden gezielt Span-Namen identifiziert, die keinen diagnostischen Wert besitzen, und auf Collector-Ebene verworfen. Ein `filter/traces`-Processor im OTel Collector verwirft Spans mit definierten Namen (unter anderem `ScheduledMessageSender.send`, `UserRegistryTask.run`, `MailScheduler.onTick` und `clientInboundChannel.process`), bevor sie an Tempo weitergeleitet werden. Dieser Filter ist transparent in der Collector-Konfiguration hinterlegt und kann bei veränderten Anforderungen angepasst werden. Das Vorgehen setzt das in Kapitel 6.2 formulierte Prinzip um, das Verhältnis von diagnostischem Nutzen zu Datenvolumen gezielt zu steuern.

7.6 Zwischenfazit

Die prototypische Implementierung zeigt, dass die in Kapitel 6 konzipierte Strategie mit vertretbarem Aufwand auf einem einzelnen Staging-Host umsetzbar ist. Die Telemetrie-Pipeline erzeugt für die instrumentierten Dienste Traces, Metriken und Logs, die im Dashboard entlang der Golden Signals dargestellt und über das Alerting-Framework mit Runbooks verknüpft werden. Die exemplarischen Validierungsszenarien in Abschnitt 7.2.5 belegen die grundlegende Funktionsfähigkeit dieser Kette. Tabelle 7.2 ordnet die umgesetzten Komponenten den Anforderungen aus Kapitel 3.5 zu und benennt die jeweiligen Einschränkungen.

Die fünf Anforderungen werden durch den Prototyp in unterschiedlicher Tiefe bearbeitet, aber nicht in allen Fällen vollständig erfüllt. Die wesentlichen Lücken betreffen die fehlende Frontend-Instrumentierung (A2), die fehlende Abdeckung des AI Providers (A1) und die nicht systematisch verifizierten Alerting-Regeln (A4). Die aufgetretenen Herausforderungen bei Ressourcendimensionierung, Netzwerktopologie und Signalqualität sind nicht spezifisch für frag.jetzt, sondern typisch für Docker-Compose-basierte Observability-Deployments auf Single-Host-Systemen (Bissig, 2024,

Tabelle 7.2: Zuordnung der prototypischen Umsetzung zu den Observability-Anforderungen.
Quelle: Eigene Darstellung.

Anf.	Umsetzung im Prototyp	Einschränkung
A1	Distributed Tracing über OTel Agent; Span-Waterfall und Service Graph in Grafana; Trace-basierte Validierung in Szenario 1	Nur Backend und WS-Gateway instrumentiert; AI Provider und Frontend nicht abgedeckt
A2	Applikations- und Infrastrukturmetriken in Prometheus; Dashboard-Sektionen entlang der vier Golden Signals	Keine Browser-Telemetrie; Frontend-Ebene fehlt
A3	Log-Weiterleitung an Loki über OTel Agent mit automatischer Trace-ID-Injektion	Debug-Level-Logging als Demonstrationskonfiguration; kein strukturiertes JSON-Format
A4	Drei symptomorientierte Alerting-Regeln mit Debounce-Logik; zwei Runbooks mit Fünf-Felder-Struktur; 5xx-Alert in Störungsszenario ausgelöst	Kurzfristige Fehler-Metriken lieferten keine konsistent aktuellen Werte; keine dynamischen Schwellenwerte
A5	cAdvisor, Node Exporter, PostgreSQL Exporter, RabbitMQ-Plugin; Infrastruktursektion im Dashboard	Im Prototyp für die vorhandene Infrastruktur abgebildet

S. 52). Ihre Dokumentation soll künftige Implementierungen erleichtern. Die Evaluation in Kapitel 8 prüft, inwieweit die erreichte Umsetzung die Forschungsfragen beantwortet und wo methodische Grenzen die Aussagekraft einschränken.

8 Evaluation der entwickelten Strategie

Das vorliegende Kapitel prüft, inwieweit die in Kapitel 6 konzipierte und in Kapitel 7 prototypisch umgesetzte Observability-Strategie die Anforderungen aus Kapitel 3 erfüllt. Abschnitt 8.1 beschreibt die angewandte Evaluationsmethodik und ihre Rückbindung an den in Kapitel 4.3 definierten Bewertungsrahmen. Abschnitt 8.2 bewertet die Zielerreichung anhand der fünf Anforderungen A1–A5 und ordnet die Ergebnisse in abgestufte Erfüllungsgrade ein. Abschnitt 8.3 ergänzt drei übergreifende Bewertungsdimensionen, und Abschnitt 8.4 reflektiert die methodischen Grenzen der Evaluation.

8.1 Evaluationsmethodik

Kapitel 4.3 hat für die Gesamtstrategie drei Evidenzquellen festgelegt: Artefaktanalyse, Konfigurations- und Instrumentierungsprüfung sowie Last- und Störungsszenarien. Gleichzeitig wurden für jede Anforderung A1–A5 konkrete Prüffragen formuliert, deren Beantwortung den Kern der nachfolgenden Bewertung bildet. Im Prototyp (Kapitel 7) wurden die ersten beiden Quellen ausgeschöpft. Die dritte blieb auf manuelle Plausibilitätstests beschränkt. Um diese Lücke zu verkleinern, wurde die Evaluation um gezielte Belastungsszenarien auf dem Staging-Server erweitert.

Konkret wurden drei Testszenarien durchgeführt, die jeweils auf die Auslösung einer der drei konfigurierten Alerting-Regeln abzielten. Für den Latenz-Alert wurde CPU-Last auf den Backend-Container erzeugt, kombiniert mit parallelem HTTP-Traffic. Für den Fehler-Alert wurde die Datenbankverbindung temporär unterbrochen, um eine erhöhte HTTP-5xx-Rate zu provozieren. Für den Sättigungs-Alert wurden langlaufende PostgreSQL-Sessions geöffnet, um den Verbindungspool zu erschöpfen. Die Szenarien folgten einem einheitlichen Grundprinzip: Last oder Störungen gezielt injizieren, die erwartete Reaktion in Dashboard und Alerting beobachten und den Diagnosepfad aus den Runbooks exemplarisch nachvollziehen. Der prototypische Charakter dieser Tests wird in Abschnitt 8.4 eingeordnet.

Ergänzend wurde die in Kapitel 7 noch nicht validierte bidirektionale Log-Trace-Korrelation konfiguriert und auf dem Staging-Server verifiziert. Die Ergebnisse fließen in die Bewertung von A1 und A3 ein.

8.2 Anforderungsbezogene Bewertung

Die Bewertung folgt den Prüffragen aus Abschnitt 4.3. Für jede Anforderung wird der Befund aus der prototypischen Umsetzung und den Belastungsszenarien dargelegt und anschließend die verbleibende Einschränkung benannt. Da es sich um eine prototypische Validierung handelt, werden die Ergebnisse nicht binär als erfüllt oder nicht erfüllt eingestuft, sondern in abgestufte

Erfüllungsgrade differenziert. Tabelle 8.1 gibt vorab einen kompakten Überblick.

Tabelle 8.1: Ergebnissynthese der anforderungsbezogenen Evaluation. Quelle: Eigene Darstellung.

Anf.	Kurzbefund	Wesentliche Einschränkung	Erfüllungsgrad
A1	Backend-Pfad rekonstruierbar, Gesamtsystem unvollständig	AI Provider und Frontend nicht instrumentiert	teilweise erfüllt
A2	Drei Messebenen für instrumentierte Dienste vorhanden	Keine Browser-Telemetrie	weitgehend erfüllt
A3	Bidirektionale Log-Trace-Korrelation validiert	Debug-Level-Logging, noch nicht produktionsorientiert strukturiert	erfüllt
A4	Latenz-Alert validiert, Fehler-Alert ausgelöst, Sättigungs-Alert metrisch vorbereitet	Kurzfristige Fehler-Metriken lieferten keine konsistent aktuellen Werte; Sättigungs-Alert nicht experimentell getestet	teilweise erfüllt
A5	Infrastrukturmetriken verfügbar und plausibel	Nicht alle Indikatoren unter Last geprüft	weitgehend erfüllt

Die folgenden Unterabschnitte begründen diese Einstufungen im Einzelnen.

8.2.1 A1: Ende-zu-Ende-Nachvollziehbarkeit

Prüffrage: Lässt sich ein Request Ende-zu-Ende rekonstruieren?

Die Instrumentierung über den OTel Java Agent erzeugt für jeden eingehenden HTTP-Request einen Root-Span, der untergeordnete Spans für Datenbankoperationen, RabbitMQ-Interaktionen und interne Verarbeitungsschritte enthält. Abbildung 8.1 zeigt exemplarisch einen Trace für den Endpunkt `GET /room/{id}/moderator` mit 11 Spans, darunter mehrere **SELECT**-Operationen auf der PostgreSQL-Datenbank. Der Verarbeitungspfad vom HTTP-Eingang über die Repository-Aufrufe bis zur Datenpersistierung ist vollständig rekonstruierbar. Diese Fähigkeit entspricht dem Entwurfsmuster *Distributed Tracing*, das darauf abzielt, Anfragen über Dienstgrenzen hinweg nachzuverfolgen und Latenzengpässe sowie Fehlerpunkte zu identifizieren (Albuquerque & Correia, 2025, S. 4–10).

Der Service Graph in Grafana, gespeist aus den Span-Metriken von Tempo, bildet die Kommunikationsbeziehungen zwischen den instrumentierten Diensten ab. Er zeigt drei Knoten (`fragjetzt-backend`, `fragjetzt-ws-gateway` und `fragjetzt-postgres`) mit Anfrageraten und durchschnittlichen Antwortzeiten. Damit wird die in Kapitel 6.2 beschriebene Zielarchitektur auf Ebene der erfassten Dienste bestätigt.

Die Nachvollziehbarkeit endet an den Grenzen der instrumentierten Komponenten. Der AI Provider (Python/FastAPI) und das Frontend (Angular) sind nicht instrumentiert. Ein Request,

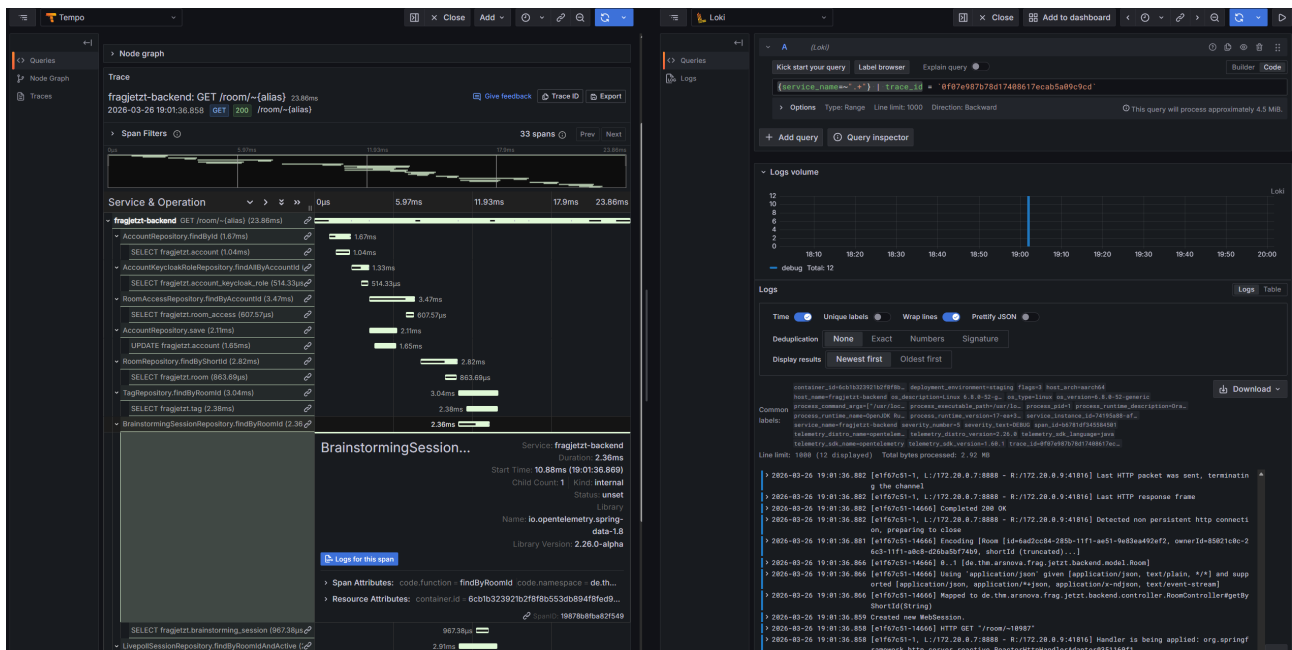


Abbildung 8.1: Trace-Waterfall in Tempo für den Endpunkt `GET /room/{id}/moderator` mit 11 Spans und zugeordneten Datenbankoperationen. Der rechte Bereich zeigt die korrelierten Loki-Logs, erreichbar über die Funktion *Logs for this span*. Quelle: Eigene Darstellung (Screenshot, Staging-Server).

der über das Backend den AI Provider aufruft, verliert an der Dienstgrenze seinen Trace-Kontext. Für den instrumentierten Backend-Pfad ist die Request-Rekonstruktion funktional gegeben. Für das Gesamtsystem, insbesondere für User Journey 3 (KI-Zusammenfassung), bleibt sie unvollständig. Der Erfüllungsgrad wird daher als *teilweise erfüllt* bewertet.

8.2.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals

Prüffrage: Deckt die Instrumentierung alle drei Messebenen ab?

Das Golden-Signals-Dashboard bildet die vier Dimensionen Latency, Traffic, Errors und Saturation in separaten Sektionen ab. Die Latenz-Sektion zeigt Backend-Antwortzeiten als Perzentil-Zeitreihen (p50, p95, p99) und aufgeschlüsselt nach Endpunkt. Im Belastungsszenario stieg die p95-Latenz unter CPU-Last von rund 20 ms auf über 500 ms, was im Dashboard sofort sichtbar wurde.

Die Traffic-Sektion stellt die Anfragerate pro Dienst aus Span-Metriken dar. Während der Belastungstests war der Anstieg der Request-Rate für das Backend klar erkennbar. Die Aufschlüsselung nach Endpunkt machte sichtbar, welche Pfade den Traffic dominierten. Die Errors-Sektion kombiniert ein Stat-Panel für die HTTP-5xx-Quote mit einer Tabelle der fehlerhäufigsten Span-Operationen. Nach dem temporären Stopp der Datenbank stieg die 5xx-Rate auf 0,446 % und die betroffenen Endpunkte erschienen in der Tabelle.

Die Saturation-Sektion zeigt JVM-Heap-Auslastung, Thread-Zähler und CPU-Nutzung. Die Infrastruktur-Sektion ergänzt PostgreSQL-Verbindungszähler, RabbitMQ-Queue-Tiefen und

Host-Ressourcen. Albuquerque und Correia (2025, S. 10, 16) beschreiben *Application Metrics* und *Infrastructure Metrics* als komplementäre Entwurfsmuster, deren Zusammenspiel erst eine vollständige Sicht auf das Verhalten eines Dienstes ermöglicht. Das vorliegende Dashboard setzt genau dieses Zusammenspiel um, indem es applikationsbezogene Span-Metriken und Exporter-basierte Infrastrukturmetriken in einer gemeinsamen Oberfläche zusammenführt.

Die drei Messebenen, Applikationsmetriken (über den OTel Agent), Infrastrukturmetriken (über Exporter) und Trace-basierte Span-Metriken (über Tempo), sind damit für die instrumentierten Dienste abgedeckt. Die Lücke betrifft die fehlende Frontend-Telemetrie, die keine browserseitige Messung von Latenzen oder Fehlerraten ermöglicht. Der Erfüllungsgrad wird als *weitgehend erfüllt* eingestuft.

8.2.3 A3: Strukturiertes, korrelierbares Logging

Prüffrage: Können Logeinträge über Korrelations-IDs verknüpft werden?

Die bidirektionale Korrelation zwischen Logs und Traces konnte auf dem Staging-Server nachgewiesen werden. Der OTel Java Agent injiziert die Felder `trace_id` und `span_id` automatisch als Stream-Labels in die an Loki weitergeleiteten Logeinträge. In Grafana sind beide Korrelationsrichtungen navigierbar: Von einem Trace in Tempo führt die Funktion *Logs for this span* zu einer Loki-Abfrage, die nach der zugehörigen Trace-ID und dem Dienstnamen filtert. Abbildung 8.1 zeigt diesen Pfad im rechten Bereich. In der Gegenrichtung enthält jeder Log-Eintrag in Loki ein klickbares *TraceID*-Feld, das direkt zum zugehörigen Trace in Tempo navigiert. Diese Verknüpfung wurde über ein *Derived Field* mit Label-Matching in der Loki-Datenquellenkonfiguration realisiert.

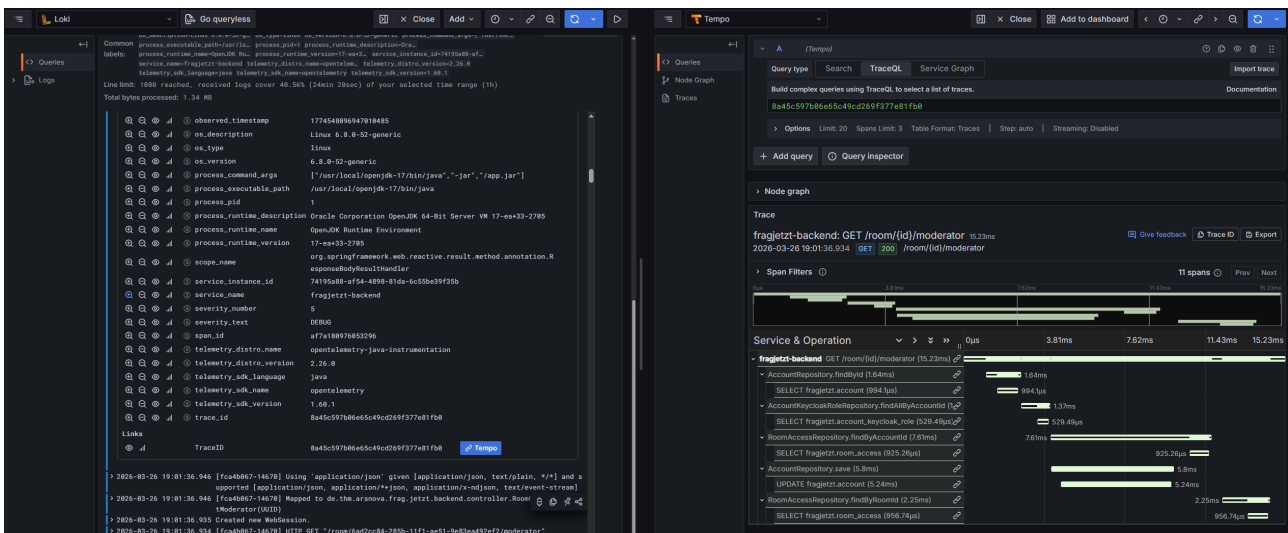


Abbildung 8.2: Log-to-Trace-Korrelation in Loki: Ein Logeintrag mit klickbarem TraceID-Link (links) navigiert zum zugehörigen Trace-Waterfall in Tempo (rechts). Quelle: Eigene Darstellung (Screenshot, Staging-Server).

Die Korrelation funktioniert zuverlässig für alle Logeinträge, die innerhalb eines aktiven Trace-

Kontexts emittiert werden. Einschränkend ist anzumerken, dass die Logging-Konfiguration im Prototyp Debug-Level-Ausgaben umfasst, die ein hohes Volumen erzeugen. Für den Produktivbetrieb wäre eine Reduktion auf strukturierte, geschäftsrelevante Logeinträge erforderlich, wie in Abschnitt 7.2.4 beschrieben. Da die Kernfunktion der Korrelierbarkeit jedoch nachgewiesen wurde, wird A3 als *erfüllt* bewertet.

8.2.4 A4: Handlungsfähige Alarmierung

Prüffrage: Reagieren Alarme auf nutzerwirksame Zustände und enthalten sie ausreichend Kontext für eine Erstreaktion?

Die Evaluation der Alarmierungsqualität profitiert von den gezielten Belastungsszenarien, die über die in Kapitel 7 beschriebenen manuellen Plausibilitätstests hinausgehen. Abbildung 8.3 zeigt den Latenz-Alert im Zustand *Firing*, ausgelöst durch künstlich erzeugten CPU-Druck auf den Backend-Container.

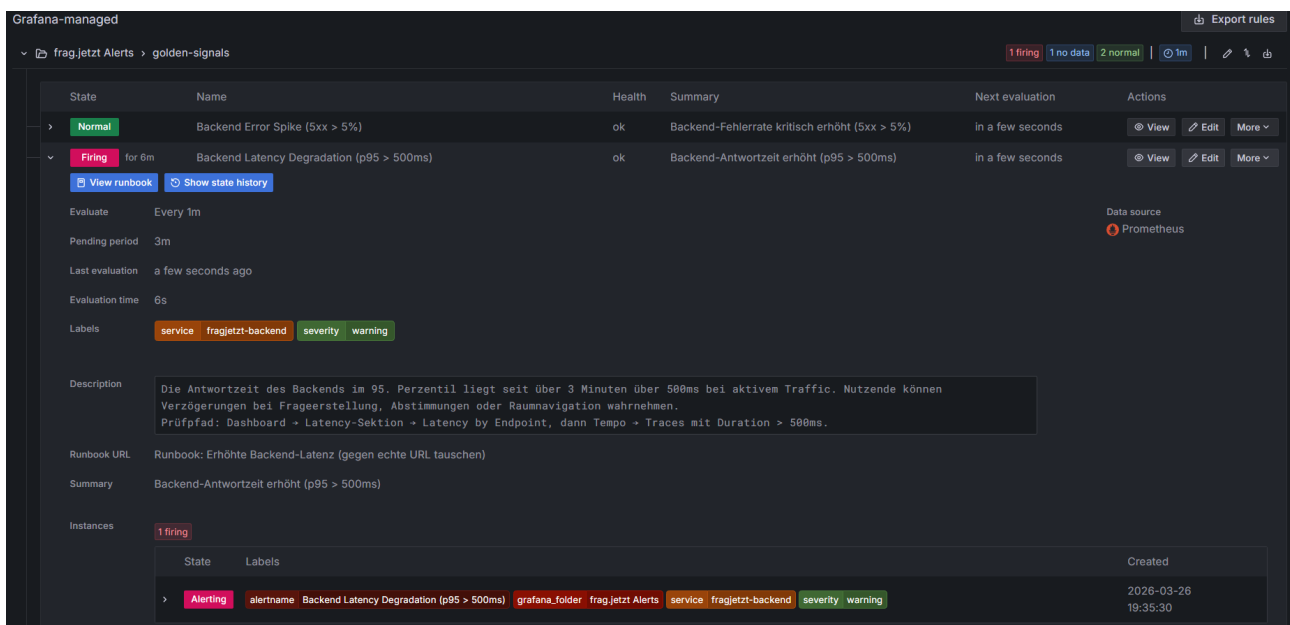


Abbildung 8.3: Latenz-Alert im Zustand *Firing* nach 6 Minuten. Die Annotations enthalten Symptombeschreibung, Prüfpfad und Runbook-Link. Quelle: Eigene Darstellung (Screenshot, Staging-Server).

Der Alarm enthält in seinen Annotations eine deutschsprachige Symptombeschreibung, einen Prüfpfad zum zugehörigen Dashboard-Panel und einen Runbook-Link. Damit sind drei der vier von Vinnakota und Kolla (2025, S. 174) geforderten Informationselemente unmittelbar verfügbar: Was ist betroffen, warum vermutlich und welche Sofortmaßnahme eingeleitet werden kann. Die vierte Frage (Wer ist zuständig?) bleibt offen, da frag.jetzt kein formales On-Call-Modell betreibt.

Die Multi-Signal-Logik der Regeln bewährte sich in einem konkreten Fall. Der Latenz-Alert erfordert neben der p95-Überschreitung gleichzeitig eine Mindest-Traffic-Rate von 0,1 req/s. In Leerlaufphasen ohne aktive Requests verhinderte diese Bedingung, dass der Alert fälschlicher-

weise auslöste, obwohl die Latenz einzelner sporadischer Anfragen über dem Schwellenwert lag. Dieses Verhalten entspricht dem in Kapitel 6.6 formulierten Grundsatz der symptomorientierten Alarmierung und deckt sich mit der Empfehlung, mehrere korrelierte Signale zu einem aussagekräftigeren Alarm zu kombinieren, anstatt auf isolierte Metriken zu reagieren (Vinnakota & Kolla, 2025, S. 175).

Der Fehler-Alert (HTTP-5xx-Rate über 5 %) konnte im Testzeitraum zwar zuverlässig ausgelöst werden, die Metrik `http_server_request_duration_seconds_count` zeigte ebenso 500er-Antworten in Prometheus, doch die Rate-Berechnung lieferte bei kurzen Betrachtungszeiträumen keine konsistenten Werte. Die Ursache liegt vermutlich in einer Diskrepanz zwischen dem dynamisch berechneten Abfragefenster und der tatsächlichen Erfassungs- bzw. Exportsemantik der OTel-basierten Metriken (vgl. Abschnitt 8.4). Der Sättigungs-Alert für PostgreSQL-Verbindungen wurde nicht als eigenständiges Szenario getestet. Die zugrunde liegende Metrik `pg_stat_activity_count` war jedoch während aller Tests korrekt in Prometheus verfügbar und im Dashboard sichtbar.

Für den Latenz-Alert ist die Alerting-Konzeption aus Kapitel 6.6 damit funktional validiert: Der Alarm feuert bei nutzerwirksamer Symptomverschlechterung und stellt über Annotations den geforderten Handlungskontext bereit. Für den Fehler-Alert zeigt sich kein Auslöseproblem, sondern ein Datenqualitätsproblem in der kurzfristigen Metrikdarstellung: Die Fehler-Metriken lieferten in kurzen Zeitfenstern keine konsistent aktuellen Werte. Dieser Befund stellt die konzeptionelle Validität der Regel nicht infrage, schränkt aber die operative Kurzfristdiagnose ein. Für den Sättigungs-Alert ist die metrische Grundlage vorhanden, eine experimentelle Absicherung steht jedoch noch aus. In der Gesamtbetrachtung wird A4 als *teilweise erfüllt* eingestuft.

8.2.5 A5: Beobachtbarkeit der Infrastrukturschicht

Prüffrage: Werden Container-Ressourcen, Datenbankpool-Auslastung und Queue-Tiefen erfasst?

Die Infrastruktur-Sektion des Dashboards beantwortet diese Frage positiv. Abbildung 8.4 zeigt die Panel-Übersicht mit PostgreSQL-Verbindungszählern, RabbitMQ-Queue-Tiefen, Nginx-Verbindungsmetriken, GC-Pausen-Dauer und Host-Ressourcen als Gauge-Darstellung. Der im unteren Bereich eingebettete Service Graph visualisiert die Abhängigkeitsstruktur der instrumentierten Dienste.

Während der Belastungstests zeigte das PostgreSQL-Panel einen Anstieg der aktiven Verbindungen von einem Normalniveau um 3 auf temporär über 10, als das Backend unter CPU-Druck mehr parallele Datenbankzugriffe öffnete. Die RabbitMQ-Queue-Tiefe blieb im niedrigen Bereich, was bei der gewählten Teststrategie ohne Message-Rückstau erwartbar war. Die Host-Gauges für Speicher (35,2 %) und CPU (2,35 %) lagen während des normalen Betriebs im unkritischen Bereich.



Abbildung 8.4: Infrastruktur-Sektion des Golden-Signals-Dashboards mit PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefe, Host-Ressourcen und Service Graph. Quelle: Eigene Darstellung (Screenshot, Staging-Server).

Alle fünf in Abschnitt 7.2.3 beschriebenen Infrastruktur-Exporter lieferten während des gesamten Evaluationszeitraums durchgängig Metriken. Dabei ist zu berücksichtigen, dass Infrastrukturmetriken ihrer Natur nach eher auf mögliche Engpässe hinweisen als auf die tatsächliche Ursache eines Problems (Albuquerque & Correia, 2025, S. 20). Diese Indikatoren entfalten ihren diagnostischen Wert erst im Zusammenspiel mit den applikationsseitigen Metriken aus A2. Da die Metriken verfügbar und plausibel sind, nicht aber alle abgeleiteten Indikatoren unter gezielter Last geprüft wurden, wird A5 als *weitgehend erfüllt* bewertet.

8.3 Übergreifende Bewertungsdimensionen

Über die anforderungsbezogene Einzelbewertung hinaus hat Abschnitt 4.3 drei übergreifende Dimensionen definiert, die den praktischen Wert der Strategie einordnen.

Stakeholder-Tauglichkeit. Forschungsfrage 3 zielt auf ein Alerting-Framework, das handlungsorientierte Informationen bereitstellt. Die Frage, ob diese Informationen die richtigen Adressaten erreichen, ist dabei eng mit der rollenbasierten Aufbereitung aus Kapitel 6.5 verknüpft. Niedermaier et al. (2019, S. 12) betonen, dass Beobachtbarkeit nicht nur ein technisches, sondern ein organisatorisches Konzept ist, das eine klare Governance mit definierten Rollen, Verantwortlichkeiten und Prozessen erfordert. Vor diesem Hintergrund fällt die Bewertung der Stakeholder-Tauglichkeit differenziert aus.

Für die *DevOps-Perspektive* ist der Prototyp funktional nutzbar: Das Golden-Signals-Dashboard liefert Latenz-, Traffic-, Fehler- und Sättigungsindikatoren in einer gemeinsamen Oberfläche, die Alerting-Regeln reagieren auf nutzerwirksame Symptome, und die Runbooks bieten strukturierte

Diagnosepfade. Für die *Entwicklungsperspektive* ist die Unterstützung teilweise gegeben. Grafana Explore ermöglicht die direkte Abfrage von Traces, Logs und Metriken, und die bidirektionale Korrelation zwischen Loki und Tempo erleichtert die Fehlersuche im Kontext konkreter Requests. Ein dediziertes Entwickler-Dashboard mit endpunktspezifischen Detailansichten wurde jedoch nicht umgesetzt. Für die *Product-Owner-Perspektive* fehlt im Prototyp eine eigene Sicht. Aggregierte Verfügbarkeitsindikatoren oder Nutzungskennzahlen, wie sie in Kapitel 6.5 konzipiert wurden, sind technisch aus den vorhandenen Prometheus- und Span-Metriken ableitbar, aber nicht als separates Dashboard realisiert. Forschungsfrage 3 wird damit für DevOps prototypisch, für die Entwicklung teilweise und für Product Owner bisher nur konzeptionell beantwortet.

Abdeckung der Risikopunkte. Von den sechs in Kapitel 3.4 identifizierten Risikopunkten werden R2 (DB-Contention bei Massenabstimmungen), R3 (Intransparenz ohne Tracing) und R5 (Ressourcenverdrängung auf dem Host) durch den Prototyp direkt adressiert. R3 wurde durch die Einführung von Distributed Tracing beseitigt. R2 ist über die PostgreSQL-Verbindungsmetriken und die endpunktspezifische Latenzaufschlüsselung beobachtbar geworden. R5 wird durch die Infrastrukturmetriken und die Sättigungsalarme abgedeckt. R1 (fehlende Timeouts) und R4 (Backend als Single Point of Failure) bleiben als architektonische Defizite bestehen, die über Beobachtbarkeit sichtbar, aber nicht lösbar sind. R6 (fehlende Ressourcenlimits auf Applikationscontainern) wurde für die Observability-Container selbst adressiert, bleibt für die Applikationscontainer aber offen.

Umsetzungsrealismus. Die prototypische Implementierung bestätigt, dass der LGTM-Stack auf einem einzelnen Host mit vertretbarem Aufwand deploybar ist. Die Gesamtdauer von der initialen Stack-Konfiguration bis zum funktionierenden Dashboard betrug wenige Arbeitstage, wobei der Großteil der Zeit auf die in Abschnitt 7.5 dokumentierten Herausforderungen entfiel, insbesondere die Netzwerktopologie und die Ressourcendimensionierung von Tempo. Bissig (2024, S. 52) berichten von vergleichbaren Erfahrungen und identifizieren den operativen Konfigurationsaufwand als den wesentlichen Kostenfaktor bei der Einführung von Observability. Dass die größte mentale Belastung nicht aus der Instrumentierung selbst, sondern aus dem Betrieb der Observability-Komponenten resultiert, deckt sich mit den Beobachtungen im vorliegenden Prototyp. Für frag.jetzt lässt sich festhalten, dass die Strategie mit den vorhandenen Mitteln implementierbar ist. Der laufende Betrieb des Observability-Stacks erfordert allerdings ein eigenes Ressourcenbudget, das bei der Kapazitätsplanung des Hosts berücksichtigt werden muss.

8.4 Grenzen und methodische Einschränkungen

Die Evaluation unterliegt Einschränkungen, die bei der Interpretation der Ergebnisse berücksichtigt werden müssen.

Erstens, unvollständige Dienstabdeckung. Backend und WS-Gateway sind instrumentiert, AI Provider und Frontend nicht. Die Bewertung von A1 und A2 gilt daher nur für einen Teil des

Gesamtsystems. Eine Aussage über die Beobachtbarkeit aller drei User Journeys aus Kapitel 3.3 ist mit dem aktuellen Prototyp nicht möglich, da die KI-Zusammenfassung (Journey 3) den nicht instrumentierten AI Provider durchläuft.

Zweitens, Stabilität des Observability-Stacks unter Last. Während der Belastungstests wurde Tempo durch den Kernel wegen Speichererschöpfung beendet (Out of Memory-Abbruch). Dieser Ausfall löste eine Kaskade aus: Der OTel Collector konnte Trace-Daten nicht mehr exportieren und stellte den Betrieb ein. Damit fielen vorübergehend sowohl Traces als auch OTel-basierte Metriken aus. Infrastrukturmetriken über die Prometheus-Exporter blieben davon unberührt. Der Vorfall verdeutlicht, dass der Observability-Stack selbst eine Fehlerquelle darstellt, deren Auswirkungen auf einem Single-Host-System unmittelbar die Beobachtbarkeit des Gesamtsystems beeinträchtigen. Die in Kapitel 7.5.1 beschriebene Ressourcendimensionierung ist ein kritischer Betriebsparameter, der kontinuierlicher Überwachung bedarf.

Drittens, eingeschränkte Aussagekraft der Rate-Berechnung. Die PromQL-Variable `$_rate_interval`, die Grafana dynamisch an das gewählte Zeitfenster anpasst, lieferte bei kurzen Betrachtungszeiträumen inkonsistente Ergebnisse für die HTTP-5xx-Rate. Die Diskrepanz deutet auf ein Zusammenwirken mehrerer Faktoren hin: das dynamisch berechnete Abfragefenster, die Aggregationslogik von PromQL und die Erfassungssemantik der per Push übermittelten OTel-Metriken, die kein reguläres Scrape-Intervall aufweisen. Die genaue Ursache ist vermutlich architekturenspezifisch und nicht allein aus dem Dashboard-Verhalten ableitbar. In längeren Zeitfenstern (ab etwa 12 Stunden) waren die Werte korrekt. Diese Einschränkung betrifft die Dashboard-Darstellung, nicht jedoch die übrigen Alerting-Regeln, die mit festen Zeitfenstern arbeiten.

Viertens, fehlende Produktivdaten. Alle Tests wurden auf dem Staging-Server mit synthetischer Last durchgeführt. Reale Nutzungsmuster, insbesondere die in Kapitel 3.3 beschriebenen Lastspitzen während Lehrveranstaltungen mit vielen gleichzeitig Teilnehmenden, konnten nicht abgebildet werden. Die Schwellenwerte der Alerting-Regeln basieren auf Erfahrungswerten und müssten im Produktivbetrieb anhand realer Lastprofile kalibriert werden, wie in Kapitel 4.2 als vierter Schritt des Ableitungsverfahrens vorgesehen.

Fünftens, prototypische Testmethodik. Die Belastungsszenarien wurden manuell über Konsolenschleifen und Container-Operationen durchgeführt, nicht über ein systematisches Lasttestwerkzeug. Die Reproduzierbarkeit und die Kontrolle über Lastvariablen wie Anfragerate, Parallelität und Dauer waren dadurch begrenzt. Für eine belastbarere Validierung wäre der Einsatz eines dedizierten HTTP-Lastgenerators empfehlenswert, der kontrollierte Lastprofile auf die Staging-Umgebung erzeugt und dabei die Auswirkungen auf die Golden-Signals-Metriken systematisch erfasst.

In der Zusammenschau zeigt die Evaluation, dass die konzipierte Strategie für den instrumentierten Backend-Kern funktional tragfähig ist. Traces, Metriken und Logs sind für die erfassten Dienste verfügbar, korrelierbar und entlang der Golden Signals aufbereitet. Der Latenz-Alert

reagiert auf nutzerwirksame Symptome und stellt über Annotations und Runbook-Links den für eine Erstreaktion benötigten Kontext bereit. Die Vollständigkeit und operative Belastbarkeit der Strategie bleiben jedoch durch die fehlende Frontend- und AI-Provider-Instrumentierung, die begrenzte Lasttesttiefe und eine noch unvollständige Alert-Validierung eingeschränkt. Diese Lücken betreffen die Reichweite der Beobachtbarkeit und die Kalibration an realen Betriebsdaten. Sie sind in einer iterativen Weiterentwicklung adressierbar, ohne die Grundarchitektur zu verändern.

9 Diskussion

Die Evaluation in Kapitel 8 hat die Ergebnisse der prototypischen Umsetzung an den Anforderungen aus Kapitel 3 gemessen und dabei sowohl funktionale Stärken als auch offene Lücken identifiziert. Das vorliegende Kapitel ordnet diese Befunde in den Kontext der wissenschaftlichen Literatur ein, bewertet die Übertragbarkeit auf vergleichbare Plattformen und grenzt den wissenschaftlichen vom praktischen Beitrag der Arbeit ab.

9.1 Einordnung der Ergebnisse im Verhältnis zur Literatur

Die Arbeit bestätigt mehrere Grundannahmen der Observability-Literatur auf konzeptioneller Ebene und konkretisiert sie zugleich um operative Erkenntnisse aus der prototypischen Umsetzung. Ein erheblicher Teil der herangezogenen Literatur behandelt Observability entweder auf konzeptioneller Ebene oder entlang konkreter Frameworks und Werkzeuge (Faseeha et al., 2025, S. 72034), während die Integration heterogener Telemetriequellen und die Interoperabilität zwischen verschiedenen Werkzeugen als eigenständige Herausforderung beschrieben werden (Sakinala, 2025, S. 105). Die vorliegende Arbeit ergänzt diese Perspektive um fallbezogene Befunde aus einer konkreten Implementierung.

Auf der Bestätigungsseite stehen drei Befunde. Erstens erweist sich die Kombination der drei Telemetriepfeiler Logs, Metriken und Traces als praktisch tragfähig. Albuquerque und Correia (2025, S. 22) beschreiben *Application Metrics* und *Infrastructure Metrics* als komplementäre Entwurfsmuster, deren Zusammenspiel eine vollständige Sicht auf einen Dienst ermöglicht. Die vorliegende Arbeit stützt diesen Befund, indem sie zeigt, dass erst die Verknüpfung von Span-basierten Applikationsmetriken mit Exporter-basierten Infrastrukturmetriken in einem gemeinsamen Dashboard eine diagnostisch nutzbare Gesamtsicht erzeugt. Ohne diese Verknüpfung blieben die Infrastrukturindikatoren interpretationsarm, da sie zwar mögliche Engpässe anzeigen, nicht aber deren tatsächliche Ursache offenlegen (Albuquerque & Correia, 2025, S. 20).

Zweitens hat sich das Prinzip der symptomorientierten Alarmierung im Prototyp für den Latenz- und den Fehler-Alert bewährt, die jeweils nach einer Mehrsignal-Logik aufgebaut sind. Der Latenz-Alert verlangt neben der p95-Überschreitung gleichzeitig eine Mindest-Traffic-Rate als Vorbedingung und verhinderte in der Evaluation falsch-positive Auslösungen in Leerlaufphasen. Damit bestätigt die Arbeit die Empfehlung, Alarme auf nutzerseitig wahrnehmbare Zustände auszurichten und mehrere korrelierte Größen zu einem aussagekräftigeren Alarm zu kombinieren (Vinnakota & Kolla, 2025, S. 175). Einschränkend ist anzumerken, dass der Infrastruktur-Sättigungsalert in der prototypischen Regeldefinition vereinfacht umgesetzt wurde, sodass eine seiner Alternativbedingungen auch ohne gleichzeitige Servicewirkung feuern konnte. Die Mehrsignal-Logik ist dort konzeptionell angelegt, aber nicht durchgängig implementiert.

Drittens belegt die Arbeit, dass Auto-Instrumentierung über den OTel Java Agent einen

niedrigschwelligen Einstieg in Distributed Tracing ermöglicht. Bissig (2024, S. 52) stellen fest, dass der mentale Aufwand für Auto-Instrumentierung in synthetischen Benchmarks gering ausfällt. Für frag.jetzt bestätigt sich dieser Befund insofern, als die Instrumentierung des Spring-Boot-Backends ohne Codeänderungen funktionierte und unmittelbar verwertbare Traces erzeugte.

Neben diesen Bestätigungen macht die Arbeit drei Aspekte sichtbar, die in der Literatur eher abstrakt behandelt werden und für den Fall frag.jetzt konkrete Gestalt annehmen. Der erste betrifft die operative Komplexität. Die Literatur beschreibt überwiegend konzeptionelle Modelle und empfohlene Vorgehensweisen. Die prototypische Umsetzung zeigt für den Fall frag.jetzt, dass infrastrukturelle und konfigurationsbezogene Herausforderungen einen erheblichen Einfluss auf die tatsächliche Nutzbarkeit eines Observability-Systems haben. Der OOM-Kill von Tempo, die Netzwerkprobleme in der Docker-Compose-Topologie und das Span-Rauschen durch die Auto-Instrumentierung traten als konkrete Problemkategorien auf, die in den herangezogenen Quellen nicht in dieser Spezifik behandelt werden. Niedermaier et al. (2019, S. 9) identifizieren zwar knappe Ressourcen und reaktive Implementierung als typische Herausforderungen (C7, C9), verbleiben aber auf einer übergeordneten Ebene. Die Kaskadeneffekte, die auf einem Single-Host-System durch den Observability-Stack selbst entstehen können, werden dort nicht adressiert.

Der zweite Aspekt betrifft die Signalqualität. Die Literatur diskutiert Sampling als Strategie zur Volumenreduktion (Albuquerque & Correia, 2025, S. 10). Die vorliegende Arbeit konkretisiert diesen Punkt, indem sie gezieltes Filtering auf Collector-Ebene einsetzt, das diagnostisch wertlose Spans verwirft, bevor sie gespeichert werden. Im Unterschied zum Sampling, das relevante Daten anteilig verlieren kann, erlaubt das Filtering eine inhaltlich begründete Selektion. Dieser Ansatz adressiert ein Problem, das Bissig (2024, S. 52) für reale Projekte unter dem Stichwort unerwarteten Verhaltens der Instrumentierung beschreiben, ohne eine Lösungsstrategie auf Collector-Ebene vorzuschlagen.

Der dritte Aspekt betrifft die Validierbarkeit von Alerting-Regeln. Vinnakota und Kolla (2025, S. 175–176) fordern ausdrücklich, Alarme vor dem produktiven Einsatz streng zu testen, etwa durch synthetisches Monitoring, Fehlerinjektion und kontrollierte Störungssimulationen in Staging-Umgebungen. Die Evaluation zeigt exemplarisch, dass diese Forderung in der Praxis auf erhebliche Reibung stoßen kann. Der Fehler-Alert ließ sich definieren, konfigurieren und auslösen; das eigentliche technische Problem lag jedoch darin, dass die Fehler-Metriken in kurzen Zeitfenstern keine konsistent aktuellen Werte lieferten. Die Literatur benennt die Notwendigkeit des Testens, beleuchtet jedoch die konkreten Schwierigkeiten, die sich aus dem Zusammenspiel von Metriksemantik, Query-Verhalten und Staging-Verifikation ergeben, nur begrenzt.

9.2 Übertragbarkeit auf ähnliche cloud-native Plattformen

Die Frage der Übertragbarkeit lässt sich in drei Stufen beantworten, die den Grad der Kontextabhängigkeit widerspiegeln.

Hoch übertragbar sind drei methodische Elemente, die unabhängig von der spezifischen Architektur von frag.jetzt funktionieren. Die *Signal-Matrix-Methodik* aus Kapitel 6.4, die für jeden Dienst systematisch die vier Golden Signals auf konkrete Messgrößen, Zwecke und Stakeholder-Rollen abbildet, ist als Strukturierungswerkzeug auf jedes verteilte System anwendbar. Sie setzt lediglich voraus, dass die beteiligten Dienste und ihre kritischen Interaktionen bekannt sind. Die *Fünf-Felder-Runbook-Struktur* (Symptom, Kontext, Diagnose, Behebung, Eskalation) ist ebenfalls stackunabhängig und lässt sich auf beliebige Alerting-Systeme anwenden. Die *symptomorientierte Alert-Logik*, die nutzerwirksame Zustände als Auslöser verwendet und interne Ursachenindikatoren als Debugging-Kontext reserviert, folgt einem Prinzip, das in der SRE-Literatur als stackübergreifend gültig beschrieben wird (Ewaschuk, 2017, Kap. *Symptoms Versus Causes*).

Bedingt übertragbar ist der OTel-Agent-basierte Instrumentierungsansatz. Der OTel Java Agent funktioniert besonders reibungsarm in JVM-basierten Umgebungen. Für Anwendungen in anderen Laufzeitumgebungen, etwa Python oder Node.js, kann der Aufwand höher ausfallen und zusätzliche manuelle Anpassungen erfordern (Bissig, 2024, S. 52). Das Grundprinzip, die Instrumentierung über einen herstellerneutralen Standard zu entkoppeln und dadurch die Bindung an ein bestimmtes Backend zu vermeiden, bleibt jedoch unabhängig von der Laufzeitumgebung gültig. Faseeha et al. (2025, S. 72034–72035) bestätigen, dass die Interoperabilität zwischen heterogenen Plattformen eine der zentralen offenen Herausforderungen darstellt, die durch standardisierte Telemetrieformate wie OpenTelemetry adressiert wird.

Kontextabhängig sind die konkreten Implementierungsentscheidungen. Der LGTM-Stack hat sich für das Single-Host-Szenario von frag.jetzt als geeignet erwiesen, stellt aber keine universelle Empfehlung dar. Für Organisationen mit Kubernetes-nativer Infrastruktur und höherem Telemetrevolumen kommen andere Stack-Kombinationen oder skalierbare Varianten infrage (Gudimetla, 2025, S. 498). Die Docker-Compose-Topologie und die konkreten Schwellenwerte der Alerting-Regeln sind systemspezifisch. Schwellenwerte müssen aus den jeweiligen Lastprofilen und nichtfunktionalen Anforderungen des Zielsystems abgeleitet werden, wie bereits in Kapitel 4.2 als methodischer Grundsatz formuliert.

Zusammengefasst sind die methodischen Konzepte, also das *Wie* der Ableitung, die strukturierende Matrix und die Alert-Logik, übertragbar. Die konkreten technischen Entscheidungen, also das *Womit* und die spezifischen Grenzwerte, bleiben kontextgebunden.

9.3 Wissenschaftlicher und praktischer Beitrag

Die Arbeit entwickelt kein neues theoretisches Modell für Observability. Ihr Beitrag liegt in der methodisch fundierten Anwendung, Integration und Operationalisierung bestehender Konzepte für einen konkreten Anwendungskontext.

Der *wissenschaftliche Beitrag* besteht in der methodisch transparenten Ableitungslogik, die von der Systemanalyse über die Anforderungsdefinition und die servicebezogene Operationalisierung der Golden Signals bis zur kriteriengestützten Evaluation reicht. Dieses Vorgehen verbindet mehrere in der Literatur häufig getrennt behandelte Konzepte zu einem integrierten Strategieentwurf. Dazu gehören die Golden Signals als Strukturierungsheuristik, das Design komplementärer Metrikebenen, die symptomorientierte Alarmierung und die rollenbasierte Informationsaufbereitung. Die explizite Rückbindung an Prüffragen und Erfüllungsgrade in der Evaluation stellt sicher, dass die Ergebnisse nachvollziehbar und an den Anforderungen messbar bleiben. Darüber hinaus liefert die Arbeit fallbezogene operative Hinweise auf Problemkategorien, etwa Ressourcenkonflikte des Observability-Stacks mit der beobachteten Anwendung, die in der vorwiegend konzeptionell argumentierenden Literatur bislang wenig Raum einnehmen.

Der *praktische Beitrag* umfasst eine funktionsfähige Observability-Infrastruktur für `frag.jetzt`, die aus Instrumentierungskonfigurationen, Dashboard-Definitionen, Alerting-Regeln und Runbooks besteht. Die dokumentierten Herausforderungen bei Netzwerktopologie, Ressourcendimensionierung und Signalfilterung sind für Teams, die Observability erstmals in einer Docker-Compose-basierten Umgebung einführen, unmittelbar verwertbar. Die Signal-Matrix und die Runbook-Struktur stehen als wiederverwendbare Artefakte zur Verfügung, die auch ohne den spezifischen Technologie-Stack adaptiert werden können.

Beide Beitragsebenen stehen in einem komplementären Verhältnis: Die methodische Ableitung gibt dem Prototyp seine wissenschaftliche Nachvollziehbarkeit, und der Prototyp gibt den methodischen Konzepten ihre praktische Überprüfbarkeit.

Literaturverzeichnis

- Albuquerque, C., & Correia, F. F. (2025). Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications. *arXiv preprint arXiv:2510.02991*. <https://doi.org/10.48550/arXiv.2510.02991>
- Banerjee, A., & Singh, R. (2024). A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability. *International Journal of Computer Applications*. <https://doi.org/10.21275/SR25917095133>
- Bissig, N. (2024). *Unified Application Observability in Heterogeneous Distributed Systems* [Diss., Hochschule für angewandte Wissenschaften München]. https://opus4.kobv.de/opus4-hm/frontdoor/deliver/index/docId/590/file/Bissig_2024_MA.pdf
- Borges, M. C., Bauer, J., Werner, S., Gebauer, M., & Tai, S. (2024). Informed and assessable observability design decisions in cloud-native microservice applications. *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, 69–78. <https://doi.org/10.1109/ICSA59870.2024.00015>
- Dasari, H. (2025). SITE RELIABILITY ENGINEERING PRACTICES FOR ERROR BUDGET MANAGEMENT IN LARGE-SCALE SYSTEMS. *International Journal of Applied Mathematics*, 38(5s), 991–1001. <https://doi.org/10.12732/ijam.v38i5s.366>
- Elastic. (n. d.). *Use OpenTelemetry with Elastic APM*. Elastic Docs. (besucht am 27. März 2026) <https://www.elastic.co/docs/solutions/observability/apm/opentelemetry>
- Elastic. (2024). *FAQ on Software Licensing*. Elastic. (besucht am 27. März 2026) <https://www.elastic.co/pricing/faq/licensing>
- Ewaschuk, R. (2017). Monitoring Distributed Systems [Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google].
- Faseeha, U., Syed, H. J., Samad, F., Zehra, S., & Ahmed, H. (2025). Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms. *IEEE Access*, 13, 72011–72035. <https://doi.org/10.1109/ACCESS.2025.3562125>
- Fischer, S., Urbanke, P., Ramler, R., Steidl, M., & Felderer, M. (2024). An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study. <https://doi.org/10.1145/3644032.3644445>
- Garimilla, M. (2024). Designing Tomorrow’s Observability: A Software Architect’s Guide to Building Effective Monitoring Solutions’. *International Journal for Research in Applied Science and Engineering Technology*, 12, 155–63. <https://doi.org/10.22214/ijraset.2024.64151>
- Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S., Malavolta, I., Islam, T., Dinga, M., Koziolk, A., Singh, S., Armbruster, M., et al. (2024). Monitoring tools for DevOps and microservices: A systematic grey literature review. *Journal of Systems and Software*, 208, 111906. <https://doi.org/10.1016/j.jss.2023.111906>

- Gowda, H. G., & Hameed, A. (2022). From alerts to automation: Building self-healing SRE frameworks with runbooks and intelligent triggers. *International Journal of Science, Engineering and Technology*, 10(6), 1. https://www.researchgate.net/profile/Abubakr-Hameed/publication/393712529_From_Alerts_to_Automation_Building_Self-Healing_SRE_Frameworks_with_Runbooks_and_Intelligent_Triggers/
- Grafana Labs. (n. d. a). *Get started with Grafana Mimir*. Grafana Mimir documentation. (besucht am 27. März 2026) <https://grafana.com/docs/mimir/latest/get-started/>
- Grafana Labs. (n. d. b). *Introduction to Grafana Tempo*. Grafana Tempo documentation. (besucht am 27. März 2026) <https://grafana.com/docs/tempo/latest/set-up-for-tracing/>
- Grafana Labs. (n. d. c). *Understand labels*. Grafana Loki documentation. (besucht am 27. März 2026) <https://grafana.com/docs/loki/latest/get-started/labels/>
- Gudimetla, S. (2025). Transitioning to Open-Source Observability: A Cost-Benefit Analysis and Migration Framework. *Journal of Computer Science and Technology Studies*, 7(12), 495–512. <https://doi.org/10.32996/jcsts.2025.7.12.55>
- Hallur, J. (2024). From Monitoring to Observability: Enhancing System Reliability and Team Productivity. *International Journal of Science and Research (IJSR)*, 13(10), 602–606. <https://doi.org/10.21275/SR241004083612>
- Jalalvand, F., Baruwat Chhetri, M., Nepal, S., & Paris, C. (2024). Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2), 1–36. <https://doi.org/10.1145/3695462>
- Kosińska, J., Baliś, B., Konieczny, M., Malawski, M., & Zieliński, S. (2023). Toward the observability of cloud-native applications: The overview of the state-of-the-art. *IEEE Access*, 11, 73036–73052. <https://doi.org/10.1109/ACCESS.2023.3281860>
- Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J., & Liu, X. (2022). Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empirical Software Engineering*, 27(1), 25. <https://doi.org/10.1007/s10664-021-10063-9>
- Niedermaier, S., Koetter, F., Freymann, A., & Wagner, S. (2019). On Observability and Monitoring of Distributed Systems – An Industry Interview Study. In *Service-Oriented Computing* (S. 36–52). Springer International Publishing. https://doi.org/10.1007/978-3-030-33702-5_3
- Sakinala, K. (2025). Monitoring and Observability for Cloud-Native Applications. *Journal of Computer Science and Technology Studies*, 7(8), 101–115. <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>
- Soldani, J., & Brogi, A. (2022). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Comput. Surv.*, 55(3). <https://doi.org/10.1145/3501297>
- Sundström, J. (2025). Finding Faults Faster: Improving Error Tracing in Reactive Monitored Systems with Distributed Tracing Tools. (besucht am 27. März 2026) <https://www.diva-portal.org/smash/record.jsf?dswid=1070&pid=diva2%3A1964604>

- TH Mittelhessen & arsnova. (2025). *frag.jetzt – Open-Source Q&A-Plattform*. (besucht am 27. März 2026) <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md>
- Tzanettis, I., Androna, C.-M., Zafeiropoulos, A., Fotopoulou, E., & Papavassiliou, S. (2022). Data fusion of observability signals for assisting orchestration of distributed applications. *Sensors*, 22(5), 2061. <https://doi.org/10.3390/s22052061>
- Usman, M., Ferlin, S., Brunstrom, A., & Taheri, J. (2022). A survey on observability of distributed edge & container-based microservices. *IEEE Access*, 10, 86904–86919. <https://doi.org/10.1109/ACCESS.2022.3193102>
- Vinnakota, K., & Kolla, M. (2025). Creating Effective Alerts for Monitoring Distributed Systems. *International Journal of Computer Trends and Technology*, 73, 172–178. <https://doi.org/10.14445/22312803/IJCTT-V73I5P122>

Index

Four Golden Signals, 1
frag.jetzt, 1

Observability, 1

A Verzeichnis der KI-Nutzung

Erklärung zur Nutzung KI-basierter Werkzeuge

Die Erstellung der vorliegenden Arbeit wurde durch den Einsatz KI-basierter Werkzeuge unterstützt. Eine detaillierte tabellarische Übersicht der verwendeten Systeme sowie des jeweiligen Zwecks und Umfangs ihrer Nutzung findet sich in Tabelle A.1.

Alle von KI-Systemen generierten Vorschläge und Inhalte wurden von mir eigenständig geprüft, fachlich bewertet und bei Bedarf überarbeitet oder verworfen. Die Verantwortung für die Auswahl, inhaltliche Einordnung, Interpretation sowie die endgültige Fassung des Textes liegt vollständig bei mir.

Grundlage für den Umgang mit den Werkzeugen bildet die Richtlinie für die Nutzung von KI im Studium der IU Internationale Hochschule.

Declaration on the Use of AI-based Tools

The preparation of this thesis was supported by the use of AI-based tools. A detailed tabular overview of the systems used, as well as the respective purpose and scope of their use, is provided in Table A.1.

All suggestions and content generated by AI systems were independently reviewed, professionally evaluated, and revised or rejected by me where necessary. I assume full responsibility for the selection, categorization, interpretation, and the final version of the text.

The handling of these tools follows the current recommendations of the Guidelines for the Use of AI in Academic Studies at IU International University.

Tabelle A.1 listet alle in dieser Arbeit verwendeten KI-Werkzeuge auf.

A.1 Copilot Prompts für die unterstützende Systemanalyse von frag.jetzt

- 1 Analysiere die Projektstruktur von frag.jetzt. Welche Module, Services und Hauptkomponenten gibt es? Erstelle eine tabellarische Uebersicht mit Modulname, Technologie/Framework und Hauptverantwortung.
- 2 Identifiziere alle externen HTTP-Aufrufe (REST-Clients, WebClients, Feign-Clients) im Spring-Boot-Backend. Welche externen Dienste werden aufgerufen, mit welchen Endpunkten, und gibt es Timeout- oder Retry-Konfigurationen?
- 3 Liste alle REST-Controller im Backend auf. Gruppiere sie nach fachlicher Funktion (z. B. Room-Management, Question-Management, Auth, AI-Service) und notiere die HTTP-Methoden und Pfade.
- 4 Gibt es bereits Observability-bezogene Konfigurationen? Suche nach: Micrometer, Actuator, OpenTelemetry, Prometheus, Logging-Frameworks (Logback, Log4j), Health-Check-Endpoints, Tracing-Bibliotheken.
- 5 Analysiere die Datenbankschicht. Welche Entities/Tabellen gibt es, welche Repositories werden verwendet, und gibt es komplexe Queries oder Stored Procedures, die als Performancerisiko gelten koennten?
- 6 Beschreibe die WebSocket-Kommunikation. Welche STOMP-Topics oder Channels gibt es, welche Nachrichten werden darueber ausgetauscht, und wie wird die Verbindung verwaltet?
- 7 Analysiere die Deployment-Konfiguration (docker-compose.yml, Kubernetes-Manifeste, CI/CD-Pipeline). Welche Container/Services werden deployt, wie sind sie vernetzt, und welche Ressourcenlimits sind konfiguriert?

Listing A.1: Copilot-Prompts zur Architekturanalyse

Tabelle A.1: Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge. Quelle: Eigene Darstellung.				
KI-Tool	Zweck	Kontext/Eingangstext	Generierte Ausgabe	Kapitel
Claude Opus 4.6	Formatierung und Formulierungsverbesserung der Runbooks	Strukturierung des methodischen Ansatzes	Vorschlag zur Gliederung der Methodik	Kap. 3.1
Claude Opus 4.6	Code-Refactoring	Optimierung bestehender Algorithmen	Vorschläge zur Effizienzverbesserung	Kap. 4.3
GitHub Copilot	Code-Vervollständigung	Hilfe bei Instrumentierung und Deployment der Observability-Komponenten	Vorschläge für Funktionsimplementierung	Kap. 4.2
Gemini 3 Pro	Literaturrecherche	Zusammenfassung aktueller Forschungsarbeiten	Überblick über Stand der Technik	Kap. 2.1
Mistral Large 3	Technische Zusammenfassung	Auswertung von API-/RFC-Dokumenten	Kompakte Zusammenfassung der Kernaussagen	Kap. 2.4
OpenAI GPT-5.2	Formulierungsverbesserungen	Überarbeitung der Einleitung	Alternative Formulierungen zur Problemstellung	Kap. 1.2
OpenAI GPT-5.3	Hilfe bei Dashboard Design	Vorschlag zu möglichen Tool-Stacks	Stichpunkt-Zusammenfassung mit Quellenhinweisen	Kap. 2.3

Bei KI-Tools genau sagen, welches Modell / Version oder ähnliches — frag KI wie man die KI referenziert.

A.2 C4-Container-Diagramm in Vollansicht

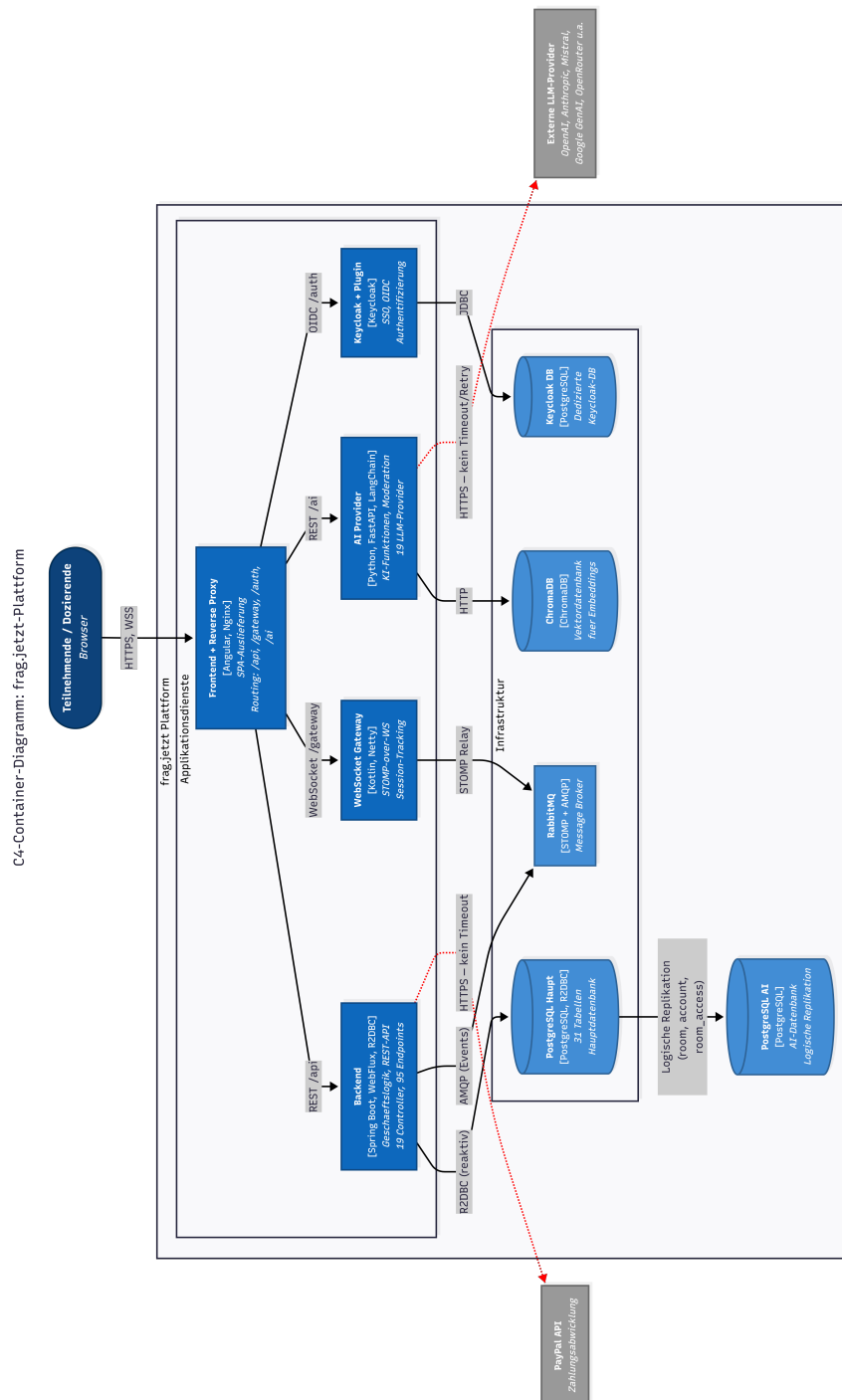


Abbildung A.1: C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht. Quelle: Eigene Darstellung.

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben.

Frankfurt am Main, den 27. März 2026

Leon Rausch